Vibe Coding Professional Mobile Apps

Vibe Coding Professional Mobile Apps

Build Food Delivery, Shopping, Social and 10-Minute Grocery Apps for Android and iOS with AI, from First Prompt to Store Release, Plus a TV Streaming App Bonus

Aadhavan Muthurengan

Vibe Coding Professional Mobile Apps
Build Food Delivery, Shopping, Social and 10-Minute Grocery Apps for Android and iOS with AI, from First Prompt to Store Release, Plus a TV Streaming App Bonus

Copyright © 2026 Aadhavan Muthurengan. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author, except for brief quotations in reviews and certain other noncommercial uses permitted by copyright law. Readers may freely use and adapt the example prompts and project code in this book for their own work. All the code is available at github.com/aadhavanshines/vibe-coding-professional-mobile-apps under the MIT License.

Disclaimer. This book is provided for educational and informational purposes only. AI products, features, and policies change frequently; verify current details with each provider. The author makes no guarantees about results and is not liable for any losses arising from the use of this information. Nothing in this book is legal, medical, financial, or other professional advice.

Trademarks. Claude, Claude Code, and Anthropic are trademarks of Anthropic. Apple, iPhone, iPad, iOS, iPadOS, tvOS, Apple TV, App Store, App Store Connect, TestFlight, Xcode, VoiceOver, Apple Pay, Keychain and Sign in with Apple are trademarks of Apple Inc. Google, Android, Android TV, Google Play, Google Pay, Android Studio, Chrome, Firebase, TalkBack and Crashlytics are trademarks of Google LLC. Expo and EAS are trademarks of 650 Industries, Inc. React and React Native are trademarks of Meta Platforms, Inc. Microsoft and Fluent are trademarks of Microsoft Corporation. Razorpay is a trademark of Razorpay Software Private Limited. Stripe is a trademark of Stripe, Inc. UPI is a trademark of National Payments Corporation of India. Maestro is a trademark of mobile.dev, Inc. Sentry, Bugsnag, Amplitude, Mixpanel, PostHog, LaunchDarkly, Statsig, GitHub, Node.js, SQLite, Hono, Zod, TanStack, Jest, Vitest, Playwright and all other product names are trademarks of their

respective owners. Swiggy, Zomato, Amazon, Flipkart, Instagram, Netflix and other company names are mentioned only as familiar examples of app quality; this book's apps are original, with their own names, designs and fictional data, and are not copies of any company's app. Product names are used solely for identification and educational discussion. This book is independent and is not affiliated with, sponsored by, or endorsed by Anthropic or any of these companies.

Examples. The people, habits, expenses, and other data in the example projects are fictional. Any resemblance to real organizations or persons is coincidental.

First edition, 2026.

To my father and my mother,
for your love, your sacrifices, and every lesson you taught me.

To my wife,
for your patience, your encouragement, and your belief in me.

And to our three children,
may you always build the things you dream of.

This book is for you.

Contents

Introduction 11

Part I: Foundations 15

1. Professional Apps, Vibe Coded 17
2. Your Mobile Workshop 23
3. Directing Claude Code on a Mobile Project 29
4. React Native and Expo in One Chapter 37
5. A Design System Like the Big Apps 45
6. One Backend for Four Apps 55

Part II: Platter, Food Delivery 65

7. Planning a Food Delivery App 67
8. Home, Discovery and Restaurant Pages 73
9. Cart, Customization and Checkout 81
10. Live Order Tracking 87

Part III: Cartly, E-Commerce 93

11. Planning a Store App 95
12. Home, Search, Filters and Product Pages 101
13. Cart, Payments and Orders 109
14. Returns, Wishlist and Notifications 113

Part IV: Lumo, Social Photos 119

15. Planning a Social App 121
16. Feed, Stories and the Double-Tap Like 127
17. Create, Explore, Profile and Messages 133

Part V: Minutely, 10-Minute Grocery 139

18. Speed as a Feature: Planning Quick Commerce 141
19. Catalog, Cart and the Countdown Delivery 147

Part VI: Production 155

20. Performance: Lists, Images, Startup and Smooth Animation 157
21. Accessibility and Localization 163
22. Testing: Unit, Component, API and End to End 169
23. Payments for Real: Razorpay, Stripe, Apple Pay, Google Pay and UPI 175
24. Push Notifications, Deep Links and Analytics 181
25. Security and Privacy 187
26. Builds and Releases With EAS 193
27. Shipping to Google Play 201
28. Shipping to the App Store 207
29. After Launch: Crashes, Reviews, Updates and Experiments 213

Part VII: Bonus 219

30. Reelhouse: Building for Android TV (and Apple TV) 221

Appendix A: The Prompt Library 241

Appendix B: Release Checklists 253

Appendix C: Troubleshooting 257

Appendix D: Glossary 261

Appendix E: Sources 265

About the Author 269

More Books by the Author 271

Introduction

Open the food delivery app on your phone and order dinner. In under a minute you search a city's restaurants, customize a dish, apply a coupon, pay, and watch a rider move toward your door on a live map. Open a shopping app and you see a deal counting down, a product page with a gallery, variants and reviews, and a delivery date for your exact pincode. Open a social app and the stories, the double-tap heart and the feed all feel effortless.

None of that is effortless to build. Behind every polished screen are dozens of decisions about layout, loading states, errors, accessibility, animation, data and trust. Teams of engineers and designers spend years on them. This book shows you how to build apps of that standard yourself, by directing an AI coding agent, Claude Code, the way a lead engineer directs a team.

What You'll Build

Four complete Android and iOS apps, one shared design system, one backend, and a bonus Android TV app:

App	Category	Highlights
Platter	Food delivery	Restaurants and menus, customization sheets, coupons, tips, live order tracking with a moving rider
Cartly	E-commerce	Deals with countdowns, search, filters and sort, product galleries, variants, delivery estimates, orders and returns
Lumo	Social photos	Stories, a feed with double-tap likes, comments, create with filters, explore, profiles, messages

App	Category	Highlights
Minutely	10-minute grocery	Category rails, instant ADD steppers, savings, a countdown-ring tracking screen
Reelhouse (bonus)	Android TV streaming	Profiles, a hero billboard, Top 10 and live sports rows, seasons and episodes, a player, a TV guide, all driven by a remote

All five are original. Their names, brands, restaurants, products, people, shows and teams are fictional, and they don't copy any real company's design. They are built to the standard you know from the biggest apps in each category, and the book points to those apps by name only to explain the patterns they made familiar.



The four phone apps: Platter, Cartly, Lumo and Minutely. Every screen in this book is a real capture of the apps you'll build.

How This Book Was Made

Every app was built by Claude Code, for real. The prompts printed in the project chapters were run in Claude Code while this book was written, one recorded session per milestone. Each chapter tells you how long the session ran, what it cost, what Claude reported, and what went wrong. Two sessions were cut off, one by a time limit and one by a usage limit, and you'll see how they were finished.

Every screen is a real capture. The phone images are screenshots of the apps running on a 393-by-852-point phone screen at three times the pixel

density, and the TV images are full 1920-by-1080 captures with the remote's focus visible. The phones and TVs around them are drawn frames.

Everything was checked. By the end, the workspace held more than 300 automated tests across the four apps, the design system, the API client and the server, plus 80 for the TV app. Typecheck, lint and tests had to pass before any app was called done.

> **Note:** The screenshots were captured from the apps' web builds, rendered by React Native for Web in a browser, because the cloud computer this book was written on can't run phone emulators. The same code runs on Android and iOS; Chapter 2 shows you how to run it on your own phone and emulator, and Chapter 22 how to test it there.

Who This Book Is For

- **Builders with app ideas** who want to ship something that looks and behaves like a top-chart app, not a demo.
- **Developers new to mobile** who know some JavaScript or TypeScript and want a fast, modern route to Android and iOS.
- **Experienced engineers** who want to see how far an AI agent can go on a real, multi-app codebase, and how to keep it on track.
- **Designers and product managers** who want to turn their designs and specs into working apps.

Some programming experience helps, but you don't need to be an expert. You need to be able to read code well enough to review it, run commands in a terminal, and judge a screen with a critical eye. The book teaches the rest.

How the Book Is Organized

Part I, Foundations, sets up your tools, shows you how to direct Claude Code on a mobile project, covers just enough React Native and Expo to review what Claude writes, and builds the shared design system and backend.

Parts II to V build the four apps, one part each: the plan, the prompt, the session, the screens, and the code worth understanding.

Part VI, Production, covers what separates a demo from a product: performance, accessibility, testing, real payments, notifications, security, builds, and shipping to Google Play and the App Store.

Part VII, the bonus, builds Reelhouse for Android TV.

The appendices collect every prompt in the book, release checklists for Android, iOS and Android TV, troubleshooting, a glossary and the sources for every fact.

Get the Code

All the code is in the book's repository: the four apps, the design system, the API client, the server, the TV app, and the prompts and records of every Claude Code session. Chapter 2 shows how to download and run it. You can follow along by giving Claude Code the same prompts in an empty folder, or start from the finished code and change it.

> **Note:** Claude's replies vary from run to run, even with the same prompt. Your code will differ from the code printed here, and that's expected. What carries over is the method: a clear spec, small sessions, real checks, and a critical look at every screen.

PART I

Foundations

CHAPTER 1 · FOUNDATIONS

Professional Apps, Vibe Coded

Anyone can get an AI to produce an app that works in a demo. The question this book answers is how to get apps that hold up the way the best consumer apps do: on a slow network, on a small phone, with a screen reader on, after the hundredth change. That requires knowing what "professional" means in concrete terms, and then turning that knowledge into specs, prompts and checks that an AI coding agent can follow.

What Makes an App Feel Professional

When people say a food delivery or shopping app feels polished, they're reacting to a long list of details, most of which they never notice consciously. Here are the ones that matter most, and that you'll see in every app in this book.

Every state is designed. A screen isn't one picture; it's at least five: loading, empty, error, partial and full. Professional apps show skeleton placeholders shaped like the content while it loads, a friendly message and an action when there's nothing to show, and a clear way to retry when something fails. Amateur apps show a spinner, a blank screen, or a crash.

Density without clutter. A restaurant card in a top delivery app packs an image, a name, a rating, a delivery time, a price for two, a distance and an offer into a few square centimeters, and it still reads calmly. That comes from

a strict type scale, consistent spacing, and a small set of colors used with intent.

The next action is always one tap away. The cart bar that appears the moment you add an item. The "Buy now" button that stays pinned while you scroll. The "Track order" button on an active order. Professional apps never make you hunt for the thing you're about to do.

Instant feedback. Buttons spring slightly when pressed. A quantity stepper changes the moment you tap, before the server answers, and quietly corrects itself if the server disagrees. A heart bursts when you double-tap a photo. Speed is a feeling as much as a measurement.

Trust signals. A bill that lists every fee, with waived fees struck through. A delivery date for your exact pincode. "Inclusive of all taxes." A live status that keeps updating. Users hand these apps their money and their address; every screen should earn it.

Accessibility and inclusion. Text that grows with the system font size, labels for screen readers on every button, touch targets at least 44 points square, and motion that calms down when the user asks for reduced motion.

Consistency. The same button, sheet, price and rating look identical everywhere, because they come from one design system. Users learn an app once.

The Five Apps

The four phone apps cover the four biggest categories of consumer apps, and each one teaches a different side of mobile development.

Platter (food delivery) teaches discovery, complex menus with customization rules, a bill with many fees, and real-time tracking over a live connection to the server.

Cartly (e-commerce) teaches information-dense commerce screens, search with suggestions, filter and sort sheets, product galleries, variants, and the long tail of orders, shipments and returns.

Lumo (social photos) teaches content-first design, gestures and animation: stories with progress bars, tap and hold, double-tap likes, a create flow with filters, and messaging.

Minutely (10-minute grocery) teaches speed as a product feature: optimistic updates everywhere, a category rail, savings called out on every screen, and a countdown tracking screen.

Reelhouse (Android TV) is the bonus. It teaches a completely different interaction model, the 10-foot screen and the D-pad remote, where focus is everything.

The four phone apps share one design system, one API client and one backend. That's how real companies build families of apps, and it's also how you keep an AI agent consistent: when there's exactly one `Price` component, every price in every app looks right.

Vibe Coding at a Professional Standard

Vibe coding means building software by describing what you want to an AI and steering by the results. At its loosest, you type a wish, accept whatever comes back and keep going. That's fine for a weekend toy. For apps people will pay through, it fails in predictable ways: inconsistent screens, untested logic, hard-coded data, missing states, and code nobody understands.

The method in this book keeps the speed and fixes the failures. It has five habits:

1. **Write the spec before the prompt.** Each app starts with a product requirements document (PRD) and a design document that Claude reads before writing a line of code. Chapter 3 shows them.
2. **Give the project rules.** A `CLAUDE.md` file at the root of the repository tells Claude the layout, the commands and the non-negotiables, such as "never hard-code a color" and "every touchable has an accessibility label."
3. **Work in sessions sized to one milestone.** Foundation, then each app, then fixes. A session that tries to do everything runs out of time or context.
4. **Make Claude check its own work, visually and with tests.** Every session ends with typecheck, lint and tests passing, and Claude captures

every screen in a headless browser at phone size and fixes what looks wrong.

5. **Review like a lead.** You read the summary, look at every screenshot, run the checks yourself, and send back what isn't right. In this book, that review caught a card sliding under the TV app's side menu, placeholder text in its TV guide, and a test that failed only when everything ran at once.

How Much It Took

Here is what the recorded sessions behind this book cost and took, as reported by Claude Code. Costs are API list-price estimates; on a subscription plan, the same work counts against your usage limits instead.

Session	What it built	Minutes	Cost
1	Workspace, design system, API client, server	64	$20.30
2	Platter, food delivery	63	$10.17
3	Cartly, e-commerce	about 120	not recorded
3b	Finishing Cartly	32	$4.24
4	Lumo, social photos	32	$7.57
5	A flaky test, and Minutely, grocery	not comparable	$9.08
6	Reelhouse, Android TV	48	$10.89
6b	Two fixes from review	8	$1.83

Session 3 ran into the two-hour limit of the tool that launched it and was stopped before it could report its cost; session 3b was then cut off by Claude Code's five-hour usage limit, after it had finished the work. Session 5's timing figure covers only its final stretch. Those stories, and how the work was finished, are in the chapters for each app.

The total recorded cost was $64.06, plus the unrecorded Cartly session. Compare that with the cost of a team building four apps and a TV app, and you'll see why this way of working is changing how software gets made.

Compare it with the cost of shipping a broken app, and you'll see why the rest of this book is about doing it carefully.

Key Takeaways

- "Professional" is concrete: designed states, calm density, one-tap next actions, instant feedback, trust signals, accessibility and consistency.
- The four apps cover food delivery, e-commerce, social and quick commerce, and share one design system and backend; the bonus app targets Android TV.
- Professional vibe coding is a method: spec first, project rules, milestone-sized sessions, self-checks with screenshots and tests, and your review.

CHAPTER 2 · FOUNDATIONS

Your Mobile Workshop

This chapter sets up everything you need: Node.js, Claude Code, the book's code, the backend, and three ways to see the apps running, in a browser, on an Android emulator or iPhone simulator, and on your own phone.

What You Need

Tool	Why	Notes
A computer	Everything	macOS, Windows or Linux. Building for iPhone locally needs a Mac; Chapter 26 shows how to build in the cloud without one
Node.js LTS	Runs Expo, the tooling and the server	The workspace requires Node 20.19 or newer
Git	Version control, downloading the code	
Claude Code	Your AI engineer	A Claude subscription or an Anthropic API account
A code editor	Reading what Claude writes	Any editor; VS Code is a popular choice
Android Studio	Android emulator and Android TV emulator	Free, all three operating systems
Xcode	iPhone simulator and iOS builds	Mac only, from the Mac App Store
A phone	The real test	Android or iPhone

You don't need everything on day one. A computer, Node.js, Git and Claude Code are enough to build and run all four apps in a browser. Add Android Studio or Xcode when you want to see them on a device.

Install Claude Code

Claude Code runs in your terminal. Install it by following the current instructions in Anthropic's Claude Code documentation; it's also published on npm as `@anthropic-ai/claude-code`. Then start it in any folder and sign in:

```
$ claude
```

The first run walks you through signing in with your Claude account or an API key. This book was written with Claude Code version 2.1.316.

> **Tip:** Run `claude --version` now and then. Claude Code updates often, and new versions add features that make mobile work easier.

Get the Book's Code

```
$ git clone https://github.com/aadhavanshines/vibe-coding-profess
  ional-mobile-apps
$ cd vibe-coding-professional-mobile-apps
$ npm install
```

The repository is a **monorepo**: one repository holding several projects that share code. Its layout is the one Claude was given in `CLAUDE.md`:

Folder	What's in it
`apps/platter`	Food delivery app
`apps/cartly`	E-commerce app
`apps/lumo`	Social photo app
`apps/minutely`	10-minute grocery app
`packages/ui`	The design system: tokens, components, the 3D illustrations

Folder	What's in it
`packages/api`	The typed API client and data hooks
`server`	The backend all four apps use
`tv`	Reelhouse, the Android TV app, with its own dependencies
`docs`	The product and design specs Claude worked from
`sessions`	The prompt and result of every Claude Code session in this book

`npm install` at the root installs all four apps and the shared packages together, using **npm workspaces**. The TV app is separate because it needs a different build of React Native (Chapter 30 explains why); install it with `npm install` inside `tv/` when you get there.

Start the Backend

All four phone apps talk to one local server. Start it in its own terminal and leave it running:

```
$ npm run server
```

The first start creates a SQLite database file and fills it with the fictional city of Northbay: 24 restaurants with 870 menu items, 60 products with 416 reviews, 20 people with 112 posts and 40 stories, and 120 grocery items. To sign in to any app, use the demo phone number 9876543210 and the one-time code 123456; the server prints codes to its terminal instead of sending text messages.

See an App in Your Browser

The quickest way to see an app is the browser:

```
$ npm run web -w apps/platter
```

Expo starts a development server and opens the app in a browser tab. Use the browser's device toolbar (in Chrome, press F12 and choose a phone size) so

the app gets a phone-shaped screen. Replace `platter` with `cartly`, `lumo` or `minutely` for the other apps.

The browser is a convenience, not the real target. It runs your React Native code through React Native for Web, which is how this book's screenshots were made, but phones differ in gestures, fonts, keyboards, safe areas and performance. Check on a device before you trust anything.

Run on an Emulator or Simulator

Android: Install Android Studio, open the Device Manager, create a phone device with a recent system image, and start it. Then:

```
$ EXPO_PUBLIC_API_URL=http://10.0.2.2:4000 npm run android -w
  apps/platter
```

The address `10.0.2.2` is how the Android emulator reaches your computer; inside the emulator, `localhost` means the emulator itself.

iPhone (Mac only): Install Xcode, open it once to finish setup, then:

```
$ npm run ios -w apps/platter
```

The iPhone simulator shares your Mac's network, so `localhost` works.

Run on Your Own Phone

There are two ways to run your app on a real phone during development.

Expo Go is an app from the App Store and Google Play that can load your project over the network by scanning a QR code from `npx expo start`. It's the fastest way to start, but it only supports the newest Expo SDK and only the native libraries that ship inside Expo Go.

A development build is your own app, built once with all its native libraries, that then loads your JavaScript from your computer just like Expo Go does. It's what professional teams use, and it's what you need as soon as your app uses a native library that Expo Go doesn't include. Chapter 26 shows how to make one with EAS Build.

Either way, your phone needs to reach the server on your computer. Find your computer's address on the local network (for example `192.168.1.20`) and start the app with:

```
$ EXPO_PUBLIC_API_URL=http://192.168.1.20:4000 npx expo start
```

Check That Everything Works

Run the same checks Claude ran before calling each app done:

```
$ npm run check
```

This runs TypeScript's type checker, ESLint and every test in every package. On the finished code, it reports more than 300 passing tests across the apps, the design system, the API client and the server.

> **Try It:** Start the server and Platter in your browser, sign in with the demo number, add two dishes to the cart, place the order, and watch the tracking screen. The order goes from "Confirmed" to "Delivered" in about two minutes, because the server compresses a real delivery's timeline for demos.

Key Takeaways

- You can build and run all four apps with only Node.js, Git and Claude Code; add Android Studio and Xcode to see them on emulators and devices.
- The code is a monorepo with npm workspaces: four apps, two shared packages and a server, plus a separate TV app.
- The server seeds a fictional city of data; sign in with 9876543210 and code 123456.
- Point the apps at the server with `EXPO_PUBLIC_API_URL`: `10.0.2.2` on the Android emulator, your computer's network address on a real phone.

CHAPTER 3 · FOUNDATIONS

Directing Claude Code on a Mobile Project

Claude Code can write a whole app in one sitting. Whether that app is any good depends mostly on what you give it before it starts and what you check after it finishes. This chapter shows the documents, prompts and review loop behind the five apps, so you can reuse them on your own projects.

Three Documents Before the First Prompt

Every session in this book began with Claude reading three documents in the `docs` folder and the repository root.

`docs/PRD.md`**, the product requirements.** For each app it lists the screens and what each one must show and do, in the vocabulary of the category: "filters chips (Rating 4.0+, Fast delivery, Pure veg, Offers)", "veg/non-veg markers, bestseller tag, ADD buttons that turn into steppers", "a floating View cart bar with item count and total". The more precisely the PRD names the patterns users expect, the less Claude has to guess.

`docs/DESIGN.md`**, the design system.** Brand colors per app, the type scale, spacing, radii, elevation, the list of components, motion rules and icon set. It's short: a table and a few lists.

`CLAUDE.md`**, the project guide.** Claude Code reads this file automatically at the start of every session. It's where the project's layout, commands and rules live. Here it is as it stood at the end of the book; Claude itself added the demo login and environment details in session 1, which is exactly what you want from it:

```
# Pro Apps: project guide for Claude Code

A monorepo of four production-quality mobile apps for Android and iOS,
↪ built
with Expo SDK 58 (React Native 0.88, React 19.3, Expo Router), one shared
design system, one shared API client and one local backend. A fifth app,
Reelhouse TV, targets Android TV and lives in `tv/`, outside the
↪ workspace.
Read `docs/PRD.md` for what each app does, and `docs/DESIGN.md` for the
↪ look.

## Layout
- `apps/platter`  food delivery (Expo app)
- `apps/cartly`   e-commerce store (Expo app)
- `apps/lumo`     social photo sharing (Expo app)
- `apps/minutely` 10-minute grocery delivery (Expo app)
- `packages/ui`   design system: tokens, themed components, icons, art
- `packages/api`  typed API client, React Query hooks, shared types
- `server`        Node + TypeScript API (Hono, SQLite via better-sqlite3,
                  seed data, JWT auth, Server-Sent Events for live status)
- `tv`            Reelhouse TV (react-native-tvos, its own node_modules)

## Commands (run from the repository root)
- `npm install`                 install the workspace
- `npm run server`              start the API on http://localhost:4000
- `npm run typecheck`           TypeScript, every package
- `npm run lint`                ESLint, every package
- `npm test`                    Jest unit and component tests, every
↪ package
- `npm run web -w apps/platter` run one app in the browser (also
↪ `start`, `android`, `ios`)
- `npm run seed`                wipe and re-seed `server/data/pro.db`
↪ (also seeded on first start)
- `npm run check`               typecheck + lint + test in one go
- `npm run gen:art -w packages/ui` regenerate the typed art registry
↪ after editing `art/index.json`

Demo login: phone 9876543210, OTP 123456. Server env: `PORT`, `DB_PATH`,
`JWT_SECRET`, `SIM_SPEED` (2 = order simulation twice as fast). Apps read
`EXPO_PUBLIC_API_URL` (Android emulator: http://10.0.2.2:4000).
Tests: Jest (jest-expo) in packages, Vitest in server/.

## Rules
- TypeScript strict mode everywhere. No `any` without a comment saying
↪ why.
- Every screen works on Android, iOS and web (react-native-web): no
  native-only library without a web fallback. Maps are drawn with
  react-native-svg, not a map SDK, so they render everywhere.
- Use only components and tokens from `@pro/ui` for colors, type, spacing,
```

```
  radii and shadows. Never hard-code a color in an app.
- Images: the 3D illustrations in `packages/ui/art` (MIT, Microsoft Fluent
  Emoji 3D). No downloaded photos, no brand logos of real companies.
- All people, restaurants, products, shops and data are fictional. Use
  `.example` domains for any email or URL. Prices are in Indian rupees (₹)
  formatted with `Intl.NumberFormat('en-IN')`.
- Accessibility: every touchable has an `accessibilityRole` and label;
↪ text
  scales with the system font size; touch targets at least 44x44.
- Tests live next to the code as `*.test.ts(x)`. New logic needs a test.
- Before saying a task is done, run typecheck, lint and tests, and fix
  everything they report.
```

Notice what the rules do. "Never hard-code a color in an app" forces every screen through the design system, so all four apps stay consistent and dark mode works. "Every screen works on Android, iOS and web" rules out libraries that would break the screenshots and keeps the apps portable. "Before saying a task is done, run typecheck, lint and tests" turns Claude's "done" into something you can trust. Each rule exists because, without it, an AI agent will cheerfully take the shortcut.

> **Tip:** Write rules as checkable facts, not wishes. "Use the design system" is a wish. "Never hard-code a color; use tokens from `@pro/ui`" is a fact Claude can follow and you can check with a quick search.

Sessions Sized to One Milestone

The work was split into milestones, each a single Claude Code session:

1. The foundation: workspace, design system, backend and API client.
2. One session per app, in order of complexity: Platter, Cartly, Lumo, Minutely.
3. The TV app.
4. Short follow-up sessions for fixes found in review.

Why not one giant prompt? Because a session has a budget of time and context. Session 3, which built Cartly, ran for two hours and was stopped by the job runner before it finished; the next session had to complete its work. A milestone per session means each one ends with something whole, checked and committed.

Running a Recorded Session

You'll normally use Claude Code interactively: type a prompt, watch it work, answer its questions, approve its plan. For this book, every session was run non-interactively so its prompt and result could be saved exactly. This small script did it:

```
#!/bin/sh
# Run one recorded Claude Code session in the book's code folder.
# Usage: sessions/run.sh NAME PROMPT_FILE [model]
# Saves NAME.prompt.md (the prompt), NAME.json (the full result with
↪ cost).
set -e
here=$(cd "$(dirname "$0")" && pwd)
name=$1; prompt_file=$2; model=${3:-claude-opus-5-5}
cp "$prompt_file" "$here/$name.prompt.md"
cd "$here/../code"
claude -p --permission-mode auto --output-format json --model "$model" \
  "$(cat "$here/$name.prompt.md")" > "$here/$name.json" 2>
  ↪ "$here/$name.err"
python3 - "$here/$name.json" <<'PY'
import json, sys
d = json.load(open(sys.argv[1]))
print("turns:", d.get("num_turns"), "cost: $%.2f" %
↪ d.get("total_cost_usd", 0),
      "minutes: %.1f" % (d.get("duration_ms", 0) / 60000), "error:",
      ↪ d.get("is_error"))
print(d.get("result", "")[-3000:])
PY
```

`claude -p` runs one prompt and exits. `--permission-mode auto` lets Claude work without stopping for approval at each step, while still blocking risky actions; use it only in a project folder you've backed up with Git, and review the results. `--output-format json` returns the final reply along with the number of turns, the duration and the cost, which is where every cost figure in this book comes from.

Anatomy of a Good Session Prompt

Here is the prompt for session 1, which built the foundation for all four apps:

```
Read CLAUDE.md, docs/PRD.md and docs/DESIGN.md. This session sets up the
foundation that all four apps will share. Build it in this order, and
↪ check
each step before moving on:

1. The npm workspace: a root package.json with workspaces apps/*,
↪ packages/*
```

```
   and server; shared tsconfig (strict); ESLint (flat config, expo +
   typescript rules) and Jest configured so `npm run typecheck`, `npm run
   lint` and `npm test` run across every package. Use the exact versions
   Expo SDK 58 expects (expo ~58.0.22, react-native 0.88.1, react 19.3.0;
   check with `npx expo install --check` once an app exists).

2. packages/ui, the design system in docs/DESIGN.md: tokens (colors per
   brand with light and dark, type scale with Inter, spacing, radii,
   shadows), a ThemeProvider that takes a brand name, and every component
   listed in DESIGN.md, built with React Native primitives and
   ↪ Reanimated 4
   so they render on Android, iOS and web. An `Art` component and ArtTile
   that load the illustrations in packages/ui/art by name (typed from
   art/index.json). Component tests with @testing-library/react-native for
   the stateful ones (QuantityStepper, Price, CountdownTimer, Button).

3. server/: the Node + TypeScript API from the PRD, Hono on port 4000,
   better-sqlite3, a seed script with rich, believable fictional data for
   all four apps (at least 24 restaurants with full menus, 60 products
   ↪ with
   reviews, 20 users with posts, stories and comments, 120 grocery items
   ↪ in
   12 categories, coupons), OTP login (code 123456) returning a JWT,
   ↪ carts,
   orders, and order-status simulation pushed over Server-Sent Events.
   Tests for the routes with Vitest or Jest and supertest-style requests.

4. packages/api: typed fetch client for every route, React Query hooks,
   an SSE helper that works in React Native and on web, shared types.
   Base URL from EXPO_PUBLIC_API_URL, defaulting to http://localhost:4000.

Do not create the four apps yet; the next sessions do that. Finish by
running typecheck, lint and tests, fixing everything, and then write a
short summary: what you built, the commands, and anything you decided that
differs from the docs.
```

It follows a pattern you'll see in every prompt in this book:

- **Context first:** read the three documents.
- **An ordered list of deliverables**, each with a check: "Build it in this order, and check each step before moving on."
- **Concrete numbers** where quality depends on them: "at least 24 restaurants with full menus, 60 products with reviews."
- **Explicit boundaries:** "Do not create the four apps yet."
- **The definition of done:** typecheck, lint and tests pass.
- **A request for a report:** what was built, the commands, and anything that differs from the docs.

That last item matters more than it looks. Claude's report from session 1 included a section headed "Where I differ from the docs." It explained that

ESLint stayed on version 9 because Expo's config didn't support version 10 yet, that the server's order timeline was compressed so a delivery takes about two minutes, and that on an Android emulator the API address must be `10.0.2.2`. Those are exactly the decisions a lead engineer needs to hear about, and asking for them by name is how you get them.

Make Claude Look at Its Work

Tests catch logic bugs. They don't catch a search hint sitting underneath its icon, or chips stretching across the whole screen. From session 2 onward, every app prompt asked Claude to export the app for the web after each screen, open it in a headless browser at phone size, take a screenshot, and compare it with the spec. The sessions reported what that caught:

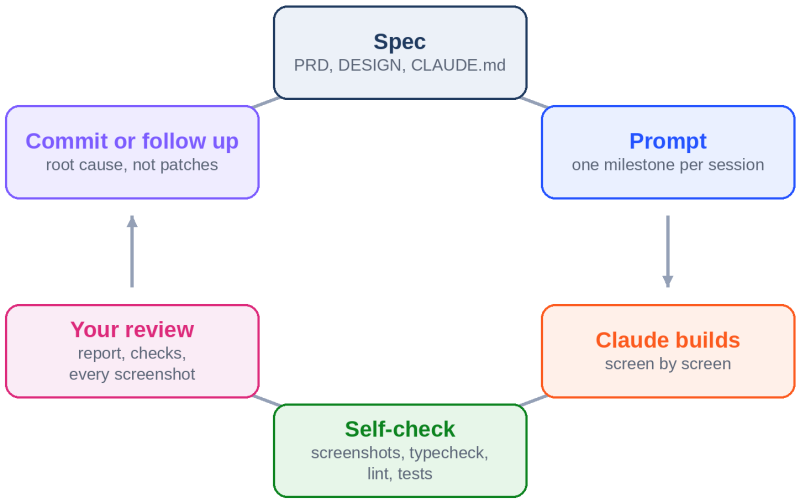
- Session 1, rendering the design system: "a search hint under its icon, chips stretching full width".
- Session 2, Platter: tab labels clipped by a short tab bar, and toasts covering the cart bar.
- Session 4, Lumo: a timestamp wrapping onto two lines, and a hard-edged dark band at the top of stories.
- Session 5, Minutely: a header wrapping onto three lines, a search hint reading `Search "Search "bread""`, and a Pay button reading "Pay ₹683UPI".

None of those would have failed a test. All of them would have made the apps look amateur.

Review Like a Lead

When a session finishes, don't just read "all tests pass" and move on. The review loop used throughout this book:

1. **Read the report,** especially anything Claude says it didn't do or did differently.
2. **Run the checks yourself:** `npm run check`. In this book, a check after session 4 caught one Cartly test failing only when the whole workspace ran in parallel; Claude had seen it pass alone.



The loop behind every app in this book: spec, prompt, build, self-check, your review, then commit or follow up.

3. **Look at every screenshot** at full size, as a picky user would.
4. **Commit** what's good, so the next session starts clean and any mistake is one command away from undone.
5. **Write a short follow-up prompt** for anything wrong, describing what you saw and where, and asking for the root cause rather than a patch.

The follow-up prompt for the flaky test is a good model:

```
A flaky test. `npm run check` failed once in
apps/cartly/src/test/screens/orders-returns.test.tsx ("Order
  details ›
tracks the shipment live and lets you cancel before it ships")
  while the
whole workspace's tests ran in parallel; run alone it passes.
  Find the real
cause (timing, fake timers, shared state, SSE) and fix it
  properly, without
skipping the test or just raising timeouts. Prove it with several
  runs of
the full `npm test` from the root.
```

"Without skipping the test or just raising timeouts" closes the two easy exits an agent might take. "Prove it with several runs" defines done. Claude found the cause, fixed it, ran the full suite four times, and on the way found and fixed a second problem: one test file had been hanging for over a minute because of a background timer left running.

Key Takeaways

- Give Claude a PRD, a design document and a `CLAUDE.md` before the first prompt; name the patterns users expect, and write rules as checkable facts.
- Size each session to one milestone, and end every session with checks passing and a commit.
- Good prompts give context, ordered deliverables with checks, concrete numbers, boundaries, a definition of done and a request for a report.
- Make Claude look at its screens in a browser at phone size; visual bugs don't fail tests.
- Review every session yourself: report, checks, screenshots, commit, follow-up.

CHAPTER 4 · FOUNDATIONS

React Native and Expo in One Chapter

You don't have to write the apps in this book by hand, but you do have to review them. This chapter gives you enough React Native and Expo to read what Claude writes, spot what's wrong, and ask for the right fix.

The Stack

Piece	What it does	Version in this book
React Native	Builds real native apps for Android and iOS from JavaScript or TypeScript, using native UI components	0.88
React	The component model: UI as functions of state	19.3
Expo	A toolkit and set of libraries around React Native: project setup, a large standard library (camera, notifications, images and more), builds and updates	SDK 58
Expo Router	File-based navigation: a file in `src/app` becomes a screen	58
TypeScript	JavaScript with types, so mistakes show up before you run the app	6.0
React Native for Web	Renders the same components in a browser	0.21

Expo SDK 58, released on 29 September 2026, pairs with React Native 0.88.1 and React 19.3.0. Each Expo SDK version is tied to one React Native version, so upgrading Expo upgrades React Native with it, and `npx expo install` picks the library versions that match your SDK.

Components and Styles

A React Native screen is a function that returns components. Instead of HTML's `div`, `span` and `img`, you use `View`, `Text` and `Image`, and the framework turns them into native views on Android and iOS and DOM elements on the web.

```
import { Text, View } from 'react-native';

export function Badge({ label }: { label: string }) {
  return (
    <View style={{ paddingHorizontal: 8, borderRadius: 999,
      backgroundColor: '#E3F6EE' }}>
      <Text style={{ fontSize: 12, fontWeight: '600', color:
        '#1BA672' }}>{label}</Text>
    </View>
  );
}
```

Styles look like CSS written as JavaScript objects, with camel-case names and numbers in **points** (density-independent units) instead of pixels. Layout uses Flexbox, with one important difference from the web: the default direction is a column, so children stack vertically unless you set `flexDirection: 'row'`.

In the apps, you'll rarely see raw colors or font sizes like the example above, because the project rule is to use the design system. Chapter 5 shows what that looks like instead.

Touch

There's no `button` element. Anything tappable is wrapped in a **Pressable**, which reports presses and lets you style the pressed state. Every touchable in the apps goes through one shared component, `PressableScale`, which springs to 97% of its size while pressed. You'll meet it in Chapter 5.

Gestures beyond a tap, such as swipes, drags and double taps, come from **React Native Gesture Handler**, and smooth animations from **Reanimated**, which runs animation logic on the UI thread so it stays fluid even when JavaScript is busy.

Lists

Mobile screens are mostly lists, and lists are where performance problems hide. A `ScrollView` renders all its children at once, which is fine for a short screen and terrible for 870 menu items. `FlatList` and Shopify's **FlashList** render only the rows near the screen and recycle them as you scroll. Chapter 20 covers when to use which.

Navigation with Expo Router

In Expo Router, the files under `src/app` are the app's screens, and the folders give them structure. Here is Platter's:

File	Screen
`src/app/_layout.tsx`	The root layout: providers and the main stack
`src/app/(tabs)/_layout.tsx`	The tab bar
`src/app/(tabs)/index.tsx`	Home tab
`src/app/(tabs)/search.tsx`	Search tab
`src/app/(tabs)/orders.tsx`	Orders tab
`src/app/(tabs)/profile.tsx`	Profile tab
`src/app/restaurant/[id].tsx`	A restaurant page; `[id]` is a parameter in the address
`src/app/cart.tsx`	Cart and checkout
`src/app/order/[id].tsx`	Order tracking

File	Screen
`src/app/rate/[id].tsx`	Rate an order, shown as a modal
`src/app/login.tsx`	Sign in

A folder name in parentheses, like `(tabs)`, groups screens without adding to the address. Square brackets mark a parameter, so `/restaurant/r-12` opens the restaurant page with `id` set to `r-12`.

The root layout wires everything together:

```
import { SessionProvider, useSession } from '@pro/api';
import { ThemeProvider, ToastProvider, useProFonts, useTheme } from
↪ '@pro/ui';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { Stack } from 'expo-router';
import * as SplashScreen from 'expo-splash-screen';
import { StatusBar } from 'expo-status-bar';
import { useEffect } from 'react';
import { GestureHandlerRootView } from 'react-native-gesture-handler';
import { SafeAreaProvider } from 'react-native-safe-area-context';

const TOKEN_KEY = 'platter.token';

void SplashScreen.preventAutoHideAsync().catch(() => undefined);

function RootStack() {
  const { colors, scheme } = useTheme();
  const { ready, signedIn } = useSession();
  const [fontsLoaded, fontError] = useProFonts();
  const loaded = ready && (fontsLoaded || !!fontError);

  useEffect(() => {
    if (loaded) void SplashScreen.hideAsync().catch(() => undefined);
  }, [loaded]);

  if (!loaded) return null;
  return (
    <>
      <StatusBar style={scheme === 'dark' ? 'light' : 'dark'} />
      <Stack screenOptions={{ headerShown: false, contentStyle: {
      ↪ backgroundColor: colors.background } }}>
        <Stack.Protected guard={signedIn}>
          <Stack.Screen name="(tabs)" />
          <Stack.Screen name="restaurant/[id]" />
          <Stack.Screen name="cart" />
          <Stack.Screen name="order/[id]" />
          <Stack.Screen name="rate/[id]" options={{ presentation:
          ↪ 'modal' }} />
        </Stack.Protected>
        <Stack.Protected guard={!signedIn}>
          <Stack.Screen name="login" />
        </Stack.Protected>
```

```
      </Stack>
    </>
  );
}

export default function RootLayout() {
  return (
    <GestureHandlerRootView style={{ flex: 1 }}>
      <SafeAreaProvider>
        <ThemeProvider brand="platter">
          <SessionProvider storageKey={TOKEN_KEY} storage={AsyncStorage}>
            <ToastProvider bottomOffset={120}>
              <RootStack />
            </ToastProvider>
          </SessionProvider>
        </ThemeProvider>
      </SafeAreaProvider>
    </GestureHandlerRootView>
  );
}
```

Read it from the bottom up. The providers wrap the whole app: gesture handling, safe areas (the notch and home indicator), the theme with Platter's brand, the signed-in session stored on the device, and toasts. `RootStack` keeps the splash screen up until the fonts and session are ready, then shows a **stack**: screens that slide in on top of each other. `Stack.Protected` is the clean way to do authentication: the tabs and order screens exist only when `signedIn` is true, and the login screen only when it's false, so a signed-out user can't reach a signed-in screen even through a deep link.

And the tab bar:

```
import { useCart } from '@pro/api';
import { Icon, useTheme, type IoniconName } from '@pro/ui';
import { Tabs } from 'expo-router/tabs';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

const TABS: { name: string; title: string; icon: IoniconName;
↪ iconActive: IoniconName }[] = [
  { name: 'index', title: 'Home', icon: 'restaurant-outline',
  ↪ iconActive: 'restaurant' },
  { name: 'search', title: 'Search', icon: 'search-outline', iconActive:
  ↪ 'search' },
  { name: 'orders', title: 'Orders', icon: 'receipt-outline',
  ↪ iconActive: 'receipt' },
  { name: 'profile', title: 'Profile', icon: 'person-outline',
  ↪ iconActive: 'person' },
];

export default function TabsLayout() {
  const { colors, fontFamilies } = useTheme();
  const insets = useSafeAreaInsets();
  // Keeps the cart warm for the cart bar on Home and Search.
```

```
  useCart('platter');
  return (
    <Tabs
      screenOptions={{
        headerShown: false,
        tabBarActiveTintColor: colors.primary,
        tabBarInactiveTintColor: colors.textMuted,
        tabBarStyle: {
          backgroundColor: colors.surface,
          borderTopColor: colors.divider,
          height: 66 + insets.bottom,
          paddingTop: 4,
          paddingBottom: insets.bottom + 6,
        },
        tabBarLabelStyle: { fontFamily: fontFamilies[600], fontSize: 11,
        ↪ lineHeight: 14 },
        sceneStyle: { backgroundColor: colors.background },
      }}
    >
      {TABS.map((tab) => (
        <Tabs.Screen
          key={tab.name}
          name={tab.name}
          options={{
            title: tab.title,
            tabBarAccessibilityLabel: tab.title,
            tabBarIcon: ({ focused }) => (
              <Icon name={focused ? tab.iconActive : tab.icon} size={22}
              ↪ color={focused ? 'primary' : 'textMuted'} />
            ),
          }}
        />
      ))}
    </Tabs>
  );
}
```

Notice that every color comes from `useTheme()`, the icon changes between outline and filled when the tab is active, and the tab bar's height adds the bottom safe-area inset so it clears the home indicator on modern phones. Session 2's screenshots caught an earlier version of this tab bar clipping its labels; the fixed height is the result.

Platform Differences

The same code runs everywhere, but the platforms aren't identical, and good apps respect the differences:

- **Shadows:** iOS uses shadow properties, Android uses `elevation`, and the web uses CSS `boxShadow`. The design system handles all three (Chapter 5).

- **Keyboard:** on iOS, a form near the bottom of the screen must move up when the keyboard opens; Android usually resizes the window itself.
- **Back:** Android has a system back gesture and button; iOS uses a swipe from the left edge. Expo Router's stacks support both.
- **Fonts and text rendering** differ slightly, so text that just fits on one platform can wrap on another. Leave room.
- **Filters:** Lumo's photo filters use a CSS filter on Android and web that iOS doesn't support, so on iOS the "Vivid" filter looks like "Normal". Claude reported this itself in session 4, which is a good example of the honesty you want from it.

When you need platform-specific code, `Platform.OS` tells you where you're running, and `Platform.select` picks a value per platform.

Key Takeaways

- React Native renders native components from TypeScript; Expo adds tooling, libraries and builds; Expo SDK 58 pairs with React Native 0.88.1.
- Screens are components; styles are objects in points with Flexbox, columns by default.
- Expo Router turns files in `src/app` into screens; `(group)` folders organize, `[param]` files take parameters, and `Stack.Protected` guards signed-in screens.
- Respect platform differences in shadows, keyboard, back navigation, text and filters.

CHAPTER 5 · FOUNDATIONS

A Design System Like the Big Apps

Big consumer apps look consistent because every screen is assembled from the same small set of parts: one button, one price, one rating pill, one bottom sheet, each with exactly one look. That set of parts, plus the rules for color, type, spacing and motion behind them, is a **design system**. In this book it lives in `packages/ui`, and all four phone apps are built from it.

A design system matters even more when an AI writes the code. Without one, Claude invents a slightly different button on every screen. With one, and a rule in `CLAUDE.md` that says "use only components and tokens from `@pro/ui`", every screen it builds comes out consistent.

Tokens: the Design Decisions as Code

A **token** is a named design decision: `spacing.lg` is 16 points, `radii.pill` is 999, the `title2` text style is 22 points, bold. Screens use the names, never the raw values, so a decision can change in one place.

Here are the spacing, radius and type tokens from session 1, taken straight from `docs/DESIGN.md`:

```
import { Platform, type ViewStyle } from 'react-native';

/** Spacing scale from DESIGN.md: 4/8/12/16/20/24/32/40. */
export const spacing = {
  xs: 4,
  sm: 8,
```

```
  md: 12,
  lg: 16,
  xl: 20,
  '2xl': 24,
  '3xl': 32,
  '4xl': 40,
} as const;
export type SpacingToken = keyof typeof spacing;

/** Corner radii from DESIGN.md: 8/12/16/24/999. */
export const radii = {
  sm: 8,
  md: 12,
  lg: 16,
  xl: 24,
  pill: 999,
} as const;
export type RadiusToken = keyof typeof radii;

/** Inter weights loaded by `useProFonts`, keyed by numeric weight. */
export const fontFamilies = {
  400: 'Inter_400Regular',
  500: 'Inter_500Medium',
  600: 'Inter_600SemiBold',
  700: 'Inter_700Bold',
  800: 'Inter_800ExtraBold',
} as const;
export type FontWeight = keyof typeof fontFamilies;

/** Type scale from DESIGN.md: 12/14/16/18/22/28/34. */
export const fontSizes = [12, 14, 16, 18, 22, 28, 34] as const;

export interface TextVariantStyle {
  fontSize: (typeof fontSizes)[number];
  lineHeight: number;
  weight: FontWeight;
  letterSpacing?: number;
  uppercase?: boolean;
}

export const typography = {
  display: { fontSize: 34, lineHeight: 40, weight: 800, letterSpacing:
  ↪ -0.6 },
  title1: { fontSize: 28, lineHeight: 34, weight: 800, letterSpacing:
  ↪ -0.4 },
  title2: { fontSize: 22, lineHeight: 28, weight: 700, letterSpacing:
  ↪ -0.2 },
  title3: { fontSize: 18, lineHeight: 24, weight: 700 },
  headline: { fontSize: 16, lineHeight: 22, weight: 600 },
  body: { fontSize: 16, lineHeight: 24, weight: 400 },
  subhead: { fontSize: 14, lineHeight: 20, weight: 500 },
  footnote: { fontSize: 14, lineHeight: 20, weight: 400 },
  caption: { fontSize: 12, lineHeight: 16, weight: 500 },
  overline: { fontSize: 12, lineHeight: 16, weight: 700, letterSpacing:
  ↪ 0.8, uppercase: true },
} as const satisfies Record<string, TextVariantStyle>;
export type TextVariant = keyof typeof typography;
```

The type scale is small on purpose: seven sizes from 12 to 34 points and named roles such as `headline`, `subhead`, `caption` and `overline`. Professional apps use far fewer text styles than you'd expect; a restaurant card uses four.

Four Brands, One System

Each app has its own brand color on a shared neutral base, in light and dark versions. Here is Platter's:

```
export const brands: Record<BrandName, Record<ColorScheme, BrandColors>>
↪ = {
  platter: {
    light: {
      primary: '#FC5A1E',
      primaryPressed: '#E04A12',
      onPrimary: '#FFFFFF',
      primarySoft: '#FFF0E9',
      onPrimarySoft: '#C2410C',
      accent: '#1BA672',
      onAccent: '#FFFFFF',
      accentSoft: '#E3F6EE',
      gradient: ['#FF7A3D', '#FC5A1E'],
    },
    dark: {
      primary: '#FF6A33',
      primaryPressed: '#FC5A1E',
      onPrimary: '#FFFFFF',
      primarySoft: '#3A1E13',
      onPrimarySoft: '#FFA27A',
      accent: '#2BC285',
      onAccent: '#06261A',
      accentSoft: '#11332A',
      gradient: ['#FF8A52', '#FC5A1E'],
    },
  },
```

Every brand defines the same set of roles: `primary` for the main action, `primaryPressed` for its pressed state, `onPrimary` for text on top of it, a soft tint for backgrounds, an accent, and a gradient for banners. Screens ask for roles, not colors: a button uses `colors.primary`, never `#FC5A1E`. That's how the same `Button` is tangerine in Platter, cobalt in Cartly and basil green in Minutely, and how dark mode works with no extra code in any screen.

App	Primary	Accent
Platter	Tangerine `#FC5A1E`	Fresh green `#1BA672`
Cartly	Cobalt `#2453FF`	Deal yellow `#FFC21A`
Lumo	Gradient, orange to magenta to purple	Blue `#0095F6`
Minutely	Basil green `#0C831F`	Butter `#F8CB46`
Reelhouse TV	Violet `#7C5CFF`	Cyan `#22D3EE`

The theme is selected once, at the top of each app, with `<ThemeProvider brand="platter">`, and any component reads it with `useTheme()`:

```
import { createContext, useContext, useMemo, type ReactNode } from
↪ 'react';
import { useColorScheme } from 'react-native';

import type { BrandName, ColorScheme } from '../tokens/colors';
import { createTheme, type Theme } from './createTheme';

const ThemeContext = createContext<Theme>(createTheme('platter',
↪ 'light'));

export interface ThemeProviderProps {
  brand: BrandName;
  /** 'system' (default) follows the device setting. */
  scheme?: ColorScheme | 'system';
  children: ReactNode;
}

export function ThemeProvider({ brand, scheme = 'system', children }:
↪ ThemeProviderProps) {
  const system = useColorScheme();
  const resolved: ColorScheme = scheme === 'system' ? (system === 'dark'
  ↪ ? 'dark' : 'light') : scheme;
  const theme = useMemo(() => createTheme(brand, resolved), [brand,
  ↪ resolved]);
  return <ThemeContext.Provider
  ↪ value={theme}>{children}</ThemeContext.Provider>;
}

/** The active theme. Outside a provider this is Platter, light. */
export function useTheme(): Theme {
  return useContext(ThemeContext);
}
```

Elevation That Looks Right Everywhere

Shadows are a classic cross-platform trap: iOS, Android and the web each do them differently. Session 1 solved it once, in a token function every card and sheet uses:

```
/**
 * Soft, low-opacity shadows: native shadow props on iOS, elevation on
 * Android, and the CSS-style `boxShadow` on web (react-native-web
 * deprecates the iOS shadow props).
 */
export function makeShadow(token: ShadowToken, shadowColor: string):
↪ ViewStyle {
  if (token === 'none') return {};
  const s = shadowSpecs[token];
  return (
    Platform.select<ViewStyle>({
      ios: {
        shadowColor,
        shadowOffset: { width: 0, height: s.y },
        shadowOpacity: s.opacity * 1.4,
        shadowRadius: s.blur / 2,
      },
      android: { elevation: s.elevation, shadowColor },
      default: { boxShadow: `0px ${s.y}px ${s.blur}px
      ↪ rgba(${hexToRgb(shadowColor)}, ${s.opacity})` },
    }) ?? {}
  );
```

The shadows are soft and low in opacity, between 6% and 12%, which is a big part of why the apps look calm rather than heavy.

Motion

```
/** Motion constants. Reanimated animations respect the system
↪ reduce-motion setting. */
export const motion = {
  pressScale: 0.97,
  duration: { fast: 150, base: 250, slow: 400 },
  spring: {
    press: { damping: 18, stiffness: 320, mass: 0.6 },
    sheet: { damping: 24, stiffness: 240, mass: 0.9 },
    pop: { damping: 10, stiffness: 260, mass: 0.7 },
  },
} as const;

/** Minimum touch target (CLAUDE.md accessibility rule). */
export const MIN_TOUCH = 44;
```

Three springs cover almost all motion in the apps: `press` for the quick squeeze of a button, `sheet` for bottom sheets sliding up, and `pop` for playful moments such as Lumo's heart burst. `MIN_TOUCH` encodes the accessibility rule that every touch target is at least 44 points.

Every touchable in every app goes through `PressableScale`:

```
import { type ReactNode } from 'react';
import { Pressable, type PressableProps, type StyleProp, type ViewStyle
↪ } from 'react-native';
import Animated, { useAnimatedStyle, useSharedValue, withSpring } from
↪ 'react-native-reanimated';

import { motion } from '../tokens/scale';

const AnimatedPressable = Animated.createAnimatedComponent(Pressable);

export interface PressableScaleProps extends Omit<PressableProps,
↪ 'style' | 'children'> {
  style?: StyleProp<ViewStyle>;
  children?: ReactNode;
  /** Scale while pressed. Defaults to 0.97 (DESIGN.md). */
  scaleTo?: number;
}

/**
 * The base touchable: a Pressable that springs to 0.97 while pressed. The
 * spring follows the system reduce-motion setting (Reanimated default).
 * Defaults `accessibilityRole` to "button".
 */
export function PressableScale({
  scaleTo = motion.pressScale,
  style,
  onPressIn,
  onPressOut,
  disabled,
  accessibilityRole = 'button',
  children,
  ...rest
}: PressableScaleProps) {
  const scale = useSharedValue(1);
  const animatedStyle = useAnimatedStyle(() => ({ transform: [{ scale:
  ↪ scale.value }] }));

  return (
    <AnimatedPressable
      accessibilityRole={accessibilityRole}
      disabled={disabled}
      onPressIn={(e) => {
        if (!disabled) scale.set(withSpring(scaleTo,
        ↪ motion.spring.press));
        onPressIn?.(e);
      }}
      onPressOut={(e) => {
        scale.set(withSpring(1, motion.spring.press));
        onPressOut?.(e);
```

```
      }}
      style={[style, animatedStyle]}
      {...rest}
    >
      {children}
    </AnimatedPressable>
  );
}
```

It's 52 lines, and it gives four apps a consistent, physical press feel, the right accessibility role by default, and motion that respects the system's reduce-motion setting.

The Component Library

By the end of the book, `packages/ui` held about fifty components. The ones you'll see on almost every screen:

Component	What it does
`Text`	Every piece of text, with a `variant` from the type scale
`Button`, `IconButton`	Primary, secondary and ghost buttons in three sizes
`Price`	A price with an optional MRP strike-through and discount percentage, formatted in rupees
`QuantityStepper`	The ADD button that turns into a minus, count and plus
`RatingPill`, `Stars`	Ratings in the category's familiar shapes
`CartBar`	The floating "3 items, View cart" bar
`BottomSheet`	Sheets for customization, coupons, payment, filters and sort
`BillDetails`	Every line of a bill, with waived fees struck through
`Skeleton`	Shimmering placeholders shaped like the content
`MapView`	The tracking map, drawn with SVG so it works on every platform
`Art`, `ArtTile`	The 3D illustrations, by name, on soft gradients
`PhotoScene`	Lumo's illustrated "photos", composed from a post's seed
`HeartBurst`, `StoryProgress`	Lumo's like animation and story progress bars

Several of these were added by the app sessions rather than the foundation session, because the app prompts said: "add missing pieces to the shared packages (with tests) rather than inside the app." Platter's session added `CartBar`, `BillDetails`, `TipSelector`, `OtpInput` and others; when Cartly needed the same address, coupon and payment sheets, its session moved them out of Platter into the shared package so both apps use one copy.

> **Tip:** Watch for duplication between apps when you review. If two apps each have their own `PaymentSheet`, ask Claude to move it into the shared package and make both apps use it. Duplicated components drift apart, and an AI will happily keep both.

The Illustrations

Photographs of food, products and people are the hardest part of a demo app: you can't use other companies' photos, stock photos have licensing terms, and AI-generated images need care. The apps in this book use something cleaner: **Microsoft's Fluent Emoji 3D** illustrations, released under the MIT license, which allows commercial use. The book's design system includes 187 of them, from biryani and burgers to phones, sneakers, mountains and astronauts, named by what they show:

```
<ArtTile name="curry-and-rice" tint="primarySoft" size={72} />
```

On soft gradient tiles, they give the apps the bright, friendly look of modern category illustrations, and Lumo composes them into layered scenes to stand in for users' photos. In your own app, you'll replace them with real restaurant, product and user photos; Chapter 20 covers loading images fast.

Key Takeaways

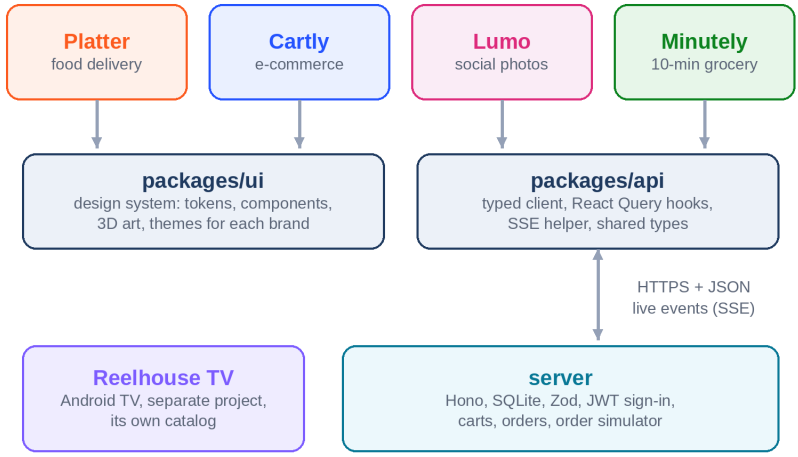
- A design system is tokens (named decisions) plus a component library; with an AI writing the code, it's what keeps every screen consistent.
- Screens ask for color roles, never hex values, so brands and dark mode come for free.
- Solve platform differences, such as shadows, once in the system.

- A few springs and one `PressableScale` give every app a consistent, accessible feel.
- Tell Claude to add missing pieces to the shared package with tests, and watch for duplication in review.

CHAPTER 6 · FOUNDATIONS

One Backend for Four Apps

A professional app is only as good as the data behind it. A food delivery home screen needs restaurants with menus, ratings and offers; a product page needs reviews and stock; a tracking screen needs an order that actually moves. This chapter covers the backend all four phone apps share, the typed client the apps use to reach it, and the live connection that makes order tracking feel alive.



Four phone apps share one design system, one API client and one backend.
The TV app is separate because it needs the react-native-tvos build of React Native.

How the pieces fit: four apps built from one design system and one typed API client, talking to one server over HTTPS and a live event stream.

The Server

Session 1 built the server in `server/`, using:

- **Node.js and TypeScript**, so the server and the apps share types.
- **Hono**, a small, fast web framework.
- **SQLite** through `better-sqlite3`: a whole database in one file, perfect for development and demos.
- **Zod** to validate every request's input.
- **JWT** (JSON Web Tokens) for signed-in sessions.

It exposes a REST API, grouped by app: `/platter/...` for restaurants and menus, `/cartly/...` for products, reviews and returns, `/lumo/...` for posts, stories, comments and messages, `/minutely/...` for the grocery catalog, plus shared routes for sign-in, the user, addresses, notifications, carts, coupons and orders.

Seed Data: a Fictional City

Demo apps die on bad data: three restaurants called "Restaurant 1", products with no reviews, a feed with one post. The foundation prompt asked for "rich, believable fictional data for all four apps" with minimum counts, and Claude delivered the city of Northbay:

Data	Count
Restaurants	24, with 870 menu items
Products	60, with 416 reviews
People	20, with 112 posts, 40 stories and 392 comments
Grocery items	120, in 12 categories
Coupons	12

Claude wrote the seed data with parallel subagents, each working against typed input shapes, and then wrote a test that checks the counts, that every illustration name exists, that every email and website uses the reserved `.example` domain, and that pure-vegetarian restaurants serve no

non-vegetarian dishes. That last check is a nice example of a domain rule turned into a test.

Signing In

The apps sign in with a phone number and a one-time code, like most consumer apps in India. The server has no SMS provider, so it accepts the code 123456 and prints codes to its console. It answers a correct code with a JWT, which the app stores on the device and sends with every request.

> **Warning:** The fixed code and console logging are for development only. Chapter 25 covers what a real sign-in needs: an SMS or WhatsApp provider, rate limits, code expiry, and secure token storage.

One Bill, Computed on the Server

Money must be calculated in exactly one place, and that place is the server. Each app's cart has its own rules: Platter has a delivery partner fee, a platform fee and GST on food; Minutely has a handling fee and a small-cart fee under ₹99; Cartly has free delivery above ₹499. The server returns a `Bill` object with every amount, and the apps only display it.

Displaying a bill well is a design problem of its own, so the shared API package turns a bill into display rows:

```
/**
 * The rows of a bill in display order, as every commerce app shows it:
 ↪ item
 * total, then fees (only the ones that apply), discounts, tip and a
 ↪ total.
 * A waived delivery fee stays visible as a strike-through so the saving
 ↪ is
 * clear; zero-value fees are left out.
 */
export function billLines(bill: Bill, app: CommerceApp, options:
↪ BillLinesOptions = {}): BillLine[] {
  const notes = FEE_NOTES[app] ?? {};
  const lines: BillLine[] = [{ key: 'itemTotal', label: 'Item total',
  ↪ amount: bill.itemTotal }];
  const mrpSavings = Math.round((bill.mrpTotal - bill.itemTotal) * 100)
  ↪ / 100;
  if (mrpSavings > 0) lines[0]!.waivedAmount = bill.mrpTotal;

  if (bill.deliveryFee > 0 || bill.deliveryFeeWaived > 0) {
```

```
    lines.push({
      key: 'deliveryFee',
      label: app === 'platter' ? 'Delivery partner fee' : 'Delivery fee',
      amount: bill.deliveryFee,
      waivedAmount: bill.deliveryFee === 0 && bill.deliveryFeeWaived > 0
      ↪ ? bill.deliveryFeeWaived : undefined,
      note: notes.deliveryFee,
    });
  }
  const fees: [keyof Bill, string][] = [
    ['platformFee', 'Platform fee'],
    ['handlingFee', 'Handling fee'],
    ['smallCartFee', 'Small cart fee'],
    ['taxes', app === 'platter' ? 'GST and restaurant charges' : 'Taxes'],
  ];
  for (const [key, label] of fees) {
    const amount = bill[key];
    if (amount > 0) lines.push({ key, label, amount, note: notes[key] });
  }
  if (bill.couponDiscount > 0) {
    lines.push({
      key: 'couponDiscount',
      label: options.couponCode ? `Coupon (${options.couponCode})` :
      ↪ 'Coupon discount',
      amount: -bill.couponDiscount,
      tone: 'discount',
    });
  }
  if (bill.tip > 0) lines.push({ key: 'tip', label: 'Delivery partner
  ↪ tip', amount: bill.tip });
  lines.push({ key: 'grandTotal', label: 'To pay', amount: bill.total,
  ↪ tone: 'total' });
  return lines;
}
```

A waived delivery fee isn't hidden; it stays visible, struck through, so the customer sees the saving. Zero-value fees disappear. A coupon shows its code. These are the small trust signals Chapter 1 talked about, and they're written once for three apps.

Making Orders Move

A tracking screen is only convincing if the order moves. The server has an **order simulator** that advances every active order on a clock:

```
/**
 * Demo timings: a whole delivery takes a couple of minutes so it can be
 * watched end to end. SIM_SPEED scales them (2 = twice as fast).
 */
const BASE: Record<CommerceApp, [OrderStatus, number][]> = {
  platter: [
    ['confirmed', 0],
```

```
    ['preparing', 6],
    ['picked_up', 30],
    ['on_the_way', 36],
    ['delivered', 120],
  ],
  minutely: [
    ['confirmed', 0],
    ['packing', 4],
    ['on_the_way', 15],
    ['delivered', 75],
  ],
  cartly: [
    ['confirmed', 0],
    ['packed', 20],
    ['shipped', 60],
    ['out_for_delivery', 120],
    ['delivered', 180],
  ],
};
```

A real food delivery takes 30 minutes; the demo takes two, so you can watch an order from "Confirmed" to "Delivered" in one sitting. `SIM_SPEED` speeds it up or slows it down. Every second, the simulator checks active orders and publishes events:

```
/**
 * Advances active orders on the clock and publishes events on the bus:
 * `status` when an order moves to a new step, and `location` every tick
 * while the rider is moving. Status is a pure function of time since the
 * order was placed, so a restart picks up where it left off.
 */
export class OrderSimulator {
  private timer: ReturnType<typeof setInterval> | null = null;

  constructor(private deps: Deps) {}

  tick(): void {
    const rows = this.deps.db.prepare("SELECT * FROM orders WHERE status
    ↪ NOT IN ('delivered', 'cancelled')").all() as OrderRow[];
    for (const row of rows) {
      const changed = syncOrder(this.deps, row);
      const order = buildOrder(this.deps, row);
      if (changed) this.emit(row.id, { type: 'status', order });
      const t = order.tracking;
      if (t && t.progress != null && t.progress < 1 && t.riderPosition) {
        this.emit(row.id, { type: 'location', orderId: row.id, progress:
        ↪ t.progress, position: t.riderPosition, etaMinutes:
        ↪ order.etaMinutes });
      }
    }
  }

  start(intervalMs = 1000): void {
    if (this.timer) return;
    this.timer = setInterval(() => {
```

```
      try {
        this.tick();
      } catch (err) {
        console.error('Order simulator tick failed', err);
      }
    }, intervalMs);
    this.timer.unref?.();
  }

  stop(): void {
    if (this.timer) clearInterval(this.timer);
    this.timer = null;
  }

  private emit(orderId: string, event: OrderStreamEvent) {
    this.deps.bus.emit(orderTopic(orderId), event);
  }
}
```

The comment at the top of the class records the important design decision: status is "a pure function of time since the order was placed", so if the server restarts, every order picks up exactly where it should be.

Live Updates with Server-Sent Events

How does the app hear about those events? Three options are common:

- **Polling:** ask the server every few seconds. Simple, but slow to update and wasteful.
- **WebSockets:** a two-way connection. Powerful, but more machinery than tracking needs.
- **Server-Sent Events (SSE):** a one-way stream of events from server to app over an ordinary HTTP response. Exactly right for "tell me when my order changes".

Here is the server side:

```
/**
 * Live order status as Server-Sent Events. Sends a `snapshot` first,
 ↪ then
 * `status` and `location` events, and closes after delivery or
 ↪ cancellation.
 * Auth: Bearer header or ?token= (EventSource cannot set headers).
 */
app.get('/:id/events', (c) => {
  const userId = currentUser(c);
  const first = orders.getOrder(deps, userId, c.req.param('id'));
  return streamSSE(c, async (stream) => {
```

```
      let closed = false;
      let seq = 0;
      let done: () => void = () => undefined;
      const finished = new Promise<void>((resolve) => (done = resolve));
      const isFinal = (e: OrderStreamEvent) => e.type !== 'location' &&
      ↪ (e.order.status === 'delivered' || e.order.status ===
      ↪ 'cancelled');
      // Writes are chained so events go out in order, and the stream
      ↪ closes
      // only after the final status has been written.
      let queue = Promise.resolve();
      const send = (event: OrderStreamEvent) => {
        queue = queue.then(async () => {
          if (closed) return;
          await stream.writeSSE({ event: event.type, data:
          ↪ JSON.stringify(event), id: String(++seq) });
          if (isFinal(event)) done();
        });
        return queue;
      };
      // Subscribe before the snapshot so no change in between is missed.
      const topic = orderTopic(first.id);
      const listener = (event: OrderStreamEvent) => void send(event);
      deps.bus.on(topic, listener);
      stream.onAbort(() => {
        closed = true;
        done();
      });
      void send({ type: 'snapshot', order: first });
      // Comments keep proxies from closing an idle stream.
      const ping = setInterval(() => void stream.write(': ping\n\n'),
      ↪ 15_000);
      await finished;
      clearInterval(ping);
      deps.bus.off(topic, listener);
      await queue;
      closed = true;
    });
  });
```

It sends a `snapshot` of the order first, then `status` and `location` events as the simulator publishes them, and closes after delivery. It subscribes before sending the snapshot, so no change can slip through the gap. It sends a comment every 15 seconds so proxies don't close an idle connection. And it chains its writes so events always arrive in order. Claude found and fixed a race in session 1 where the final status could be dropped; the chained queue is the fix.

React Native doesn't ship a browser `EventSource`, so `packages/api/src/sse.ts` includes a small SSE parser that works on Android, iOS and the web, and a hook wraps it for screens:

```
/**
 * Live order tracking over SSE: keeps `useOrder(id)` in the cache up to
 ↪ date
 * and exposes the moving rider. Closes when the order finishes or the
 * component unmounts.
 */
export function useOrderStream(orderId: string | undefined):
↪ OrderStreamState {
  const api = useApi();
  const qc = useQueryClient();
  const order = useOrder(orderId).data;
  const [rider, setRider] = useState<OrderStreamState['rider']>(null);
  const [connected, setConnected] = useState(false);
  const [error, setError] = useState<Error | null>(null);
  const finished = order?.status === 'delivered' || order?.status ===
  ↪ 'cancelled';

  useEffect(() => {
    if (!orderId || finished) return;
    const stream = api.orders.subscribe(orderId, {
      onOpen: () => {
        setConnected(true);
        setError(null);
      },
      onError: (e) => {
        setConnected(false);
        setError(e);
      },
      onEvent: (event: OrderStreamEvent) => {
        if (event.type === 'location') {
          setRider({ position: event.position, progress: event.progress,
          ↪ etaMinutes: event.etaMinutes });
          qc.setQueryData<Order>(queryKeys.order(event.orderId), (old) =>
            old?.tracking
              ? { ...old, etaMinutes: event.etaMinutes, tracking: {
              ↪ ...old.tracking, progress: event.progress, riderPosition:
              ↪ event.position } }
              : old,
          );
          return;
        }
        qc.setQueryData(queryKeys.order(event.order.id), event.order);
        if (event.type === 'status') void qc.invalidateQueries({
        ↪ queryKey: ['orders', 'list'] });
        if (event.order.status === 'delivered' || event.order.status ===
        ↪ 'cancelled') setConnected(false);
      },
    });
    return () => stream.close();
  }, [api, qc, orderId, finished]);

  return { order, rider, connected, error };
```

`useOrderStream` writes every event into the React Query cache, so any screen showing that order updates by itself: the tracking screen, the orders list, the active order card on the home screen. Location events update only the rider's position and the estimated arrival time, which is what the map animates.

The Typed Client and React Query

The apps never call `fetch` directly. `packages/api` has a typed client method for every route, and **React Query** hooks on top: `useRestaurants`, `useProduct`, `useCart`, `useOrderStream` and dozens more. React Query handles caching, background refresh, retries and loading and error states, which makes "every state is designed" much easier: each screen just reads `isLoading`, `error` and `data`.

Changes that should feel instant, such as adding to the cart, liking a post or toggling the wishlist, use **optimistic updates**: the hook updates the cache immediately, sends the request, and rolls back if the server refuses. Minutely's session described the effect: "ADD steppers and the green cart bar change on the tap itself, before the server answers."

Finally, a **contract test** runs the real client against the real server. If someone changes a route's shape on one side only, the test fails, so the four apps and the server can't silently drift apart.

Key Takeaways

- One server (Hono, SQLite, Zod, JWT) serves all four apps, with rich seeded fictional data checked by tests.
- Money is computed on the server; the apps display bills through one shared function that keeps waived fees visible.
- A clock-driven simulator makes orders move; status as a function of time survives restarts.
- Server-Sent Events push status and rider locations; a cross-platform SSE helper and `useOrderStream` keep every screen's data live.

- A typed client with React Query, optimistic updates and a contract test keeps apps and server in step.

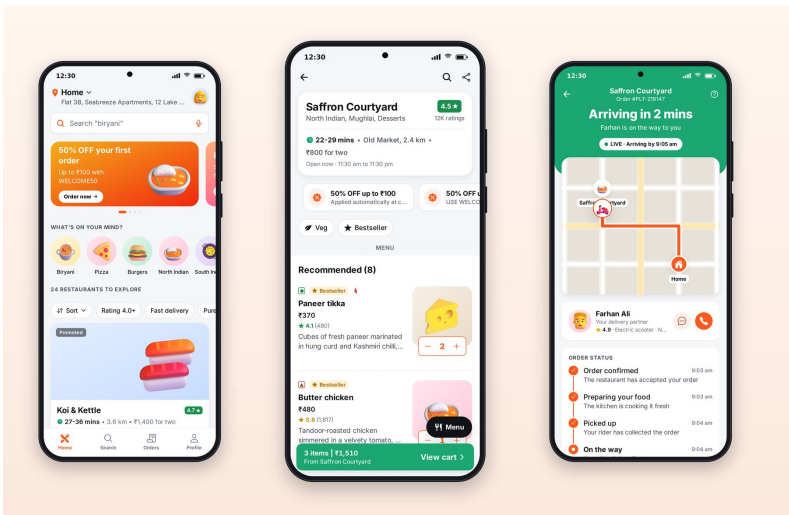
PART II

Platter, Food Delivery

CHAPTER 7 · FOOD DELIVERY

Planning a Food Delivery App

Food delivery apps are among the most refined consumer apps ever built. They have to help hungry people choose quickly from thousands of dishes, handle menus with sizes, add-ons and dietary marks, add up a bill with half a dozen fees honestly, take payment, and then turn a 30-minute wait into something reassuring to watch. This part builds Platter, an original app in that category, and this chapter covers the plan and the prompt behind it.



Platter: the home screen, a restaurant menu, and live order tracking with the rider on the way.

Study the Category Before You Prompt

Before writing a spec for a category, spend twenty minutes in the leading apps and write down the patterns users have learned to expect. For food delivery in India, that list looks like this:

- **A location-first header**: "Deliver to Home" with the full address, because everything (restaurants, delivery times, fees) depends on it.
- **A search bar with rotating hints**: "Search 'biryani'", then "Search 'pizza'", teaching users what they can search for.
- **"What's on your mind?"**: a row of round cuisine tiles, the fastest path from craving to restaurant.
- **Filter chips**: Rating 4.0+, Fast delivery, Pure veg, Offers, plus a sort menu.
- **Dense restaurant cards**: banner image, name, rating pill in green, delivery time, distance, cost for two, an offer strip.
- **Veg and non-veg marks**: a green dot in a square for vegetarian, a red triangle for non-vegetarian, on every dish. In India these are a legal labelling convention, not decoration.
- **ADD buttons that turn into steppers** (minus, count, plus) the moment you tap.
- **A customization sheet** for dishes with sizes and add-ons, with "Required" and "Choose up to" rules.
- **A floating cart bar**: "3 items, ₹1,510, View cart".
- **A transparent bill**: item total, delivery partner fee, platform fee, GST, coupon discount, tip, to pay.
- **Live tracking**: status steps, a map with a moving rider, an ETA that counts down, and the rider's name with call and chat buttons.

That list became section 1 of `docs/PRD.md`. Notice that it's written in the category's own vocabulary. "ADD buttons that turn into steppers" gives Claude a precise, familiar target; "a way to add items" would not.

Plan the States, Not Just the Screens

For each screen, the PRD and the prompt asked for designed states. The home screen alone needs:

State	What the user sees
Loading	Skeleton cards shaped like restaurant cards, shimmering
Full	Header, search, carousel, cuisines, chips, restaurant cards
Filtered to nothing	"No restaurants match these filters" with a Clear filters button
Error	"Couldn't load restaurants" with Retry
Restaurant closed	The card grayed out with its opening time
Cart has items	The floating cart bar over the list

Writing these down before prompting does two things: it tells Claude what to build, and it tells you what to check.

The Prompt

Session 2 built the whole app from this prompt:

```
Build Platter, the food delivery app in docs/PRD.md section 1, as
apps/platter: an Expo SDK 58 app with Expo Router, using @pro/ui (brand
"platter") and @pro/api. It should feel as polished as the best food
delivery apps: dense but calm layouts, real loading skeletons, empty and
error states, smooth press feedback, a sticky cart bar, bottom sheets.

Screens: OTP login, Home, Search, Restaurant (menu, customization sheet),
Cart and checkout (coupons, tip, bill details, address and payment
↪ sheets),
Order tracking (live SSE status, SVG map with moving rider, ETA, rider
card), Orders, Rate order, Profile. Tabs: Home, Search, Orders, Profile.

Work like a professional team:
1. Plan first: list the routes, components and data each screen needs, and
   anything missing from @pro/ui or @pro/api. Add missing pieces to the
   shared packages (with tests) rather than inside the app.
2. Build screen by screen. After each one, export the app for web and
   look at it: run the server, serve the web build, and capture the screen
   in headless Chromium at 393x852 CSS pixels with device scale 3
   ↪ (Playwright
   is installed globally; require('playwright') works). Compare it with
   ↪ the
   PRD and fix anything that looks unpolished: alignment, spacing, text
   cut off, images missing, overlapping elements.
3. Add tests for the app's logic (bill calculation display, cart behavior,
   tracking state) and component tests for the main screens.
```

```
4. Make sure `npx expo export -p web` works for the app, and add an
   `npm run export:web -w apps/platter` script.
5. Save your final screenshots of every screen to
↳ apps/platter/screenshots/
   (PNG, named 01-login.png, 02-home.png, ...), with the tracking screen
   captured mid-delivery.

Finish with typecheck, lint and tests passing for the whole workspace,
↳ then
a summary of the screens, the components you added to the shared packages,
and any problems you hit.
```

The prompt doesn't repeat the PRD; it points to it. What it adds is the standard ("as polished as the best food delivery apps"), the list of screens and tabs, and a way of working: plan first, add missing pieces to the shared packages, build screen by screen, check each screen visually at phone size, test the logic, save screenshots, and finish with all checks passing.

What the Session Did

The session ran for 63 minutes over 121 turns and cost $10.17. Claude started, as asked, by writing a plan to `apps/platter/PLAN.md`: the routes, the components and data each screen needed, and what was missing from the shared packages. Then it built screen by screen.

Its report summarized the result in its own words: "Every screenshot comes from a Playwright run that drives the real UI against the live server: log in with OTP, search, add dishes, apply a coupon, tip, place the order, then watch tracking." Playwright is a browser automation tool; Claude wrote a script that used the app like a real user and photographed each screen along the way.

It also listed the problems it hit and fixed:

- The search screen crashed on the web build when switching between its Restaurants and Dishes lists; each list needed its own key.
- Tab labels were clipped because the tab bar was too short, and toasts covered the cart and pay bars.
- Lint caught a state update inside an effect, and the API client being used before it was declared.
- A tracking test passed alone and failed when the whole suite ran.

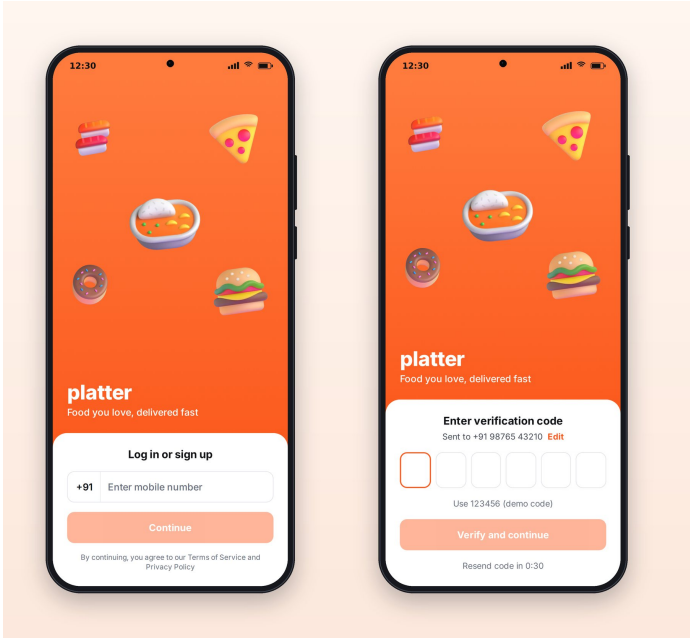
- Jest hung after the screen tests, because React Query keeps cleanup timers running for five minutes; tests now use a query client with those timers off.

And one honest note: "`docs/PRD.md` has changes I didn't make: the Reelhouse TV section was rewritten, probably by you or another session. I left them as they are." That was true; the TV section of the PRD had been expanded while the session was running. Claude noticed, didn't touch it, and said so.

The session added nine components to the shared design system (`CartBar`, `BillDetails`, `TipSelector`, `OtpInput`, `OptionRow`, `StarRatingInput`, `Carousel`, `ScreenHeader` and `TextField`) and two bill and order helpers to the API package, all reused later by Cartly and Minutely.

Signing In

The first screens a user sees are the sign-in screens: a phone number, then a six-box one-time code that verifies itself when the last digit is typed, with a resend timer.



Platter's sign-in: phone number, then a six-digit code that verifies automatically.

Key Takeaways

- Before prompting, list the category's familiar patterns in its own vocabulary; that list is the core of the PRD.
- Plan every screen's states (loading, empty, error, partial, full) before prompting, so Claude builds them and you can check them.
- A good app prompt points to the spec and adds the standard, the screens, and a way of working with visual checks.
- Read the session's report for the problems it hit and anything it noticed but didn't change.

CHAPTER 8 · FOOD DELIVERY

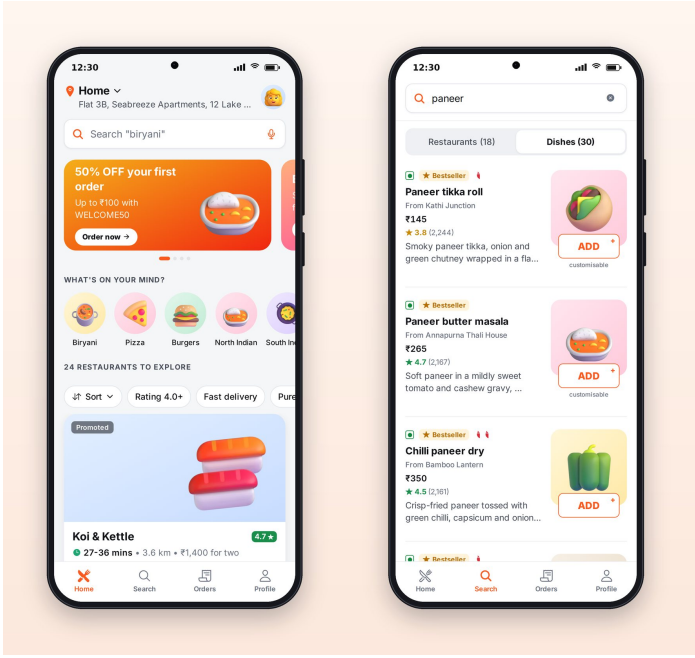
Home, Discovery and Restaurant Pages

This chapter walks through Platter's discovery screens, the home page, search and the restaurant page, and the pieces of code that make them work. Use it as a guide to what to look for when you review screens like these.

The Home Screen

Read the home screen from top to bottom and notice how every element earns its place:

1. **Address header.** "Home" with the full address, and a chevron that opens an address sheet. Delivery times and fees depend on it, so it comes first.
2. **Search bar** with a rotating hint and a microphone icon.
3. **Promo carousel** with a dot indicator, advancing on its own. Each banner is a deep link: "/offers", "/cuisine/biryani" or a specific restaurant.
4. **"What's on your mind?"** cuisine tiles, using the 3D illustrations on soft circles.
5. **Filter chips** that stay pinned under the header while you scroll.
6. **Restaurant cards**, with "Promoted" tags where appropriate.
7. **The tab bar**, with the current tab in the brand color.



The home screen and search. Home leads with the address, search, offers and cuisines; search splits results into restaurants and dishes, with ADD buttons right on the dish results.

Filters as Pure Functions

The chips and the sort menu look like UI, but the logic behind them is plain data, kept out of the screen in `lib/filters.ts`:

```
import type { RestaurantQuery, RestaurantSort } from '@pro/api';

export const FILTERS = [
  { key: 'rating', label: 'Rating 4.0+' },
  { key: 'fast', label: 'Fast delivery' },
  { key: 'veg', label: 'Pure veg' },
  { key: 'offers', label: 'Offers' },
] as const;
export type FilterKey = (typeof FILTERS)[number]['key'];

export const SORTS: { key: RestaurantSort; label: string }[] = [
  { key: 'relevance', label: 'Relevance' },
  { key: 'rating', label: 'Rating: high to low' },
  { key: 'delivery_time', label: 'Delivery time' },
  { key: 'cost_low', label: 'Cost: low to high' },
  { key: 'cost_high', label: 'Cost: high to low' },
];
```

```
export interface FilterState {
  active: readonly FilterKey[];
  sort: RestaurantSort;
}

export const NO_FILTERS: FilterState = { active: [], sort: 'relevance' };

export function toggleFilter(state: FilterState, key: FilterKey):
↪ FilterState {
  const active = state.active.includes(key) ? state.active.filter((k) =>
  ↪ k !== key) : [...state.active, key];
  return { ...state, active };
}

export function hasFilters(state: FilterState): boolean {
  return state.active.length > 0 || state.sort !== 'relevance';
}

/** Home chips to the restaurant list query. Unset filters are left out.
↪ */
export function toRestaurantQuery(state: FilterState, extra:
↪ Pick<RestaurantQuery, 'q' | 'cuisine'> = {}): RestaurantQuery {
  const query: RestaurantQuery = { ...extra };
  if (state.active.includes('rating')) query.minRating = 4;
  if (state.active.includes('fast')) query.fast = true;
  if (state.active.includes('veg')) query.veg = true;
  if (state.active.includes('offers')) query.offers = true;
  if (state.sort !== 'relevance') query.sort = state.sort;
  return query;
}
```

`toRestaurantQuery` turns the chips into the API query. Because it's a pure function (same input, same output, no side effects), it's trivial to test, and the screen stays focused on layout. When you review an AI's screens, look for this separation; logic buried in a screen component is harder to test and easier to break.

The same file shows a professional detail: banner links are web-style paths, so the marketing team can point a banner anywhere without an app release.

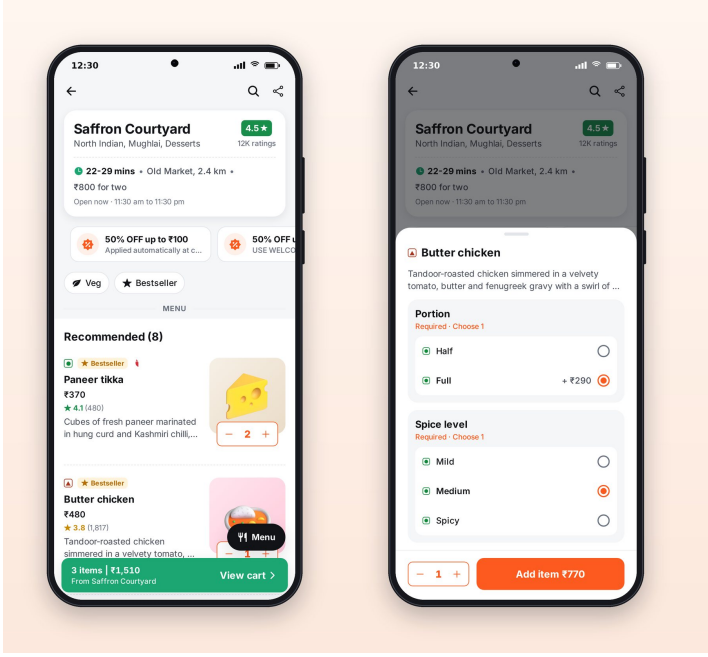
```
/**
 * Where a server banner link goes in the app. Banners name web-style
 ↪ paths
 * ("/offers", "/cuisine/biryani", "/restaurant/r-01"); unknown ones
 ↪ open search.
 */
export function bannerRoute(link: string | null): { pathname: string;
↪ params?: SearchParams } {
  if (!link) return { pathname: '/search' };
  const [, section, value] = link.split('/');
  if (section === 'restaurant' && value) return { pathname:
  ↪ `/restaurant/${value}` };
  if (section === 'cuisine' && value) return { pathname: '/search',
  ↪ params: { cuisine: titleCase(value) } };
```

```
  if (section === 'offers') return { pathname: '/search', params: {
  ↪ offers: '1' } };
  if (section === 'collection' && value) return { pathname: '/search',
  ↪ params: { q: value.replace(/-/g, ' ') } };
  return { pathname: '/search' };
}

function titleCase(slug: string): string {
  return slug
    .split('-')
    .map((w) => w.charAt(0).toUpperCase() + w.slice(1))
    .join(' ');
}
```

Unknown links fall back to search rather than crashing or doing nothing.

The Restaurant Page



A restaurant page and the customization sheet. Dishes carry veg or non-veg marks, Bestseller tags, ratings and ADD steppers; the sheet enforces "Required, Choose 1" and shows the price as you choose.

The restaurant page has a lot to do: an info card with rating, cuisines, delivery time, distance and cost for two; an offers strip you can swipe; Veg and

Bestseller filters; a search box for the menu; the menu grouped by category; a floating "Menu" button that jumps between categories; and the cart bar.

Each dish row shows the veg or non-veg mark, a Bestseller tag, the name, price, rating and description, and an illustration with the ADD stepper overlapping its bottom edge, a layout users of every big food app will recognize.

Customization: Rules Users Never Think About

Tapping ADD on a dish with options opens the customization sheet. Butter chicken, for example, has a required portion (Half or Full, choose one) and an optional spice level. The rules are subtle:

- A required single-choice group always has something selected; tapping the selected option again does nothing.
- An optional single-choice group can be cleared by tapping its selected option.
- A multiple-choice group stops accepting options at its maximum.
- The Add button is disabled until every required group has a choice.
- The same dish with different options is a different cart line: "Butter chicken, Full, Medium" and "Butter chicken, Half, Mild" are two lines.

Here is that logic, again as pure functions:

```
/**
 * Toggles an option: a single-choice group swaps to it; a multiple-choice
 * group adds or removes it, ignoring additions past `max`.
 */
export function toggleOption(selection: Selection, group:
↪ CustomizationGroup, optionId: string): Selection {
  const current = selection[group.id] ?? [];
  if (group.kind === 'single') {
    if (current[0] === optionId && !group.required) return {
    ↪ ...selection, [group.id]: [] };
    return { ...selection, [group.id]: [optionId] };
  }
  if (current.includes(optionId)) return { ...selection, [group.id]:
  ↪ current.filter((id) => id !== optionId) };
  if (group.max != null && current.length >= group.max) return selection;
  return { ...selection, [group.id]: [...current, optionId] };
}

/** The first group that still needs a choice, or null when the
↪ selection is complete. */
```

```
export function missingRequired(item: Pick<MenuItem, 'customizations'>,
↪ selection: Selection): CustomizationGroup | null {
  return item.customizations.find((g) => g.required &&
  ↪ (selection[g.id]?.length ?? 0) === 0) ?? null;
}

/** Base price plus every picked option. */
export function unitPrice(item: Pick<MenuItem, 'price' |
↪ 'customizations'>, selection: Selection): number {
  let total = item.price;
  for (const group of item.customizations) {
    for (const option of group.options) if
    ↪ (selection[group.id]?.includes(option.id)) total += option.price;
  }
  return total;
}

/** "Choose 1", "Choose up to 3", "Optional". */
export function groupHint(group: CustomizationGroup): string {
  if (group.kind === 'single') return group.required ? 'Required ·
  ↪ Choose 1' : 'Optional · Choose 1';
  return group.max != null ? `Optional · Choose up to ${group.max}` :
  ↪ 'Optional';
}

/** The selection as the API's `options` list. */
export function toCartOptions(selection: Selection): { groupId: string;
↪ optionId: string }[] {
  return Object.entries(selection).flatMap(([groupId, ids]) =>
  ↪ ids.map((optionId) => ({ groupId, optionId })));
}

function optionKey(options: readonly { groupId: string; optionId: string
↪ }[]): string {
  return options
    .map((o) => `${o.groupId}:${o.optionId}`)
    .sort()
    .join('|');
}

/** The cart line for this exact item and selection, if one exists. */
export function lineForSelection(cart: Cart | undefined, itemId: string,
↪ selection: Selection): CartLine | undefined {
  const key = optionKey(toCartOptions(selection));
  return cart?.items.find((l) => l.productId === itemId &&
  ↪ optionKey(l.options) === key);
}
```

And part of its test:

```
describe('customization', () => {
  it('starts from the default options', () => {
    expect(defaultSelection(butterChicken)).toEqual({ portion: ['half'],
    ↪ addons: [] });
    expect(defaultSelection(dalMakhani)).toEqual({});
  });
```

```
it('picks the first option of a required group with no default', () => {
  const item = { customizations: [{ ...portion, options:
  ↪ portion.options.map((o) => ({ ...o, isDefault: false })) }] };
  expect(defaultSelection(item)).toEqual({ portion: ['half'] });
});

it('swaps single choices and keeps required groups filled', () => {
  let s = defaultSelection(butterChicken);
  s = toggleOption(s, portion, 'full');
  expect(s.portion).toEqual(['full']);
  s = toggleOption(s, portion, 'full');
  expect(s.portion).toEqual(['full']);
});

it('adds add-ons up to the max and removes them again', () => {
  let s = defaultSelection(butterChicken);
  s = toggleOption(s, addons, 'naan');
  s = toggleOption(s, addons, 'rice');
```

`optionKey` sorts the chosen options before joining them, so the same selection made in a different order still matches the same cart line. That's the kind of detail an AI gets right when the tests describe the behavior, and gets subtly wrong when nobody checks.

One Cart, One Restaurant

Food delivery carts hold dishes from one restaurant at a time. When you add a dish from a second restaurant, Platter shows a "Replace cart item?" sheet instead of mixing kitchens or silently emptying the cart. The rule lives on the server; the app shows the sheet when the server reports the conflict.

> **Try It:** Ask Claude to add a "Repeat last order" card to Platter's home screen for the signed-in user. Point it at `useOrders` in `packages/api`, ask it to handle the case where a dish is no longer on the menu, and to add a screen test. Then review the screenshot it captures.

Key Takeaways

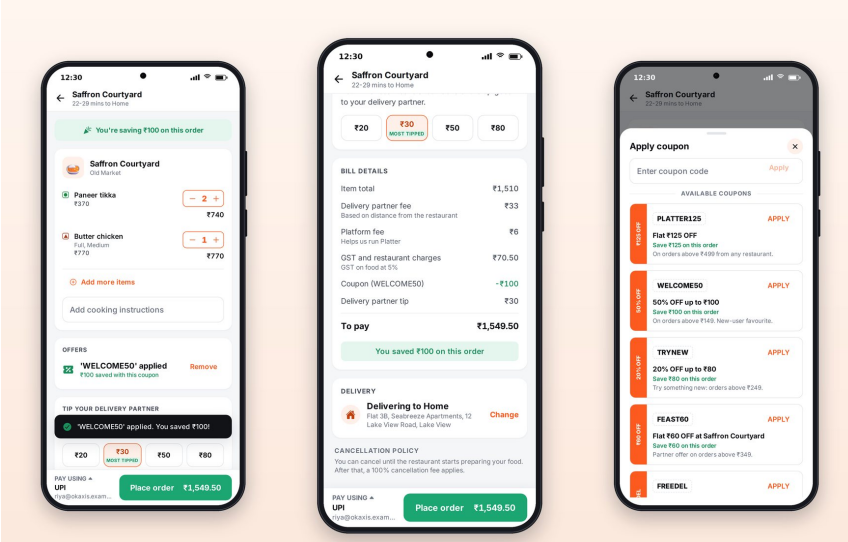
- Every element of a professional home screen earns its place: address, search, offers, cuisines, pinned filters, dense cards.
- Keep logic such as filters and customization rules in pure functions next to the screens, with tests; screens should mostly be layout.

- Customization has subtle rules (required, maximum, separate cart lines per selection); write them as tests.
- Small professional touches, such as banner deep links with safe fallbacks and a replace-cart sheet, are cheap to ask for and expensive to forget.

CHAPTER 9 · FOOD DELIVERY

Cart, Customization and Checkout

The cart is where a food delivery app earns or loses the order. Users abandon carts when the total jumps unexpectedly, when they can't find a coupon, or when a payment step feels unsafe. This chapter looks at how Platter's cart and checkout are built to keep them going.



The cart, its bill and the coupon sheet. The saving is announced at the top, every fee is listed, and coupons explain exactly what they do.

Anatomy of the Cart

From top to bottom, Platter's cart screen has:

1. **A header** with the restaurant's name and the delivery time to the chosen address.
2. **A savings banner**: "You're saving ₹100 on this order", celebrating the coupon.
3. **The items**, each with its options ("Full, Medium"), a stepper and a line total, and an "Add more items" link back to the menu.
4. **Cooking instructions**, a free-text field restaurants actually read.
5. **Offers**: the applied coupon with a Remove button, or a row that opens the coupon sheet.
6. **Tip your delivery partner**, with preset amounts and the most popular one marked "Most tipped".
7. **Bill details**, line by line.
8. **A sticky pay bar**: the payment method on the left ("Pay using UPI"), and "Place order ₹1,549.50" on the right.

The pay bar never scrolls away. Whatever the user is looking at, the next action is visible.

The Bill Tells the Truth

The bill comes from the server and is displayed by the shared `BillDetails` component, using the `billLines` function you saw in Chapter 6. The rules that make it trustworthy:

- Every fee is its own line, with a short note under it: "Based on distance from the restaurant", "Helps us run Platter", "GST on food at 5%".
- A waived fee isn't hidden; it's shown struck through with "FREE", so the user sees the saving.
- The coupon line names the coupon: "Coupon (WELCOME50)".
- The tip is shown as its own line, never folded into fees.
- The total is labelled "To pay", in bold.

> **Warning:** Never compute money in the app. The app displays what the server calculated. If the app adds up its own total, sooner or later it will disagree with the server by a rupee, and the user will see two different prices for the same order.

Coupons

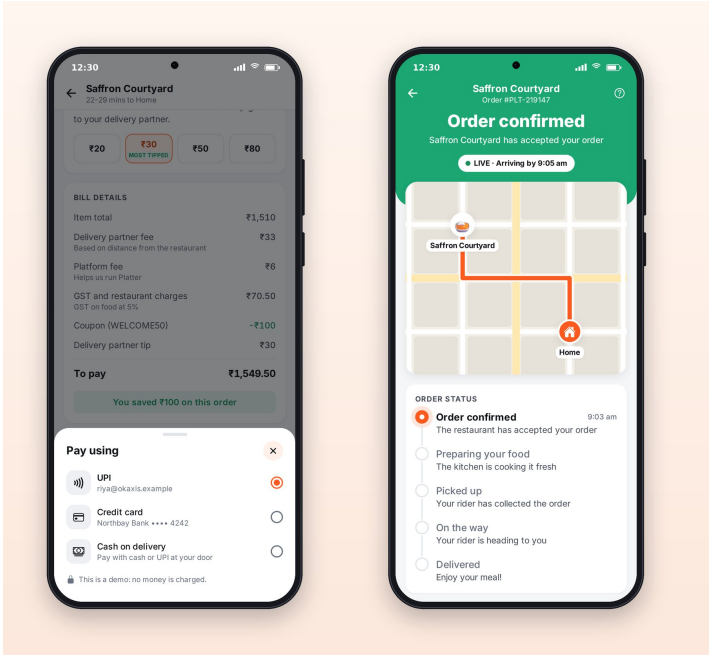
The coupon sheet lists the coupons available to this user and this cart, each with its code, a one-line description, the saving it would give, and its conditions ("on orders above ₹199"). Coupons the cart doesn't qualify for are shown but disabled, with what's missing, such as "Add ₹120 more to apply". Applying one updates the bill and shows a toast, "'WELCOME50' applied. You saved ₹100!"

Session 2's screenshots caught that toast covering the pay bar; the fix was to give the toast provider a bottom offset, which is why the root layout in Chapter 4 has `<ToastProvider bottomOffset={120}>`.

Payment

Platter's payment step is a bottom sheet with UPI, a saved card and cash on delivery. In this book's apps, no real payment is taken; the methods are defined as demo data:

```
/** Demo payment methods (no real payment is taken). */
export const PAYMENT_OPTIONS: PaymentOption[] = [
  {
    id: 'upi',
    input: { kind: 'upi', upiId: 'riya@okaxis.example' },
    title: 'UPI',
    subtitle: 'riya@okaxis.example',
    icon: { set: 'mci', name: 'contactless-payment' },
  },
  {
    id: 'card',
    input: { kind: 'card', cardLast4: '4242' },
    title: 'Credit card',
    subtitle: 'Northbay Bank •••• 4242',
    icon: { name: 'card-outline' },
  },
  {
    id: 'cash',
    input: { kind: 'cash' },
    title: 'Cash on delivery',
```



The payment sheet, and the order confirmed. The sheet keeps UPI first, the most common way to pay in India; the tracking screen opens the moment the order is placed.

```
    subtitle: 'Pay with cash or UPI at your door',
    icon: { name: 'cash-outline' },
  },
];
```

Note the details even in demo data: a UPI ID on a `.example` domain, a card shown only by its last four digits. Chapter 23 replaces this with real payment providers (Razorpay, Stripe, Apple Pay and Google Pay), and explains why the app must never decide on its own that a payment succeeded.

Placing the Order

When the user taps "Place order", the app sends the cart, address, tip, coupon and payment method to the server. The server recalculates the bill, creates the order, starts the simulator for it, and returns the order. The app then replaces the cart screen with the tracking screen, so Back from tracking goes to the restaurant or home, not to an empty cart.

The button shows a loading state while the request is in flight and can't be tapped twice. Double orders are a classic bug in hurried apps, and a classic support headache. If you build your own app, also ask Claude to send an **idempotency key** with the request, a unique ID the server uses to recognize a retried request so it never creates the same order twice.

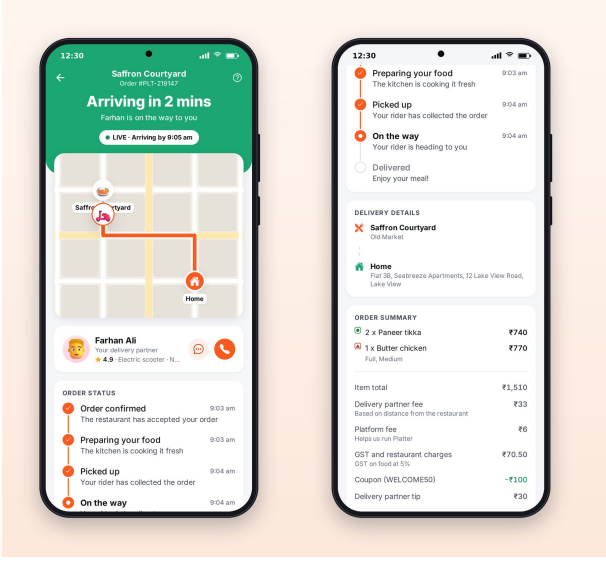
> **Try It:** Ask Claude to add a "Delivery instructions" option to Platter's address step (for example "Leave at the door", "Avoid calling", "Pet at home"), stored with the order and shown on the tracking screen. Ask for server validation, a test for the new field, and a screenshot of both screens.

Key Takeaways

- A professional cart announces savings, lists every line, offers coupons with clear conditions, invites a tip, and keeps the pay button pinned.
- The server computes every amount; the app only displays it.
- Waived fees stay visible, struck through; coupons say what they do and what's missing.
- Guard order placement against double taps and retries.

Live Order Tracking

The 30 minutes after an order is placed are the most anxious part of the food delivery experience, and the tracking screen exists to make them calm. A great tracking screen answers four questions at a glance: Is my order really happening? Where is it now? When will it arrive? Who is bringing it? This chapter shows how Platter answers them, using the live event stream from Chapter 6 and a map drawn from scratch.



Live tracking. The header gives the ETA in big type with a LIVE badge; the map shows the restaurant, the rider and home; the rider card has call and chat; the timeline shows each step with its time.

The Layout

1. **A colored header** with the restaurant, the order number, the headline ("Arriving in 1 min"), a subtitle ("Imran is on the way to you") and a pill: "LIVE · Arriving by 6:05 am". The pill reads "Connecting" when the live stream isn't open, so the user is never shown stale information as if it were live.
2. **The map**, overlapping the header's curved bottom edge.
3. **The rider card**, with avatar, name, rating, vehicle and number plate, and chat and call buttons.
4. **The order status timeline**: Order confirmed, Preparing your food, Picked up, On the way, Delivered, each with its time.
5. **The bill**, so the user can check what they paid, and a **Cancel** button while cancelling is still allowed, with a confirmation sheet.

Words That Change with the Order

The headline and subtitle change with each step, and the copy is written as a pure function:

```
export interface TrackingCopy {
  title: string;
  subtitle: string;
  /** Minutes to show in the ETA badge; null when not counting down. */
  etaMinutes: number | null;
}

const TITLES: Partial<Record<OrderStatus, string>> = {
  confirmed: 'Order confirmed',
  preparing: 'Preparing your food',
  picked_up: 'Rider has your order',
};

/** The headline and ETA for the tracking screen. `liveEta` is from the
↪ latest location event. */
export function trackingCopy(order: Pick<Order, 'status' | 'timeline' |
↪ 'etaMinutes' | 'restaurant' | 'rider'>, liveEta?: number | null):
↪ TrackingCopy {
  const p = orderProgress(order);
  const eta = Math.max(1, liveEta ?? order.etaMinutes);
  const place = order.restaurant?.name ?? 'The restaurant';
  if (p.isDelivered) return { title: 'Order delivered', subtitle: 'Enjoy
  ↪ your meal! Tell us how it was.', etaMinutes: null };
  if (p.isCancelled) return { title: 'Order cancelled', subtitle: 'Any
  ↪ amount paid will be refunded in 3 to 5 days.', etaMinutes: null };
  if (p.isMoving) {
```

```
    const who = order.rider?.name.split(' ')[0] ?? 'Your rider';
    return { title: `Arriving in ${eta} ${eta === 1 ? 'min' : 'mins'}`,
    ↪ subtitle: `${who} is on the way to you`, etaMinutes: eta };
  }
  const step = order.timeline.find((s) => s.status === order.status);
  const subtitle =
    order.status === 'confirmed'
      ? `${place} has accepted your order`
      : order.status === 'preparing'
        ? `${place} is cooking it fresh`
        : (step?.description ?? '');
  return { title: TITLES[order.status] ?? step?.label ?? 'Order placed',
  ↪ subtitle, etaMinutes: eta };
}
```

Three details are worth copying. The ETA never shows "0 mins"; it stays at least 1 until the order arrives. "1 min" is singular, "2 mins" plural. And a cancelled order says what happens to the money: "Any amount paid will be refunded in 3 to 5 days." Users read status copy like a contract.

A Map Without a Map SDK

Most delivery apps use a map SDK such as Google Maps or Mapbox. Those need API keys, billing accounts and native setup, and they don't render in a browser screenshot the same way. The project rules asked for something simpler: a map of the fictional city of Northbay, drawn with SVG, that works the same on Android, iOS and the web.

```
/**
 * A delivery map drawn with react-native-svg (no map SDK), so it
 ↪ renders on
 * Android, iOS and web. Shows the route, pins and a rider that glides to
 * each new `progress` value.
 */
export function MapView({
  route = [],
  progress = null,
  markers = [],
  riderArt = 'motor-scooter',
  aspectRatio = 1.25,
  padding = 80,
  radius = radii.lg,
  accessibilityLabel,
  style,
  children,
}: MapViewProps) {
  const theme = useTheme();
  const { colors } = theme;
  const [size, setSize] = useState({ width: 0, height: 0 });

  const box = useMemo(
```

```
    () => fitViewBox([...route, ...markers.map((mk) => mk.point)],
    ↪ aspectRatio, padding),
    [route, markers, aspectRatio, padding],
  );
  const showRider = progress != null && route.length > 1;
  const [done, rest] = useMemo(
    () => (showRider ? splitRoute(route, progress) : [[], [...route]]),
    [route, progress, showRider],
  );

  const riderX = useSharedValue(0);
  const riderY = useSharedValue(0);
  const riderTarget = showRider && size.width > 0 ?
  ↪ toPixels(pointAlongRoute(route, progress), box, size) : null;
  const tx = riderTarget?.x;
  const ty = riderTarget?.y;
  useEffect(() => {
    if (tx == null || ty == null) return;
    const first = riderX.get() === 0 && riderY.get() === 0;
    const timing = { duration: first ? 0 : 900, easing: Easing.linear };
    riderX.set(withTiming(tx - RIDER / 2, timing));
    riderY.set(withTiming(ty - RIDER / 2, timing));
  }, [tx, ty, riderX, riderY]);
  const riderStyle = useAnimatedStyle(() => ({
    transform: [{ translateX: riderX.value }, { translateY: riderY.value
    ↪ }],
  }));

  const pct = showRider ? Math.round(Math.min(1, Math.max(0, progress))
  ↪ * 100) : null;
  const label =
    accessibilityLabel ?? (pct != null ? `Delivery map. Rider is ${pct}%
    ↪ of the way to you.` : 'Delivery map');
```

The server sends the rider's route as a list of points along Northbay's street grid, and a `progress` value from 0 to 1. The map splits the route into the part already travelled and the part ahead, draws them in different styles, and moves the rider with a 900-millisecond linear animation to each new position, so one update per second looks like smooth motion. The first position is set instantly, so the rider doesn't slide in from the corner when the screen opens.

The map is also a single accessible element with a meaningful label: "Delivery map. Rider is 47% of the way to you." A screen reader user gets the same information a sighted user does, in one sentence.

> **Note:** For a real app, you'll use a real map. The `MapView` component's props, a route, a progress value and markers, are a good contract to keep: build a version on Google Maps or Mapbox with the same props, and the tracking screen won't need to change. Chapter 20 covers keeping map rendering smooth.

Keeping It Live

The tracking screen gets everything from one hook:

```
const { order, rider, connected, error } = useOrderStream(id);
```

As you saw in Chapter 6, `useOrderStream` opens the server's event stream, puts every status change into the shared cache, and exposes the rider's latest position. When the order is delivered or cancelled, the stream closes itself. When the screen unmounts, the hook closes the stream, so a user leaving the screen doesn't leave a connection open.

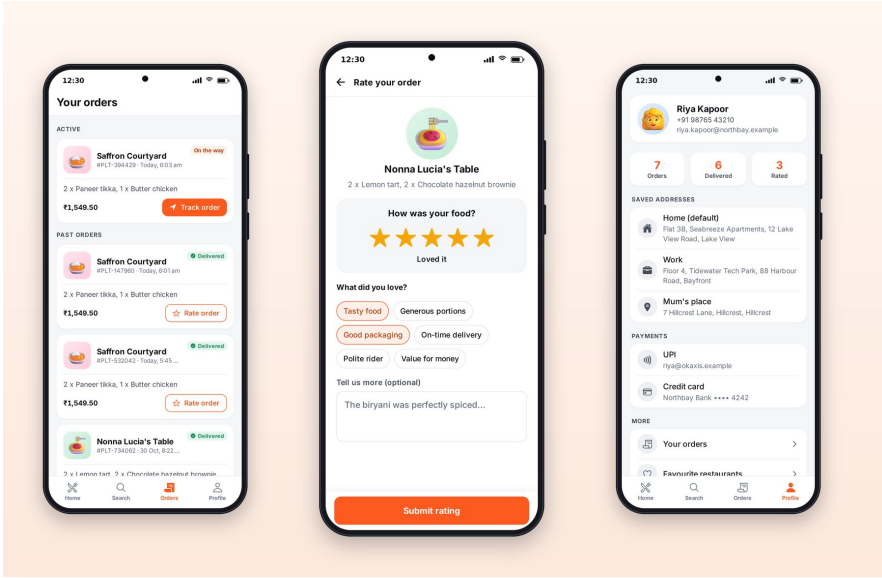
What happens on a flaky network? The SSE helper reconnects, and the server always sends a fresh snapshot of the order first, so the screen catches up even if it missed events while disconnected. Meanwhile the pill says "Connecting", and the last known state stays on screen rather than a spinner.

After Delivery

When the order is delivered, the tracking screen offers "Rate your order". The rating screen asks for stars first, then offers quick tags, which are much faster to tap than writing, and an optional comment. The orders tab keeps active orders at the top with a Track button, and past orders below with Rate or a rating already given. The profile has the user's stats, saved addresses, payment methods and sign-out with a confirmation.

How Platter Was Checked

Session 2 checked tracking with a Playwright script that signed in, added dishes, applied a coupon, added a tip, placed an order and then photographed the tracking screen about 75 seconds later, with the rider part-way along the



After delivery: the orders list with Track and Rate actions, the rating screen with stars, quick tags and a comment, and the profile.

route. Its tests cover the tracking copy at every step, the live update path with a fake event stream, cancellation, and the rating flow. Session 2 also reported that a tracking test passed alone but failed in the full suite, and fixed it before finishing.

Key Takeaways

- A tracking screen answers: is it happening, where is it, when will it arrive, who is bringing it.
- Write status copy as a pure function and test every step; small details like "1 min" and refund wording matter.
- An SVG map with a route, progress and markers renders everywhere; keep the same props when you switch to a real map SDK.
- Live updates should degrade gracefully: say "Connecting", keep the last known state, and resync with a snapshot.

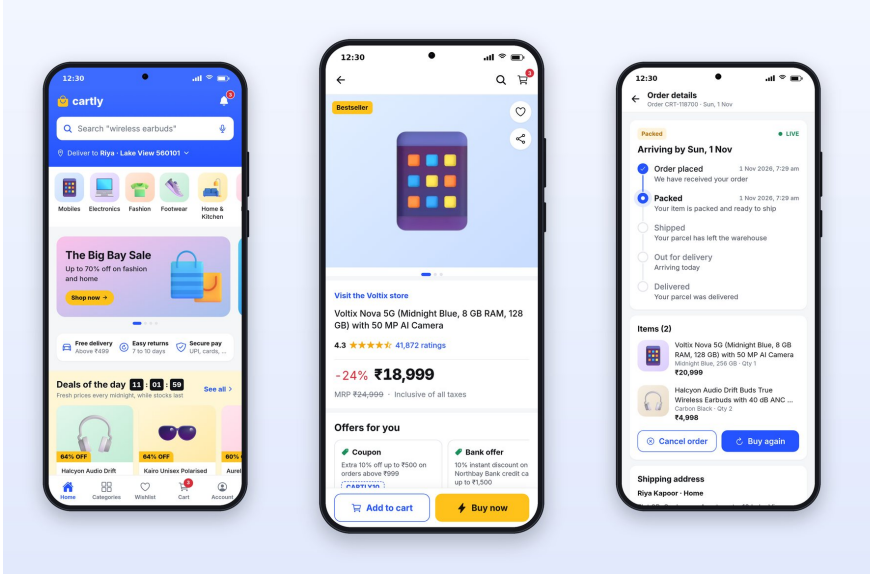
PART III

Cartly, E-Commerce

CHAPTER 11 · E-COMMERCE

Planning a Store App

Shopping apps are information-dense in a way few other apps are. A single product card carries an image, a title, a rating with a count, a price, a struck-through maximum retail price, a discount percentage and sometimes a delivery promise, and a home screen stacks dozens of them under banners, categories and deals. The challenge is to make all that density easy to scan and to build trust at every step toward "Buy now". This part builds Cartly.



Cartly: the home screen with categories, a sale banner and countdown deals; a product page with offers and a sticky buy bar; and an order's live shipment tracking.

The Patterns Users Expect

The PRD's section on Cartly lists them, in the category's vocabulary:

- **A branded header** with search ("Search 'wireless earbuds'"), a microphone, and "Deliver to Riya · Lake View 560101": the delivery pincode is always visible because prices and dates depend on it.
- **A category strip** with illustrated tiles: Mobiles, Electronics, Fashion, Footwear, Home and Kitchen.
- **Hero banners** for sales, and trust badges: free delivery above a threshold, easy returns, secure payment.
- **Deals of the day** with a live countdown, "Fresh prices every midnight, while stocks last".
- **Product cards** with a discount badge, rating, price, MRP strike-through and a wishlist heart.
- **Search with suggestions** as you type, recent and trending searches, and highlighted matches.
- **Filter and sort sheets**: price, brand, rating, discount; relevance, price low to high, newest.
- **A product page** with a swipeable gallery, the brand store link, title, rating summary, price block, variant pickers, a pincode delivery check, offers, highlights, reviews, and a sticky "Add to cart" / "Buy now" bar.
- **Cart with "Save for later"**, checkout with address and payment, an order confirmation, order details with shipment steps, and returns.

The Prompt

Session 3's prompt for Cartly:

```
Build Cartly, the e-commerce store in docs/PRD.md section 2, as
apps/cartly: an Expo SDK 58 app with Expo Router, using @pro/ui (brand
"cartly") and @pro/api. Look at apps/platter first and follow the same
structure and conventions (tests in src/test, export:web script,
screenshots folder), and reuse the shared components instead of copying.

It should feel like the best shopping apps: information-dense product
cards with ratings, MRP strike-through and discount badges, deals with
↪ live
countdowns, fast search with suggestions, filter and sort sheets, a
product page with an image gallery and a sticky buy bar, and a checkout
```

```
that builds trust (delivery date, offers, secure payment note).

Screens: Home, Search with suggestions, Product listing with
filters/sort, Product detail (gallery, variants, pincode delivery
check, offers, highlights, reviews), Cart (save for later), Checkout
(address, payment sheet), Order confirmation, Orders and Order details
↪ with
shipment tracking steps, Return request flow, Wishlist, Notifications,
Account. Tabs: Home, Categories, Wishlist, Cart (with count badge),
↪ Account.

As before: plan first and add missing shared pieces to @pro/ui or @pro/api
with tests; build screen by screen and check each one in headless Chromium
at 393x852 CSS pixels, device scale 3, against the PRD, fixing anything
unpolished; add logic and screen tests; save final screenshots of every
screen to apps/cartly/screenshots/ (01-home.png, ...). Finish with
typecheck, lint and tests passing for the whole workspace, and a summary.
```

It's shorter than Platter's because the method was now established: "Look at apps/platter first and follow the same structure and conventions … and reuse the shared components instead of copying." The prompt spends its words on what's specific to shopping: density, deals with live countdowns, fast search, filter and sort sheets, the gallery, the sticky buy bar, and trust in checkout.

When a Session Runs Out of Time

Session 3 is the book's lesson in what happens when a milestone is too big. The session built all of Cartly's screens and captured 24 screenshots, then ran past the two-hour limit of the job running it and was stopped before it reported back. When it stopped, it was halfway through a refactor: moving the address, coupon and payment sheets and the sign-in session out of Platter into the shared packages, so both apps could use one copy. Platter's files had been edited, some deleted, and nothing had been checked or committed.

This is exactly the situation the review loop exists for. The recovery took three steps:

1. **Look before acting.** `git status` and `git diff` showed precisely what the session had changed: Cartly's new folder, Platter's edited screens, and the moved components.
2. **Write a focused follow-up prompt** that explains the situation and asks for a finish, not a restart:

```
The previous session built Cartly (apps/cartly) from docs/PRD.md section 2
and was cut off by a time limit before it finished. It had also started
moving shared pieces out of apps/platter into @pro/ui and @pro/api (see
`git status` and `git diff`); that refactor may be half done.

Finish the job:
1. Review the uncommitted changes and complete the refactor so Platter and
   Cartly both use the shared pieces, with nothing duplicated and nothing
   left broken.
2. Run typecheck, lint and tests for the whole workspace and fix every
   problem.
3. Re-export both apps for web and recapture any screenshot in
   apps/platter/screenshots or apps/cartly/screenshots whose screen
   ↪ changed.
4. Write a summary of what Cartly contains, what moved into the shared
   packages, and anything still missing from the PRD.
```

1. **Verify independently.** The follow-up session was itself cut off, this time by Claude Code's five-hour usage limit, after 32 minutes. Its last message was simply the limit notice. So the work was verified directly: `npm run check` passed across the whole workspace, both apps built for the web, and the screenshots were current. Only then was it committed.

> **Tip:** Size sessions so they finish well within your limits, and commit after every successful session. A session that stops halfway leaves the working tree in an in-between state; with a clean commit behind it, you can always see exactly what changed and decide whether to finish it or throw it away with `git restore`.

What Moved to the Shared Packages

The refactor is worth understanding, because it's what keeps a family of apps consistent. After session 3:

- `AddressSheet`, `CouponSheet` and `PaymentSheet` live in `@pro/ui`, used by Platter, Cartly and later Minutely.
- The signed-in session (token storage, sign-in state, sign-out) lives in `@pro/api` as `SessionProvider`, the provider you saw in Platter's root layout in Chapter 4.
- Phone number formatting, debouncing and the query error component moved too.

Each app now holds only what's unique to it. When you review an AI's work on a second or third app, check for this. The natural tendency, for people and for AI agents alike, is to copy what worked in the first app; the professional move is to share it.

Key Takeaways

- Store apps are dense; scannability (consistent cards, clear price blocks) and trust (delivery pincode, badges, honest totals) are the design goals.
- Later app prompts can be shorter: point to the first app's conventions and spend words on what's new.
- When a session is cut off, look at `git status` and `git diff`, write a focused finishing prompt, and verify independently before committing.
- Move shared pieces into shared packages as soon as a second app needs them.

CHAPTER 12 · E-COMMERCE

Home, Search, Filters and Product Pages

This chapter walks through how Cartly helps people find a product and decide to buy it: the home screen, search, listings with filters, and the product page.

The Home Screen

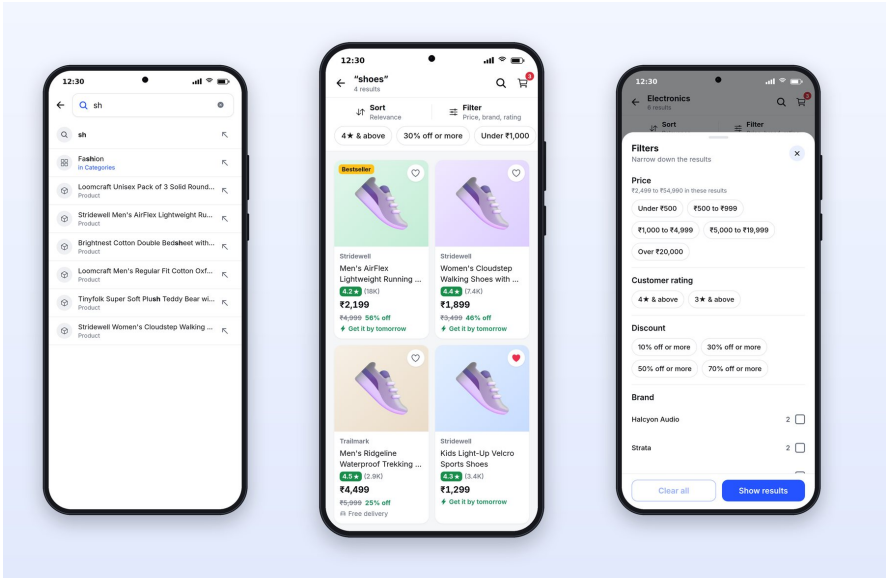


Cartly's home screen, the deals of the day, and the full category list.

Cartly's home screen leads with its brand color and three things a shopper needs before anything else: search, the delivery location and the categories. Then it sells: a hero banner, trust badges, and "Deals of the day" with a live countdown in boxes, "11 : 01 : 59".

A countdown is a strong motivator, and it must be honest. Cartly's deals end at midnight, the countdown is computed from the server's end time, and it ticks once a second from the shared `CountdownTimer` component. Avoid fake urgency, such as countdowns that reset when the app reopens; users notice, and app stores and consumer law in many countries treat it as a deceptive pattern.

Search



Search with suggestions as you type, a product listing with filters, and the filter sheet.

Search is where shopping apps win or lose customers. Cartly's search screen shows recent searches and trending searches before you type, then suggestions of four kinds as you type: products, categories, brands and plain queries, with the matching text in bold. The logic is small and testable:

```
/** Adds a search to the front of the recent list (deduplicated,
↪ capped). */
export function addRecent(list: readonly string[], term: string, max =
↪ MAX_RECENT): string[] {
  const t = term.trim();
  if (!t) return [...list];
  return [t, ...list.filter((x) => x.toLowerCase() !==
  ↪ t.toLowerCase())].slice(0, max);
}

/** Splits text around the first case-insensitive match of `query`, for
↪ bold highlights. */
export function highlightParts(text: string, query: string): { text:
↪ string; match: boolean }[] {
  const q = query.trim();
  const at = q ? text.toLowerCase().indexOf(q.toLowerCase()) : -1;
  if (at < 0) return [{ text, match: false }];
  return [
    { text: text.slice(0, at), match: false },
    { text: text.slice(at, at + q.length), match: true },
    { text: text.slice(at + q.length), match: false },
  ].filter((p) => p.text.length > 0);
}

/** Where tapping a suggestion goes. */
export function suggestionHref(s: SearchSuggestion): AppHref {
  switch (s.kind) {
    case 'product':
      return s.id ? `/product/${s.id}` : { pathname: '/products',
      ↪ params: { q: s.text } };
    case 'category':
      return { pathname: '/products', params: s.id ? { category: s.id }
      ↪ : { q: s.text } };
    case 'brand':
      return { pathname: '/products', params: { brand: s.text } };
    default:
      return { pathname: '/products', params: { q: s.text } };
  }
}
```

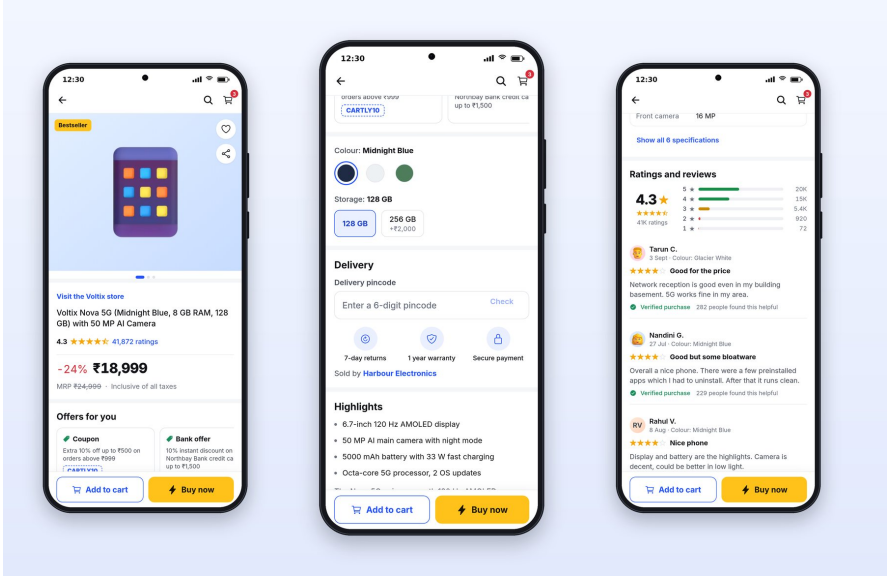
`addRecent` puts the newest search first, removes case-insensitive duplicates and caps the list at eight. `highlightParts` splits a suggestion around the match for bold display. `suggestionHref` decides where each kind of suggestion goes: a product opens its page, a category or brand opens a filtered listing.

Listings, Filters and Sort

The listing shows results in a two-column grid of product cards. Above it, a bar shows the result count and buttons for Sort and Filter, each opening a bottom sheet. Applied filters appear as chips you can remove one by one. The filter sheet groups its options into sections (price, brands with a count beside each, rating, discount) and ends with two buttons: "Clear all" and "Show results". Changes are kept as a draft until "Show results", so a user can experiment and back out without disturbing the listing.

Bottom sheets are the right component here. They keep the listing visible behind them, so the user doesn't lose their place, and they're easy to dismiss with a swipe down.

The Product Page



The product page, variants with a pincode delivery check, and reviews.

The product page is the most important screen in any store app, and it follows a well-established order:

1. **Gallery** with dots, a Bestseller badge, wishlist and share buttons.
2. **"Visit the Voltix store"**, the brand link.

3. **Title** with the key variant in brackets: "Voltix Nova 5G (Midnight Blue, 8 GB RAM, 128 GB) with 50 MP AI Camera".
4. **Rating** with stars and the number of ratings, linking to reviews.
5. **Price block**: "-24%" in red, the price large and bold, "MRP ₹24,999" struck through, "Inclusive of all taxes".
6. **Offers**: coupons and bank offers in swipeable cards.
7. **Variants**: color and storage pickers, with unavailable options shown but disabled.
8. **Delivery**: a pincode field and the estimated delivery date for that pincode.
9. **Highlights** and **reviews**, with a rating breakdown and the most helpful reviews first.
10. **A sticky bar**: "Add to cart" and "Buy now", always visible.

The variant logic shows how to keep that page honest:

```
/** The first available option of every group. */
export function defaultSelection(groups: readonly VariantGroup[]):
↪ Selection {
  const sel: Selection = {};
  for (const g of groups) {
    const o = g.options.find((x) => x.available) ?? g.options[0];
    if (o) sel[g.kind] = o.id;
  }
  return sel;
}

function chosen(groups: readonly VariantGroup[], sel: Selection) {
  return groups.map((g) => g.options.find((o) => o.id ===
  ↪ sel[g.kind])).filter((o) => o != null);
}

/** Price and MRP with the chosen options' price deltas added. */
export function selectionPrice(base: { price: number; mrp: number },
↪ groups: readonly VariantGroup[], sel: Selection): { price: number; mrp:
↪ number } {
  const delta = chosen(groups, sel).reduce((s, o) => s + o.priceDelta, 0);
  return { price: base.price + delta, mrp: base.mrp + delta };
}

/** "Midnight Blue, 256 GB". */
export function selectionLabel(groups: readonly VariantGroup[], sel:
↪ Selection): string {
  return chosen(groups, sel)
    .map((o) => o.label)
    .join(', ');
}

/** Every group has an option picked and all picks are in stock. */
```

```
export function isSelectionAvailable(groups: readonly VariantGroup[],
↪ sel: Selection): boolean {
  return groups.every((g) => {
    const o = g.options.find((x) => x.id === sel[g.kind]);
    return o != null && o.available;
  });
}

/** The cart API's `variant` field. */
export function toVariantInput(groups: readonly VariantGroup[], sel:
↪ Selection): SetCartItemInput['variant'] {
  if (!groups.length) return undefined;
  const out: NonNullable<SetCartItemInput['variant']> = {};
  for (const g of groups) if (sel[g.kind]) out[g.kind] = sel[g.kind];
  return out;
}
```

The default selection is the first option that's in stock, not just the first option. The price updates with each variant's price difference. And the buy buttons check `isSelectionAvailable`, so an out-of-stock combination can't be added to the cart. Every price on the screen, including the struck-through MRP and the discount, comes from the shared `Price` component:

```
/** "₹249  ₹–2–9–9–  17% off", with tabular figures and one spoken
↪ label. */
export function Price({ amount, mrp, size = 'md', showDiscount = true,
↪ stacked = false, style, testID }: PriceProps) {
  const off = discountPercent(amount, mrp);
  const hasMrp = off > 0 && mrp != null;
  const v = variants[size];
  const label = hasMrp
    ? `Price ${formatINR(amount)}, MRP ${formatINR(mrp)}${showDiscount ?
    ↪ `, ${off}% off` : ''}`
    : `Price ${formatINR(amount)}`;

  const meta = hasMrp ? (
    <View style={styles.row}>
      <Text variant={v.meta} color="textMuted" strike tabular>
        {formatINR(mrp)}
      </Text>
      {showDiscount ? (
        <Text variant={v.meta} weight={700} color="success" tabular>
          {off}% off
        </Text>
      ) : null}
    </View>
  ) : null;

  return (
    <View
      testID={testID}
      accessible
      accessibilityRole="text"
      accessibilityLabel={label}
      style={[stacked ? styles.stack : styles.row, style]}
```

```
    >
      <Text variant={v.price} weight={size === 'sm' ? 600 : 700} tabular>
        {formatINR(amount)}
      </Text>
      {meta}
    </View>
  );
}
```

Two professional touches: prices use tabular figures, so digits line up in lists, and the whole price block is one accessible element with one spoken label, "Price ₹18,999, MRP ₹24,999, 24% off", instead of three separate fragments a screen reader would read out of context.

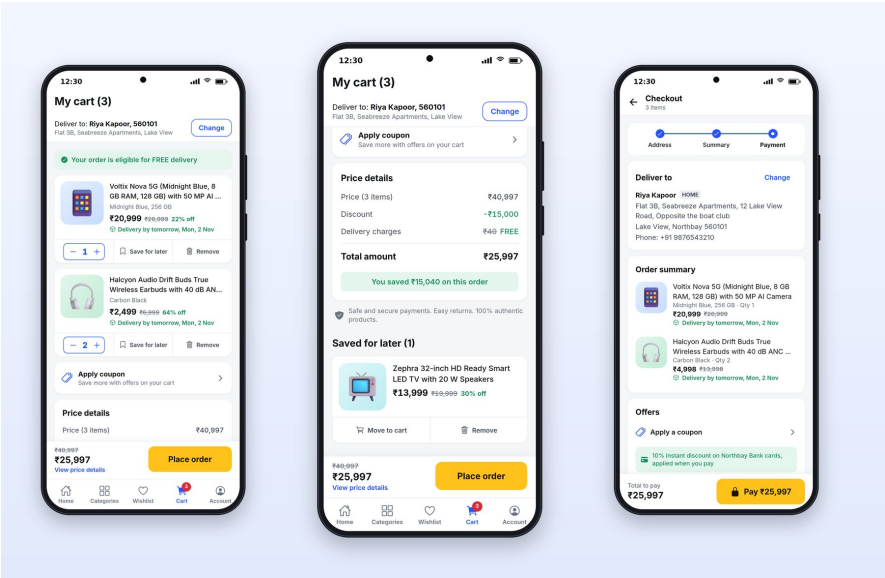
Key Takeaways

- Lead the home screen with search, delivery location and categories; sell with banners, trust badges and honest deals.
- Search needs recents, trending, typed suggestions of several kinds and highlighted matches; keep the logic in tested functions.
- Filters and sort belong in bottom sheets; show applied filters as removable chips.
- A product page follows a familiar order, ending in a sticky buy bar; default variants to what's in stock and block unavailable combinations.
- Format prices in one component with tabular figures and one spoken label.

CHAPTER 13 · E-COMMERCE

Cart, Payments and Orders

A shopping cart is not a food cart. It holds items from many sellers, often for days, and people treat it as a shortlist with a checkout button. This chapter covers how Cartly handles the cart, checkout, payment and the life of an order after it's placed.



The cart, its price details and "Saved for later", and checkout. Every line shows its own delivery date; the savings are spelled out; checkout shows its three steps at the top.

A Cart People Leave Things In

From top to bottom, Cartly's cart shows:

1. **"Deliver to: Riya Kapoor, 560101"** with the address and a Change button, because delivery dates depend on it.
2. **A good-news banner:** "Your order is eligible for FREE delivery".
3. **Each item** with its variant, price, MRP and discount, its own delivery date ("Delivery by tomorrow, Mon, 2 Nov"), a stepper, **Save for later** and **Remove**.
4. **Apply coupon.**
5. **Price details:** price, discount, delivery charges (struck through, "FREE"), the total, and a green band: "You saved ₹15,040 on this order".
6. **A trust line:** "Safe and secure payments. Easy returns. 100% authentic products."
7. **Saved for later**, with Move to cart and Remove.
8. **A sticky bar** with the MRP struck through, the total, "View price details", and a yellow "Place order" button.

Save for later is the feature food carts don't need. People use it to decide between two phones, to wait for payday, or to watch a price. It keeps items out of the total without losing them. And the savings band is the most persuasive number on the screen: it turns a ₹25,997 total into a ₹15,040 win.

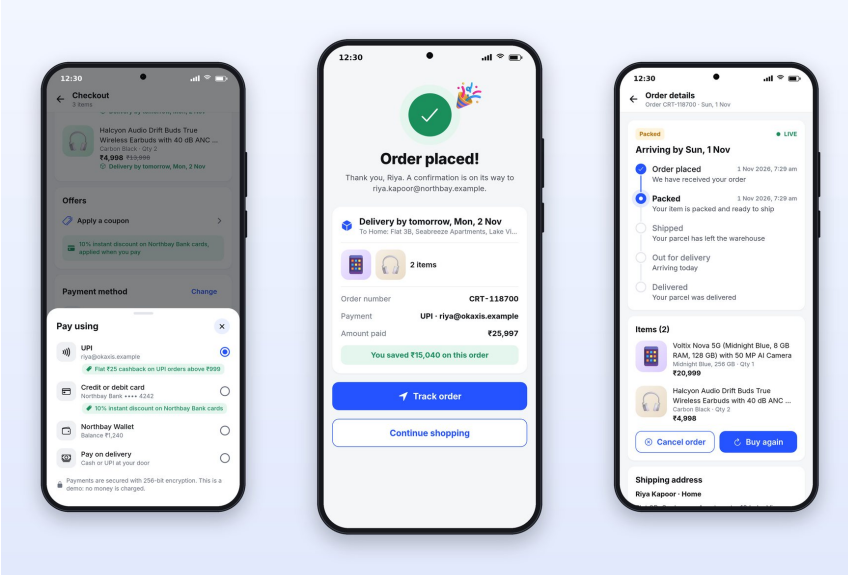
Quantity steppers respect each product's limit; the server rejects more with a specific message ("You can add up to 5 of …"), and the app shows it as a toast instead of failing silently.

Checkout

Checkout is one scrolling screen with a step indicator at the top (Address, Summary, Payment), so the user knows exactly how far they are from done. It confirms the delivery address, lists each item with its delivery date, shows the offers that apply ("10% instant discount on Northbay Bank cards, applied when you pay"), and ends with the payment method and a sticky bar: "Total to pay ₹25,997" and a "Pay ₹25,997" button with a padlock.

Putting the amount on the button is a small, powerful detail. The user knows exactly what tapping it will do.

Payment, Confirmation and Order Details



The payment sheet, the order confirmation, and the order details with live shipment steps.

The payment sheet lists UPI, a card, a wallet with its balance, and pay on delivery. Each method can carry its own offer ("Flat ₹25 cashback on UPI orders above ₹999"), and the sheet ends with a plain statement of what happens: "Payments are secured with 256-bit encryption. This is a demo: no money is charged." In your real app, the second sentence goes; Chapter 23 replaces the demo with real payment providers.

The confirmation celebrates: a check mark, a party popper, "Order placed!", and "Thank you, Riya. A confirmation is on its way to riya.kapoor@northbay.example." It shows the delivery date and address, the items, the order number, the payment method, the amount paid and the savings, then two clear next steps: **Track order** and **Continue shopping**.

Order details becomes the home of the order: a status badge and LIVE indicator, "Arriving by Sun, 1 Nov", the shipment steps (Order placed, Packed, Shipped, Out for delivery, Delivered) with their times, the items, the

shipping address, and actions that fit the status. Before shipping, that's **Cancel order** and **Buy again**; after delivery, it becomes a return.

The steps update live over the same event stream Platter uses for its rider, from the same shared `ProgressSteps` component as Platter's timeline, with labels from the server. Cartly's demo schedule takes an order from placed to delivered in about three minutes.

Key Takeaways

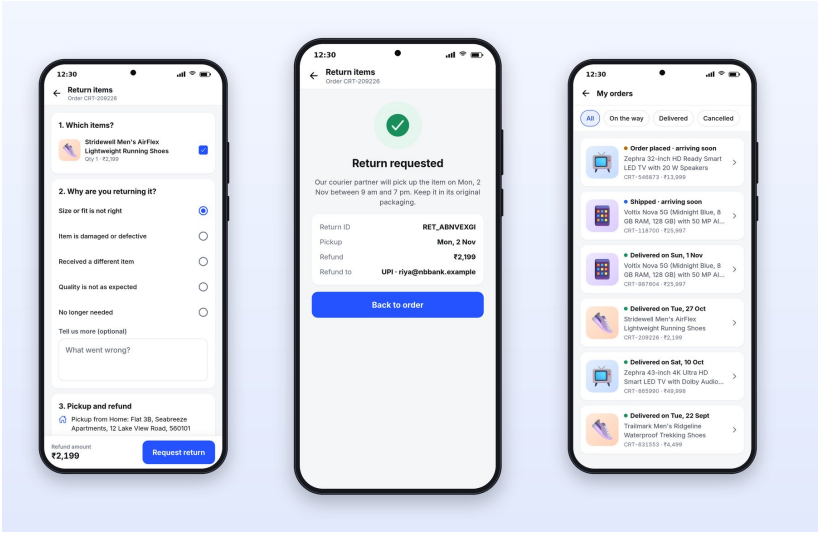
- A shopping cart is a waiting room: support save for later, show each item's delivery date, and spell out the savings.
- Show server errors such as quantity limits as friendly messages, never silent failures.
- Show checkout progress with steps, and put the amount on the pay button.
- After purchase, celebrate, give two clear next steps, and make order details the live home of the order.

CHAPTER 14 · E-COMMERCE

Returns, Wishlist and Notifications

The screens in this chapter are the ones demo apps skip and real apps can't: returns, the orders list, the wishlist, notifications and the account. They don't sell anything directly, but they decide whether a customer buys a second time.

Returns



A return request in three numbered steps, the confirmation with a pickup window, and the orders list with status filters.

A good return flow is quick and leaves no doubt about what happens next. Cartly's is one screen in three numbered steps:

1. **Which items?** Each item in the order with a checkbox.
2. **Why are you returning it?** Five reasons, and an optional "Tell us more" box.
3. **Pickup and refund:** where the courier will collect it from, and where the money will go.

A sticky bar shows the refund amount and a "Request return" button. The rules behind the screen are pure functions:

```
import type { Order } from '@pro/api';

export const RETURN_REASONS = [
  'Size or fit is not right',
  'Item is damaged or defective',
  'Received a different item',
  'Quality is not as expected',
  'No longer needed',
] as const;

/** Refund for the chosen lines (the full line totals). */
export function refundTotal(order: Pick<Order, 'items'>, lineIds:
↪ readonly string[]): number {
  const ids = new Set(lineIds);
  return order.items.filter((l) => ids.has(l.id)).reduce((s, l) => s +
  ↪ l.lineTotal, 0);
}

export function canSubmitReturn(lineIds: readonly string[], reason:
↪ string | null): boolean {
  return lineIds.length > 0 && !!reason;
}

export function toggleId(ids: readonly string[], id: string): string[] {
  return ids.includes(id) ? ids.filter((x) => x !== id) : [...ids, id];
}

export const RETURN_STATUS_LABEL:
↪ Record<Order['returns'][number]['status'], string> = {
  requested: 'Return requested',
  pickup_scheduled: 'Pickup scheduled',
  refunded: 'Refund completed',
};
```

The reasons are the ones that matter to the business (fit, damage, wrong item, quality, no longer needed), short enough to scan and specific enough to act on. The refund is calculated from the items selected and shown before the

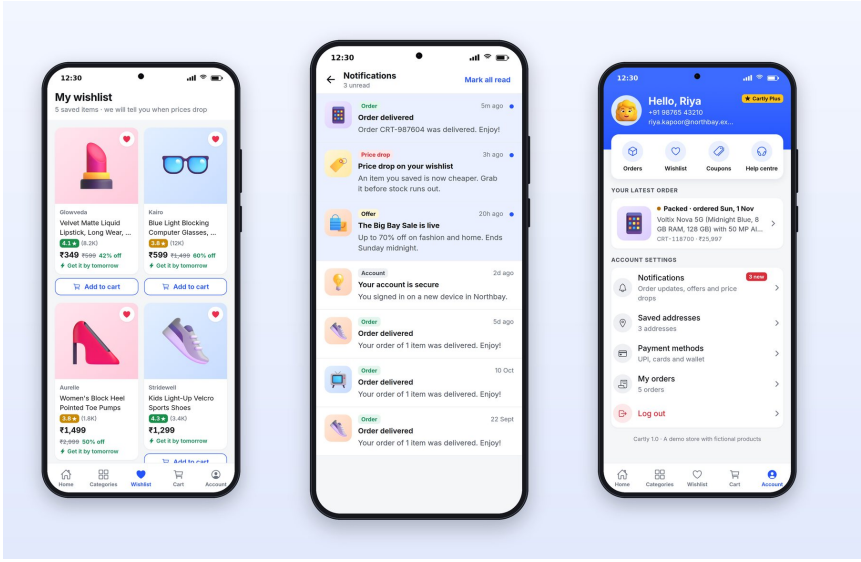
user submits, and the button stays disabled until at least one item and a reason are chosen.

The confirmation answers every question a customer has: "Our courier partner will pick up the item on Mon, 2 Nov between 9 am and 7 pm. Keep it in its original packaging." Then the return ID, the pickup date, the refund amount and where it's going ("UPI · riya@nbbank.example"), and one button: **Back to order**. Return statuses are written in plain words, "Return requested", "Pickup scheduled", "Refund completed", never internal codes.

The Orders List

The orders list opens with filter chips (All, On the way, Delivered, Cancelled), then one card per order with a colored status line ("Shipped · arriving soon", "Delivered on Sun, 1 Nov"), the product, the order number and the amount. It's the screen people open most after buying, so its data shows instantly from the cache and refreshes in the background, which React Query does by default.

Wishlist, Notifications and Account



The wishlist, notifications with unread markers, and the account screen.

The wishlist promises something useful in its subtitle: "5 saved items · we will tell you when prices drop". Each card has the product, rating, price, discount, delivery promise and an "Add to cart" button. Tapping a heart anywhere in the app updates the wishlist instantly, with an optimistic update that rolls back if the server refuses.

Notifications show a count of unread ones in the header ("3 unread") and a "Mark all read" action. Each has a colored type label (Order, Price drop, Offer, Account), a time, and an unread dot. Security notices belong here too: "Your account is secure. You signed in on a new device in Northbay." Each notification is a deep link; tapping an order update opens that order. Chapter 24 shows how to deliver them as push notifications when the app is closed.

The account screen greets the user by name ("Hello, Riya") with their phone, email and membership badge, then quick tiles (Orders, Wishlist, Coupons, Help centre), the latest order, and settings: notifications with a "3 new" badge, saved addresses, payment methods, orders, and **Log out** in red. A small footer says what the app is: "Cartly 1.0 · A demo store with fictional products."

Logging out clears the stored token and every cached query, so the cart badge, orders and wishlist disappear with it, and the next person to use the phone sees nothing of the last. Platter asks "Log out?" first; Cartly logs out straight away. Both are common, and asking is kinder when logging back in takes a one-time code.

> **Try It:** Ask Claude to add a "Price drop alert" bell to Cartly's product page: it saves the current price and, through the notifications list, tells the user when the price falls. Ask for the server route, the client hook with an optimistic update, a test for the server rule, and a screenshot.

Key Takeaways

- Returns take one screen in numbered steps: items, a short list of reasons, pickup and refund, with the refund amount shown before submitting.
- The confirmation should answer every next question: when, where, how much, and to which account.
- Give the orders list status filters and plain-word statuses.

- Wishlist toggles are optimistic; notifications carry types, unread state and deep links; the account screen gathers everything personal.
- Logging out clears the token, the cache and every trace of the user.

PART IV

Lumo, Social Photos

CHAPTER 15 · SOCIAL

Planning a Social App

Social apps are the opposite of commerce apps. Instead of dense information, they show one thing at a time, full width, and get everything else out of the way. The chrome is quiet, black and white with a single accent; the content is loud. And the quality is in the motion: a story bar filling, a heart bursting, a sheet sliding up for comments. This part builds Lumo.



Lumo: the feed with a double-tap heart, a story in progress, and a profile.

The Patterns Users Expect

The PRD for Lumo, in the category's vocabulary:

- **A stories row** at the top of the feed: round avatars with a gradient ring for new stories and a gray ring for seen ones, "Your story" first with a plus badge.
- **Posts**: avatar, username and location; a square photo; like, comment, share and save icons; "Liked by kabir.trails and 5,406 others"; the caption with the username in bold; "View all 3 comments"; a couple of comments inline.
- **Double-tap to like**, with a big heart that pops over the photo.
- **A story viewer**: full screen, progress bars at the top, tap right for next, tap left for previous, hold to pause, a reply box at the bottom.
- **A comments sheet** with quick emoji reactions.
- **Create**: pick, edit, caption, tag, share.
- **Explore**: a grid with some large tiles, and search for people.
- **Profile**: avatar, posts, followers and following counts, name, bio, link, highlights, and grid and tagged tabs.
- **Activity**: likes, comments and follows grouped by time.
- **Messages**: a conversation list with unread dots and a chat thread.

Photos Without Photos

A social app needs photos, and this book can't use real people's photos. The prompt solved it with a new shared component:

> The "photos" are rich illustrated compositions: layered gradients plus the 3D art from @pro/ui (a scene component in @pro/ui that composes a backdrop, a hero illustration and small accents from a post's seed, so every post looks distinct).

The result is `PhotoScene`. Each post in the seed data names a tone and a few illustrations; the component draws a backdrop of layered gradients, a soft light, bokeh circles, a horizon and sparkles, all chosen deterministically from the post's ID, and places the illustrations on top:

```
export interface PhotoSceneProps {
  scene: { tone: ArtToneName; layers: readonly ArtLayer[] };
  /** Seeds the backdrop (glow, bokeh, horizon, sparkles). Use the post
  ↪ or story id. */
  seed: string;
  filter?: PhotoFilterName;
  /** width / height. Defaults to 1 (square); stories use 9 / 16. */
  aspectRatio?: number;
  radius?: number;
  /** Fewer details for small grid tiles. */
  compact?: boolean;
  accessibilityLabel?: string;
  style?: StyleProp<ViewStyle>;
  children?: ReactNode;
}

/** A four-point sparkle, centered on 0,0 with radius 1. */
const SPARKLE = 'M0 -1 C0.12 -0.12 0.12 -0.12 1 0 C0.12 0.12 0.12 0.12 0
↪ 1 C-0.12 0.12 -0.12 0.12 -1 0 C-0.12 -0.12 -0.12 -0.12 0 -1 Z';

/**
 * A Lumo "photo": the scene's hero and accents over a seeded backdrop of
 * layered gradients, a soft light, bokeh, a horizon and sparkles,
 ↪ finished
 * with the photo filter. Photos keep their colors in dark mode.
 */
export function PhotoScene({ scene, seed, filter = 'normal', aspectRatio
↪ = 1, radius = 0, compact, accessibilityLabel, style, children }:
↪ PhotoSceneProps) {
  const uid = `p${useId().replace(/[^a-zA-Z0-9_-]/g, '')}`;
  const [size, setSize] = useState({ width: 0, height: 0 });
  const width = size.width;
  // Stretched scenes (a story filling the screen) use their real height.
  const height = size.height > 0 ? size.height : width / aspectRatio;
  const backdrop = useMemo(() => photoBackdrop(seed, scene.tone), [seed,
  ↪ scene.tone]);
  const stops = artTones[scene.tone].light;
  const spec = photoFilters[filter];
  const hero = scene.layers[0];
  const label = accessibilityLabel ?? `Photo: ${scene.layers.map((l) =>
  ↪ humanize(l.art)).join(', ')}`;
```

The same post always looks the same, every post looks different, and the scenes work at feed size, as small grid tiles (with `compact` dropping the fine details) and as full-screen stories. In your own app, `PhotoScene` is replaced by an image component loading the user's photo.

The Prompt

```
Build Lumo, the social photo app in docs/PRD.md section 3, as apps/lumo:
an Expo SDK 58 app with Expo Router, using @pro/ui (brand "lumo") and
@pro/api. Follow the structure of apps/platter and apps/cartly and reuse
↪ the
shared packages (session, sheets, components) instead of copying.

It should feel like the best photo-sharing apps: content first, quiet
chrome, crisp icons, smooth gestures. The "photos" are rich illustrated
compositions: layered gradients plus the 3D art from @pro/ui (a scene
component in @pro/ui that composes a backdrop, a hero illustration and
small accents from a post's seed, so every post looks distinct).

Screens: Feed (stories row with unseen/seen rings, posts with double-tap
like and heart burst, like/comment/share/save, captions, comment count),
Story viewer (progress bars, tap left/right, hold to pause), Comments
sheet, Create post (pick art, filter strip, caption, share), Explore grid
with mixed tile sizes, Profile (stats, bio, highlights, grid and tagged
tabs), Notifications, Messages list and a chat thread. Tabs: Home,
Explore, Create, Activity, Profile.

As before: plan first and add missing shared pieces to @pro/ui or @pro/api
with tests; build screen by screen and check each one in headless Chromium
at 393x852 CSS pixels, device scale 3, fixing anything unpolished; add
logic and screen tests; save final screenshots of every screen to
apps/lumo/screenshots/ (01-feed.png, ...), with the story viewer captured
mid-story and a heart burst captured on a post if you can. Keep the work
focused so it fits in one session of about an hour. Finish with typecheck,
lint and tests passing for the whole workspace, and a summary.
```

Two lines are new compared with the earlier app prompts. "With the story viewer captured mid-story and a heart burst captured on a post if you can" asks for screenshots of motion, which is hard to capture and easy to skip. And "keep the work focused so it fits in one session of about an hour" came straight from the lesson of Cartly's overlong session.

What the Session Did

Session 4 ran for 32 minutes over 112 turns and cost $7.57, the shortest of the four app sessions, helped by everything the earlier sessions had put into the shared packages. It added `PhotoScene`, seven photo filters, `StoryProgress`, `HeartBurst` and a `useDoubleTap` hook to the design system, and story ordering, story stepping and chat titles to the API package. The server already had the routes Lumo needed.

Its report included three fixes it made while checking screenshots (a timestamp wrapping in Messages, a hard-edged dark band at the top of

stories, and loud pink outline buttons on profiles, which it made quiet gray) and two honest gaps: story replies don't send anything yet, because the server has no endpoint for them, and tagging offers only the people in your stories row, because there's no "following" list endpoint. It also warned that running `expo start` had rewritten two configuration files, which it restored.

Key Takeaways

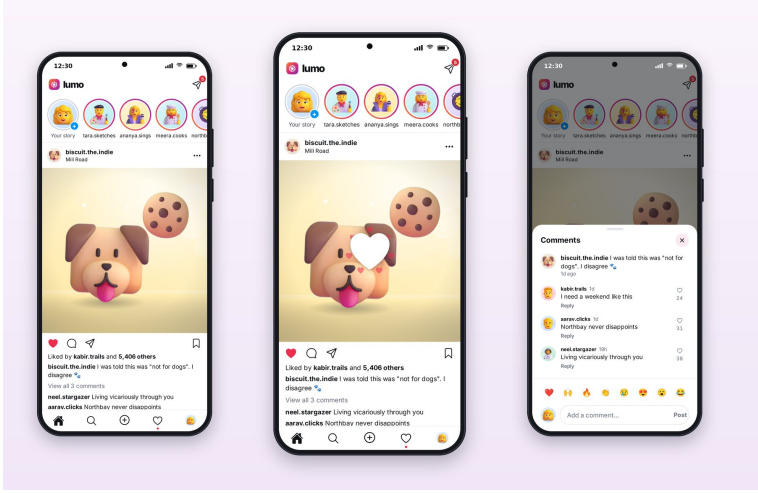
- Social apps put content first: full-width media, quiet black-and-white chrome, one accent color, and a lot of motion.
- When you can't use real photos, generate deterministic, distinct scenes from each item's ID, and keep the component's props easy to swap for a real image.
- Ask explicitly for screenshots of motion states, and size each session to the work.
- Note the honest gaps a session reports (here, story replies and tagging), and turn them into follow-up tasks.

CHAPTER 16 · SOCIAL

Feed, Stories and the Double-Tap Like

The feed and the story viewer are where users spend almost all their time in a social app, so they're where polish matters most. This chapter looks at how Lumo builds them, and at the small pieces of logic and animation that make them feel right.

The Feed



The feed, the heart burst after a double tap, and the comments sheet with quick emoji reactions.

Lumo's feed follows the pattern every social app user knows, and its details are what make it feel professional:

- **The header** is just the logo and a messages icon with an unread count.
- **Stories** use a gradient ring for new stories and a gray ring for seen ones, and the row is ordered the way users expect: your own story first, then people with new stories, then seen ones. That ordering is a tested function in the API package.
- **Captions** color @mentions and #tags and collapse long text behind "more".
- **The tab bar** shows icons only, a red dot when there's new activity, and your own avatar for the profile tab.

The Double Tap

Telling a double tap from a single tap is a timing problem: if a second tap comes within about 280 milliseconds of the first, it's a double tap. Lumo's hook keeps that logic small and testable:

```
import { useCallback, useRef } from 'react';

/**
 * Tells a double tap from a single one by timing. `tap(now)` returns
↪ true on
 * the second tap within `delayMs` of the first; a third tap starts over.
 */
export function createTapDetector(delayMs = 280) {
  let last = -Infinity;
  return {
    tap(now: number): boolean {
      if (now - last <= delayMs) {
        last = -Infinity;
        return true;
      }
      last = now;
      return false;
    },
  };
}

/** A press handler that calls `onDouble` on a double tap (and
↪ `onSingle`, if given, right away on the others). */
export function useDoubleTap(onDouble: () => void, onSingle?: () =>
↪ void, delayMs = 280) {
  const detector = useRef(createTapDetector(delayMs));
  return useCallback(() => {
    if (detector.current.tap(Date.now())) onDouble();
```

```
    else onSingle?.();
  }, [onDouble, onSingle]);
}
```

`createTapDetector` is a plain function with a clock passed in, so its test can feed it exact times instead of waiting. A third quick tap starts a new sequence instead of counting as a second double tap.

A double tap sets the like, if it isn't set already (a double tap never un-likes, as in every big social app), and plays the heart burst:

```
/**
 * The double-tap like: a big heart pops over a photo while small hearts
 ↪ fly
 * out. Under reduce motion only the big heart fades in and out.
 */
export function HeartBurst({ trigger, size = 96 }: HeartBurstProps) {
  const reduceMotion = useReducedMotion();
  const scale = useSharedValue(0);
  const opacity = useSharedValue(0);
  const particles = useSharedValue(0);

  useEffect(() => {
    if (trigger <= 0) return;
    if (reduceMotion) {
      scale.set(1);
      opacity.set(withSequence(withTiming(1, { duration: 120 }),
      ↪ withDelay(450, withTiming(0, { duration: 200 }))));
      return;
    }
    scale.set(withSequence(withTiming(0, { duration: 0 }), withSpring(1,
    ↪ motion.spring.pop), withDelay(260, withTiming(0.2, { duration: 180
    ↪ }))));
    opacity.set(withSequence(withTiming(1, { duration: 80 }),
    ↪ withDelay(520, withTiming(0, { duration: 180 }))));
    particles.set(withSequence(withTiming(0, { duration: 0 }),
    ↪ withDelay(60, withTiming(1, { duration: 620 }))));
  }, [trigger, reduceMotion, scale, opacity, particles]);

  const heart = useAnimatedStyle(() => ({ opacity: opacity.value,
  ↪ transform: [{ scale: scale.value }] }));

  return (
    <View pointerEvents="none" style={styles.overlay}
    ↪ importantForAccessibility="no-hide-descendants"
    ↪ accessibilityElementsHidden>
      {!reduceMotion
        ? Array.from({ length: PARTICLES }, (_, i) => <Particle key={i}
        ↪ index={i} progress={particles} size={size} />)
        : null}
      <Animated.View style={heart}>
        <Icon name="heart" size={size} color="textInverse"
        ↪ style={styles.shadow} />
      </Animated.View>
    </View>
```

```
  );
}
```

A big white heart springs up with the `pop` spring while seven small hearts fly outward and fade, then everything fades away. Under the system's reduce-motion setting, only the big heart fades in and out, with no movement. The overlay ignores touches and is hidden from screen readers, because the like itself is announced by the like button.

The like button reacts too, with a small bounce, and the count updates immediately through an optimistic update. If the server refuses, the like rolls back.

> **Note:** Session 4 reported that the heart burst's animation kept running after its test finished and stalled the next test in the same file, so the double-tap and story tests were moved into their own files. Animations and timers are the most common cause of flaky tests in React Native; when a test passes alone and fails in a group, look for one first.

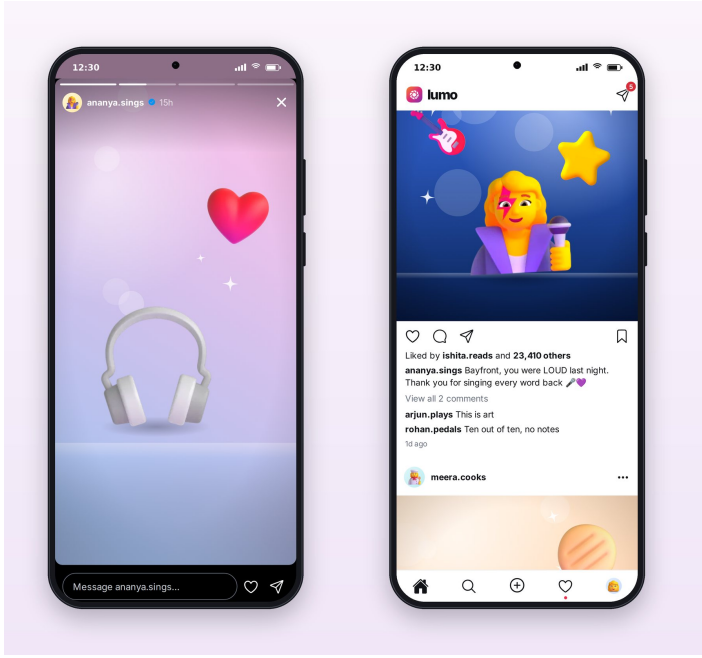
Comments

The comments sheet slides up over the post, starting with the caption as its first item, then comments with avatar, username, time, like count and a Reply action. At the bottom sits a row of quick emoji replies and the comment field. The sheet keeps the post visible behind it, so the conversation stays in context.

The Story Viewer

The story viewer is a small state machine:

- **Progress bars** at the top, one per story, filling over each story's duration (`StoryProgress` in the design system).
- **Tap the right side** to go to the next story; at the end of one person's stories, move to the next person's.
- **Tap the left side** to go back.
- **Press and hold** to pause; release to resume.



A story mid-play, with progress bars at the top and a reply box at the bottom, and the feed scrolled to further posts.

- **Swipe down or tap the close button** to leave.
- **Seen state**: each story is marked seen as it's shown, so the ring in the feed turns gray.

Stepping across people (the last story of one person, then the first unseen story of the next) is logic in the API package, tested without any UI. Profile highlights play in the same viewer.

Key Takeaways

- Order stories as users expect: yours, then new, then seen; keep that order in a tested function.
- Detect double taps with a small timing function you can test with fake clocks; a double tap only ever likes.
- Animate with springs, respect reduce motion, hide decorative overlays from screen readers, and use optimistic updates for likes.

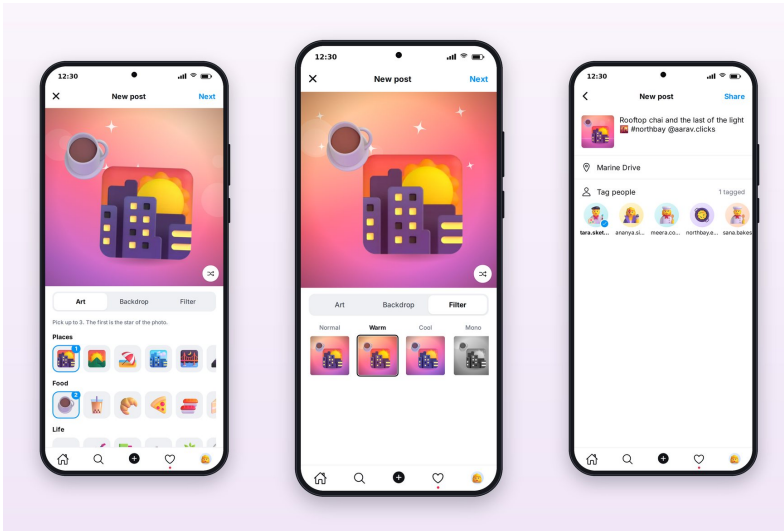
- Treat the story viewer as a state machine: next, previous, pause, close, seen, and stepping across people.
- When a test fails only in a group, look for animations and timers that outlive it.

CHAPTER 17 · SOCIAL

Create, Explore, Profile and Messages

The rest of a social app is about making things and finding people: creating a post, exploring, profiles, activity and messages. This chapter walks through Lumo's versions of each.

Create



Creating a post: choosing art (numbered, the first is the star), choosing a filter, then the caption, location and tags.

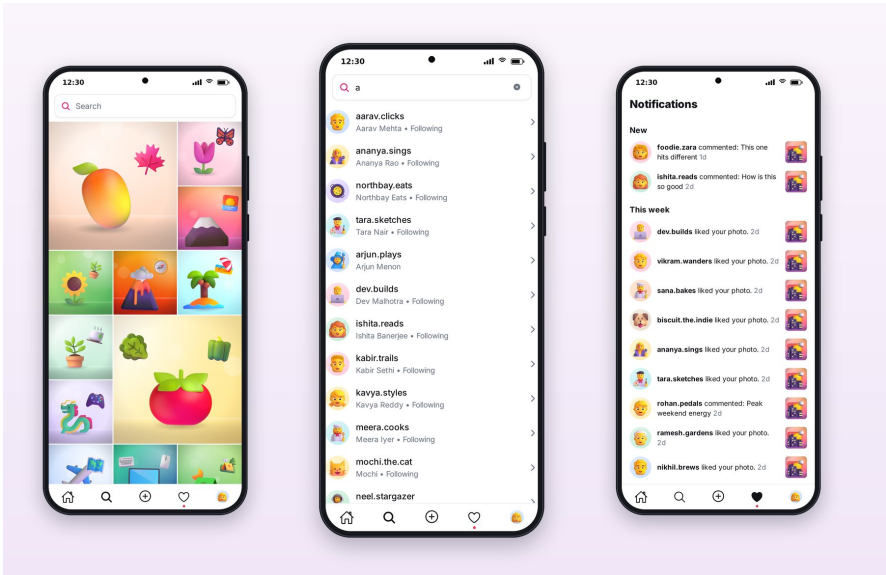
Lumo's create flow has two steps, as most photo apps do. The first builds the photo; the second adds everything about it.

Step one, the photo. Three tabs: **Art**, **Backdrop** and **Filter**. In Art, you pick up to three illustrations from categories (Places, Food, Life and more), and the picks are numbered: "Pick up to 3. The first is the star of the photo." A shuffle button rearranges the layout. The Filter tab shows the photo with each filter as a thumbnail (Normal, Warm, Cool, Mono and more), so you choose by seeing.

Step two, the details. The caption (with #tags and @mentions), a location, and **Tag people**, with avatars to tap and a count ("1 tagged"). Then **Share**.

Seven filters were built. Most use color transforms that work everywhere; session 4 reported that one, Vivid, uses a CSS filter that iOS doesn't support, so on iOS it looks like Normal. That's the kind of platform difference to write down in your release notes, or to ask Claude to fix with a different technique.

Explore and Activity

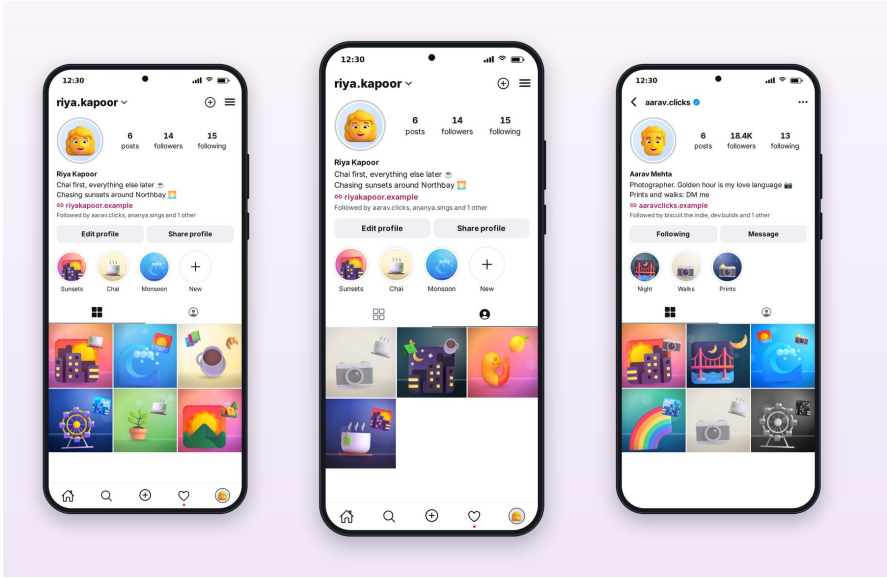


Explore with its mixed grid, people search, and activity grouped by time.

Explore is a grid with a large tile every six posts, alternating sides, which breaks the monotony of a uniform grid and gives the most engaging posts more room. The search bar at the top searches people: typing "a" lists matching usernames with their names and "Following" where it applies.

Activity groups notifications into New, Today, This week and Earlier, with the post's thumbnail on the right and a Follow back button for new followers. Notifications are marked read when you leave the tab, and the tab bar's red dot disappears with them.

Profiles



Your profile, its tagged tab, and someone else's profile with Following and Message buttons.

Profiles follow the familiar layout: avatar, counts (posts, followers, following), name, a bio that can contain emoji, a link, and "Followed by aarav.clicks, ananya.sings and 1 other". On your own profile the buttons are **Edit profile** and **Share profile**; on someone else's, **Following** and **Message**. Highlights sit below as labelled circles, then the **grid** and **tagged** tabs.

Session 4's first draft gave the profile buttons loud pink outlines; it noticed in its own screenshots and changed them to quiet gray, which is what users of

the category expect. Small as it is, that change is a good example of what the visual check is for: nothing was broken, but it didn't look right.

Messages



The conversation list with unread dots, and a chat thread.

The messages list shows each conversation with the person's avatar (with a green dot when they're online), their name in bold when there are unread messages, the last message, its age and an unread dot. Group chats are titled from their members ("Kabir, Neel and 1 other"), a small rule written as a tested function.

The chat thread groups consecutive bubbles from the same person, shows the sender's avatar only on the last bubble of a group, colors your own bubbles in the accent blue on the right, and shows a timestamp only when the day or a long gap changes. The header carries the person's name, a verified badge, their username, and call and video buttons.

> **Try It:** Session 4 reported that story replies don't send yet, because the server has no endpoint for them. Ask Claude to add one: a server route that turns a story reply into a message in the right conversation, the client hook, a test for each, and an update to the story viewer's reply box. Then check that the reply appears in Messages.

Key Takeaways

- Split creation into two steps: make the photo, then describe it; choose filters by seeing them.
- Vary an explore grid with large tiles; let search find people.
- Group activity by time and offer the obvious next action (Follow back).
- Profiles change their buttons for you versus others; keep the chrome quiet.
- Chat threads group bubbles, show avatars and timestamps sparingly, and title group chats from their members.

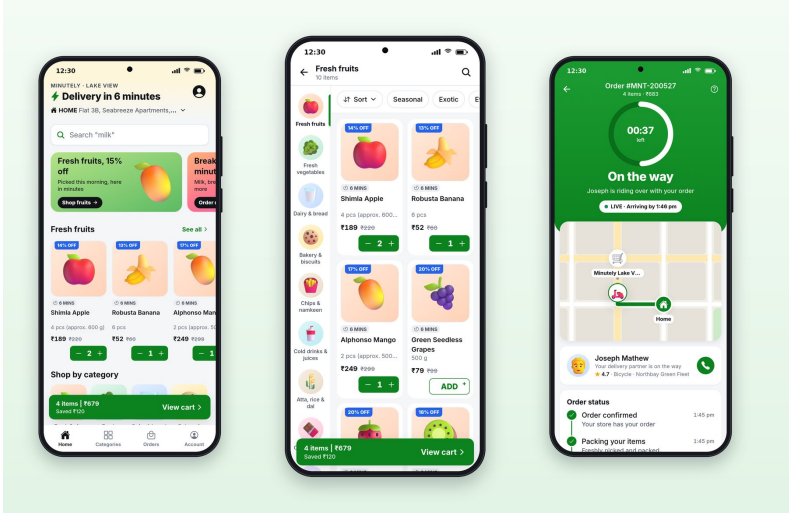
PART V

Minutely, 10-Minute Grocery

CHAPTER 18 · QUICK COMMERCE

Speed as a Feature: Planning Quick Commerce

Quick-commerce apps sell one thing above everything else: time. The groceries are the same as in any supermarket; the promise is that they'll be at your door before the kettle boils. So the app has to feel as fast as the delivery. Every tap answers immediately, the delivery time is the biggest text on the home screen, and the cart is never more than one tap away. This part builds Minutely.



Minutely: home with "Delivery in 6 minutes", a category with its left rail, and live tracking with a countdown ring.

The Patterns Users Expect

Quick commerce borrows from both food delivery and shopping, and adds its own patterns:

- **The delivery time as the headline.** "Delivery in 6 minutes" in bold at the top of the home screen, with a lightning bolt, above the address.
- **ADD buttons that turn into steppers.** A product card's "ADD" becomes "− 1 +" in place, on the card, without opening anything.
- **A floating cart bar.** As soon as the cart has something in it, a bar at the bottom shows "4 items | ₹679", the savings, and "View cart". It sits above the tab bar on every screen.
- **Bright category tiles** in a grid, each with its own soft color.
- **A category page with a left rail**: categories as a vertical list on the left, products in a two-column grid on the right, filter chips across the top.
- **Savings everywhere**: discount ribbons on every card ("14% OFF"), MRP struck through, "You're saving ₹150 on this order" in green.
- **A cart that nudges**: progress toward free delivery, a tip for the delivery partner, a small-cart fee under a minimum.
- **Tracking with a countdown**: a ring counting down the minutes, a map with the rider, and the steps.

Speed Is a Design Decision

Most of what makes Minutely feel fast isn't performance tuning; it's design:

- **Nothing waits for the server when it doesn't have to.** The stepper shows the new quantity the moment it's tapped.
- **Nothing opens a new screen when a sheet will do.** Tapping a product slides a product sheet over the current screen, so closing it returns you exactly where you were, scrolled to the same place.
- **The next action is always visible.** The cart bar is on every browsing screen; the "Proceed to pay" bar is on the cart; the pay button names the amount and the method.
- **The time shown is the real time.** Every screen uses the store's estimate from the server.

The last point came from a real mistake. Session 5's first draft hard-coded "9 minutes" in some places while the home screen, which reads the server, showed 6. The screenshots caught it. A speed promise that disagrees with itself is worse than no promise at all.

The Prompt

Session 5 had two jobs: fix a flaky test in Cartly, and build Minutely.

```
Two jobs in this session.

1. A flaky test. `npm run check` failed once in
apps/cartly/src/test/screens/orders-returns.test.tsx ("Order details ›
tracks the shipment live and lets you cancel before it ships") while the
whole workspace's tests ran in parallel; run alone it passes. Find the
↳ real
cause (timing, fake timers, shared state, SSE) and fix it properly,
↳ without
skipping the test or just raising timeouts. Prove it with several runs of
the full `npm test` from the root.

2. Build Minutely, the 10-minute grocery app in docs/PRD.md section 4, as
apps/minutely: an Expo SDK 58 app with Expo Router, using @pro/ui (brand
"minutely") and @pro/api, following the structure of the other apps and
reusing the shared packages.

It should feel like the best quick-commerce apps: speed everywhere (a bold
"Delivery in 9 minutes" header, instant ADD steppers, a cart bar that's
always one tap away), bright category tiles, savings called out in green,
and a tracking screen with a countdown ring and the SVG map.

Screens: Login, Home (address, search, category grid, offer banners,
product rails with ADD steppers), Search, Category page (left category
rail, product grid, filters), Product sheet, Cart (savings banner,
delivery partner tip, bill), Checkout and payment, Tracking (countdown
↳ ring,
SVG map, status steps, partner card), Orders, Account. Tabs: Home,
Categories, Orders, Account.

As before: plan first, add missing shared pieces with tests, check each
screen in headless Chromium at 393x852 CSS pixels with device scale 3 and
fix anything unpolished, add logic and screen tests, save final
↳ screenshots
of every screen to apps/minutely/screenshots/ (01-home.png, ...), with
tracking captured mid-delivery. Keep it focused enough for one session of
about an hour. Finish with typecheck, lint and tests passing for the whole
workspace, and a summary.
```

Notice how specific the "feel" paragraph is: "a bold 'Delivery in 9 minutes' header, instant ADD steppers, a cart bar that's always one tap away, … savings called out in green." Words like "fast" and "modern" give Claude

nothing to check. A list of visible patterns gives it a checklist for the screenshot review.

The Flaky Test, Solved Properly

The first job is a lesson in its own right. A Cartly test sometimes failed when the whole workspace's tests ran together and always passed alone. The easy fixes are to skip it or raise its timeout; the prompt ruled both out and asked for the real cause.

Session 5 found it. React Query hands cache updates to components with a zero-delay timer. The test pushed a "shipped" event and then checked at once that "Cancel order" had gone, which only works if that timer fires first. On a busy machine it sometimes didn't. Session 5 proved the cause by delaying React Query's notifications by 5 milliseconds, which made the test fail every time, then fixed it in two places: the shared test query client now delivers updates in a microtask, which always runs before the test continues, and the test now waits for "Cancel order" to disappear rather than assuming it already has. With the fix, the test passes even with a 200-millisecond delay. It also found and fixed a second problem on the way: a test file that left five-minute timers running, which made Jest hang at the end of a run.

> **Tip:** When you ask Claude to fix a flaky test, ask for proof: a way to make the failure happen every time, and the same check passing after the fix. "It passed three times" proves nothing about a test that fails one time in ten.

What the Session Did

Session 5 ran for about 53 minutes and cost $9.08. It wrote a plan to `apps/minutely/PLAN.md`, built the app on the shared packages, and found that the server already had everything Minutely needed. It added two things to the design system, each with a test: `CountdownTimer` gained fill and track colors, because the white ring was invisible on the green tracking header, and `CartBar` gained `tone="primary"`, so Minutely's bar could be green instead of the accent yellow.

Its screenshot review fixed six problems before it finished: the header wrapping onto three lines, a search hint that read `Search "Search "bread""`, a category grid with three columns instead of four, prices breaking across lines on narrow cards, filter chips cut off at the edge, and a pay button that read "Pay ₹683UPI".

Key Takeaways

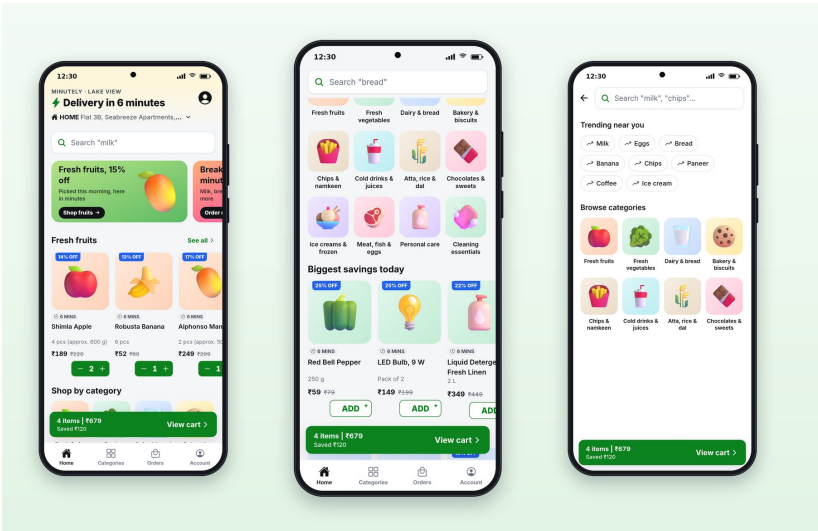
- Quick commerce sells time; the app must feel as fast as the delivery promise.
- Speed is mostly design: instant steppers, sheets instead of screens, the next action always visible, and one real delivery time everywhere.
- Describe the feel you want as visible patterns Claude can check in screenshots.
- Fix flaky tests at their cause, with a way to reproduce the failure and proof that the fix holds.

CHAPTER 19 · QUICK COMMERCE

Catalog, Cart and the Countdown Delivery

This chapter walks through Minutely from the home screen to the doorbell, and looks closely at the one piece of logic that makes it feel instant: a stepper that moves before the server answers, on every screen at once.

Home and Search



Home with the delivery time, offer banners and a product rail; further down, the category grid and today's biggest savings; and search with trending terms.

The home screen is built around its first line. "MINUTELY · LAKE VIEW" names the store; "Delivery in 6 minutes" with a lightning bolt is the promise; the address sits under it with a chevron to change it. Then:

- **A search bar** whose hint rotates through common items ("milk", "bread", "chips"), a quiet suggestion of what the store sells.
- **Offer banners**: "Fresh fruits, 15% off. Picked this morning, here in minutes", with a "Shop fruits" button.
- **Product rails** such as Fresh fruits, each card with a discount ribbon, the delivery time ("6 MINS"), the name, the pack size, the price with MRP struck through, and an ADD button or stepper.
- **Shop by category**, a grid of four columns of colored tiles.
- **Biggest savings today**, a rail sorted by discount.

The delivery time comes from one helper that never says "0 minutes" or "1 minutes":

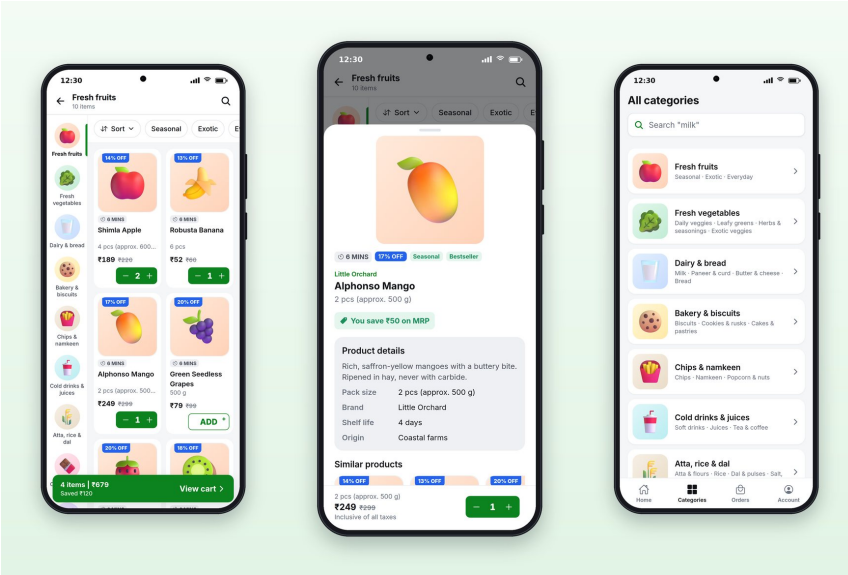
```
/** "Delivery in 9 minutes" (never "0 minutes", never "1 minutes"). */
export function etaHeadline(minutes: number): string {
  const m = Math.max(1, Math.round(minutes));
  return `Delivery in ${m} ${m === 1 ? 'minute' : 'minutes'}`;
}
```

Search opens with "Trending near you" as chips and the category tiles below, so a user who doesn't know what to type still has somewhere to go. Results use the same product cards, and out-of-stock items sink to the bottom rather than disappearing.

The Category Page and the Product Sheet

The category page is the quick-commerce layout: a narrow rail of categories on the left, each with its picture and name, the selected one marked with a green bar; a row of chips across the top (Sort, then subcategories such as Seasonal and Exotic); and a two-column grid of products. Switching categories in the rail changes the grid without leaving the screen.

Tapping a product slides up the **product sheet**: a large picture, badges (delivery time, discount, Seasonal, Bestseller), the brand in green, the name and pack size, "You save ₹50 on MRP", the product details (pack size, brand,



A category with its left rail, filter chips and product grid; the product sheet sliding over it; and the full list of categories.

shelf life, origin), similar products, and a bar with the price and the stepper. Because it's a sheet, closing it puts you back on the grid exactly where you were.

The **Categories** tab lists every category as a card with its subcategories underneath ("Milk · Paneer & curd · Butter & cheese · Bread"), which helps people who browse by aisle.

A Stepper That Moves Before the Server Answers

The same product can appear in three places at once: a home rail, a category grid and the product sheet. When the user taps "+" in one, all three, and the cart bar, should change at once, before the server replies. Minutely does this with a small shop context that wraps the app:

```
export function ShopProvider({ children }: { children: ReactNode }) {
  const { data: cart } = useCart('minutely');
  const setItem = useSetCartItem('minutely');
  const toast = useToast();
  const etaMinutes = useMinutelyHome().data?.etaMinutes ?? 9;
  const [pending, setPending] = useState<Record<string,
  ↪ PendingChange>>({});
```

```
  const [productId, setProductId] = useState<string | null>(null);
  // Only the latest tap on a product may clear its pending quantity.
  const seq = useRef<Record<string, number>>({});

  const quantityOf = useCallback((id: string) => pending[id]?.quantity
  ↪ ?? quantityInCart(cart, id), [pending, cart]);

  const change = useCallback(
    (item: Stockable, quantity: number) => {
      const next = Math.max(0, Math.min(quantity, item.maxPerOrder));
      const mine = (seq.current[item.id] ?? 0) + 1;
      seq.current[item.id] = mine;
      setPending((p) => ({ ...p, [item.id]: { quantity: next, price:
      ↪ item.price, mrp: item.mrp } }));
      setItem.mutate(
        { productId: item.id, quantity: next },
        {
          onError: (e) => toast.show({ message: e.message, tone:
          ↪ 'danger' }),
          onSettled: () => {
            if (seq.current[item.id] !== mine) return;
            setPending(({ [item.id]: _done, ...rest }) => rest);
          },
        },
      );
    },
    [setItem, toast],
  );
```

Each tap records a **pending** quantity for that product and sends the request. Every stepper asks `quantityOf(id)`, which returns the pending quantity if there is one and the server's otherwise, so every copy of the product shows the same number. When the request settles, the pending entry is cleared, and the server's cart, now updated, takes over. If the request fails, a toast explains why ("You can add up to 5 of …") and the stepper falls back to the server's number.

The `seq` counter handles fast tapping. If a user taps "+" three times quickly, three requests are in flight; only the answer to the last tap may clear the pending quantity, otherwise the stepper would flicker back to 1, then 2, then 3 as the answers arrive.

The cart bar needs totals that include the pending taps, and that is a pure function, tested without any UI:

```
import { quantityInCart, type Cart } from '@pro/api';

/** A stepper tap the server has not answered yet. */
export interface PendingChange {
  quantity: number;
  price: number;
```

```
  mrp: number;
}

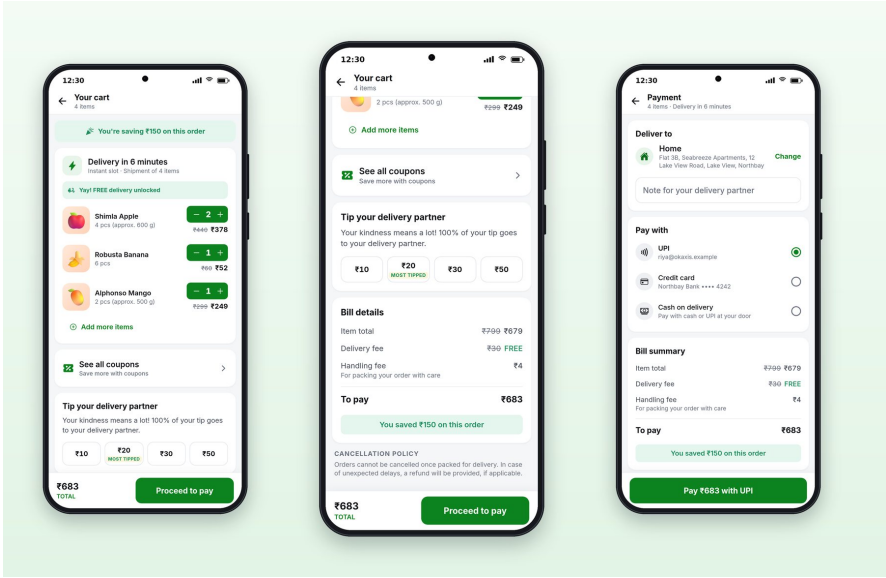
/**
 * The cart bar's count and totals right now: the server's cart plus the
 * difference each unanswered tap makes, so the bar moves with the
 ↪ stepper.
 */
export function pendingSummary(
  cart: Pick<Cart, 'items' | 'itemCount' | 'bill'> | undefined,
  pending: Readonly<Record<string, PendingChange>>,
): { itemCount: number; total: number; mrpTotal: number } {
  let itemCount = cart?.itemCount ?? 0;
  let total = cart?.bill.itemTotal ?? 0;
  let mrpTotal = cart?.bill.mrpTotal ?? 0;
  for (const [productId, change] of Object.entries(pending)) {
    const diff = change.quantity - quantityInCart(cart as Cart |
    ↪ undefined, productId);
    itemCount += diff;
    total += diff * change.price;
    mrpTotal += diff * change.mrp;
  }
  return { itemCount: Math.max(0, itemCount), total: Math.max(0, total),
  ↪ mrpTotal: Math.max(0, mrpTotal) };
}
```

> **Note:** This is a different shape of optimistic update from the cache-based ones in Chapter 6. Those edit the React Query cache directly; this one keeps pending taps beside the cache and combines them for display. It suits a stepper that may be tapped many times in a second, because the cache is never rewritten by an out-of-date answer.

The Cart and Payment

From top to bottom, Minutely's cart shows:

1. **"You're saving ₹150 on this order"** in green, first, before anything else.
2. **The delivery slot**: "Delivery in 6 minutes. Instant slot · Shipment of 4 items".
3. **The free-delivery nudge**: "Add ₹51 more for FREE delivery" with a progress bar, which becomes "Yay! FREE delivery unlocked" once the cart passes ₹199.
4. **The items**, each with a stepper and its price with MRP struck through, and "Add more items".
5. **Coupons**: "See all coupons. Save more with coupons."



The cart with savings, the free-delivery banner and tips; the bill; and the payment screen.

6. **Tip your delivery partner**: "Your kindness means a lot! 100% of your tip goes to your delivery partner", with ₹10, ₹20, ₹30 and ₹50, and ₹20 marked "MOST TIPPED".
7. **Bill details**: item total with MRP struck through, the delivery fee (₹30 struck through, "FREE"), a ₹4 handling fee "for packing your order with care", "To pay ₹683", and the savings again.
8. **The cancellation policy**, in plain words: orders can't be cancelled once packed.
9. **A sticky bar**: "₹683 TOTAL" and "Proceed to pay".

The fees are the server's (Chapter 6): a ₹30 delivery fee waived above ₹199, a ₹4 handling fee, and a ₹15 small-cart fee under ₹99. The app only shows them, with a one-line reason under each fee, because unexplained fees are the most common complaint about quick-commerce apps.

The **payment** screen confirms the address with a Change link, offers a note for the delivery partner, lists UPI, a credit card and cash on delivery, repeats the bill summary, and ends with one button that says exactly what will happen: "Pay ₹683 with UPI".

Tracking With a Countdown



Tracking just after ordering, with the rider on the way, and further down the screen: the partner, the steps and the bill.

The tracking screen puts the promise at the top again, this time as a countdown. A green header carries the order number and a ring counting down the time left ("01:14 left"), with the stage in large type: "Order confirmed", "Packing your order", "On the way", "Delivered!". Under it, a LIVE pill shows the arrival time.

The ring's window runs from placing the order to its promised arrival, both from the server:

```
import { orderProgress, type Order } from '@pro/api';

export interface Countdown {
  /** When the order should arrive (epoch ms). */
  endsAt: number;
  /** The whole promised window, for the ring's progress. */
  totalSeconds: number;
}

/** The countdown ring's window: from placing the order to its promised
↪ arrival. */
export function countdownWindow(order: Pick<Order, 'placedAt' |
↪ 'etaAt'>): Countdown {
  const start = Date.parse(order.placedAt);
```

```
  const end = Date.parse(order.etaAt);
  return { endsAt: end, totalSeconds: Math.max(60, Math.round((end -
  ↪ start) / 1000)) };
}
```

The ring is the shared `CountdownTimer` with `variant="ring"`; session 5 added `fillColor` and `trackColor` to it so a white ring on a dark green track could sit on the green header. When the order is delivered, the ring is replaced by a party popper and "Delivered in 1 minute. Enjoy!", calculated from the real times.

Below the header are the same SVG map as Platter's (store, rider, home), the delivery partner card with a call button, the four status steps with their times, the items, and the bill. All of it updates live over the order's event stream.

The demo server delivers a Minutely order in 75 seconds (confirmed, packing at 4 seconds, on the way at 15, delivered at 75), so the ring counts seconds rather than minutes. That keeps a demo short; in a real app, the server sends real times, and nothing in the client changes.

> **Try It:** Ask Claude to add "Order again" to Minutely's Orders tab: one tap puts every item from a past order back into the cart, skips anything out of stock, and shows a toast such as "3 items added, 1 out of stock". Ask for the server route, a test for the out-of-stock rule, a screen test, and a screenshot.

Key Takeaways

- Lead with the delivery time; make the ADD button turn into a stepper in place; keep the cart bar on every browsing screen.
- Use a left category rail and a product sheet so users browse without losing their place.
- Keep pending stepper taps beside the server's cart, let only the latest tap clear them, and compute the cart bar's totals with a tested pure function.
- Explain every fee in one line, show the savings first, and put the amount and the method on the pay button.
- Track with a countdown ring driven by the server's times, so a demo's 75 seconds and a real delivery's 10 minutes use the same code.

PART VI

Production

CHAPTER 20 · PRODUCTION

Performance: Lists, Images, Startup and Smooth Animation

The four apps work. This part makes them ready for real users, on real phones, from a real store. It starts with performance, because the apps people call "professional" are, above all, the ones that never stutter.

Performance on a phone comes down to three numbers users feel without measuring: how long the app takes to open, whether scrolling stays smooth, and how quickly a tap shows a response. This chapter covers each, with prompts you can give Claude for your own app.

Measure on a Real, Cheap Phone

Your development phone is probably fast. Many of your users' phones are not. Before tuning anything, install a **release build** (Chapter 26) on the slowest Android phone you can borrow and use the app for ten minutes. Development builds are much slower than release builds, because they include debugging tools and don't optimize the JavaScript, so never judge performance from one.

Then measure:

- **Startup:** time from tapping the icon to a usable home screen. Aim for under two seconds on a mid-range phone.

- **Frame rate:** React Native's performance monitor (in the developer menu) shows the JavaScript and UI frame rates. Both should stay near 60 while scrolling, or near your screen's refresh rate.
- **Profiles:** React DevTools' profiler shows which components re-render and how long they take. Android Studio's and Xcode's profilers show native CPU and memory.

> **Note:** The screenshots in this book come from web builds, and no app in it was profiled on a physical phone. Everything in this chapter is what you should do on your devices before release.

Long Lists

Feeds, product grids and order histories are the most common source of jank. React Native's `FlatList` renders items lazily, but each item that scrolls into view still mounts new components. Three rules cover most problems:

1. **Keep row components cheap and stable.** Wrap rows in `memo`, pass stable callbacks (`useCallback`), and give each item a stable `keyExtractor`. A row that re-renders whenever its parent does is the most common cause of a slow list.
2. **Don't nest lists in a scroll view.** A `FlatList` inside a `ScrollView` renders everything at once and loses virtualization. Use the list's header and footer components instead.
3. **Use a recycling list for large or complex feeds.** FlashList (version 2.3.2 is the one bundled with Expo SDK 58) reuses row views instead of creating new ones, which makes a big difference with long feeds of rich cards. It's a near drop-in replacement for `FlatList`.

The apps in this book use `FlatList` and ordinary scroll views, because their seed data is small. A prompt to move a real feed onto FlashList:

```
Lumo's feed and Cartly's product list will hold thousands of
  items in
production. Move both to FlashList (the version bundled with our
  Expo
SDK), keep every existing test passing, make sure post and
  product rows
```

```
are memoized with stable props, and tell me anything that behaves
differently, such as scroll restoration or the sticky header.
```

Images

The apps draw their pictures with illustrations, but a real app loads photos from a server, and photos are usually the heaviest thing it does. Professional apps:

- **Request the right size.** Ask your image service for a 400-pixel-wide thumbnail for a 130-point card, not the 4,000-pixel original.
- **Cache on disk.** `expo-image` caches downloaded images in memory and on disk, so scrolling back up doesn't download again.
- **Show something immediately.** A blurred placeholder (a short "blurhash" string sent with the data) or the dominant color fills the space while the photo loads, and a short fade-in replaces it.
- **Use modern formats.** WebP or AVIF are much smaller than JPEG at the same quality.

In this book's code, `PhotoScene` (Chapter 15) and `ArtTile` are where real images would go. Replacing them with `expo-image` is a contained change: the components' props already carry an ID and a size.

Startup

Startup time is spent loading the JavaScript bundle, running your top-level code, and fetching the first data. To shorten it:

- **Keep the splash screen up until the first screen is ready**, then hide it, so users never see a blank frame. The apps already do this with `expo-splash-screen` while their fonts load.
- **Do less at launch.** Don't import heavy libraries at the top of the root layout if only one screen needs them.
- **Show cached data first.** React Query can persist its cache to storage, so the home screen draws instantly from last time's data and refreshes in the background.

- **Keep the bundle small.** Remove unused dependencies, and check what's large with a bundle analyzer before release.

Animations at 60 Frames per Second

React Native runs your JavaScript on one thread and draws the interface on another. An animation driven from JavaScript stutters whenever JavaScript is busy, for example while a list renders. Reanimated (4.7.1 with SDK 58) runs animations on the UI thread instead, which is why the apps' heart burst, sheets and pressable scale stay smooth while data loads.

The rules:

- **Animate transforms and opacity**, not width, height or margins, which force layout on every frame.
- **Use springs for things users touch** (they feel physical), and short timings for things that appear on their own.
- **Respect reduce motion.** The design system's animations check the system setting and replace movement with a simple fade (Chapter 16).

Taps That Feel Instant

The fastest request is the one the user doesn't wait for. The apps use optimistic updates for likes, wishlists and cart steppers (Chapters 6 and 19), give every pressable a visual response on press, and keep heavy work off the press handler. If a tap must wait for the server, as payment does, show progress on the button itself.

A Performance Review Prompt

When you're ready to tune your own app, ask for a review before you ask for changes:

```
Review this app for performance problems a user would feel on a
mid-range Android phone. Look for: lists that re-render whole rows,
lists nested in scroll views, images loaded at full size, work
  done at
startup that could wait, animations driven from JavaScript, and
  anything
```

```
that blocks a press handler. Rank what you find by how much a
  user would
notice it, explain each in one sentence, and don't change any
  code yet.
```

Then fix the top items one at a time, with a release build on a real phone to check each.

Key Takeaways

- Judge performance only in release builds, on a cheap phone.
- Memoize list rows, never nest lists in scroll views, and use FlashList for long, rich feeds.
- Load images at the right size, cache them, and show a placeholder at once.
- Keep the splash screen up until the first screen is ready, do less at launch, and show cached data first.
- Animate transforms and opacity on the UI thread with Reanimated, and respect reduce motion.

CHAPTER 21 · PRODUCTION

Accessibility and Localization

An app that only works for people who see well, tap precisely and read English leaves out a large share of its customers. The biggest apps invest heavily in accessibility and localization because it pays: more people can use them, and the same work makes the app better for everyone. This chapter shows what the apps in this book already do and what to add before release.

Accessibility Was a Rule From Day One

The project's `CLAUDE.md` (Chapter 3) included one line that shaped every screen:

> Accessibility: every touchable has an `accessibilityRole` and label; text scales with the system font size; touch targets at least 44x44.

Because it was a rule from the first session, Claude wrote accessible components from the start, and the tests could use the same labels a screen reader announces. Look at a test from Minutely:

```
  it('adds to the cart at once and the cart bar follows every tap',
  ↪ async () => {
    const user = userEvent.setup();
    const { client, calls, cart } = minutelyServer();
    await renderScreen(<HomeScreen />, client);
    await user.press(await screen.findByRole('button', { name: 'Add
    ↪ Shimla Apple' }));
```

```
    expect(screen.getByRole('button', { name: '1 item, ₹189. View cart'
    ↪ })).toBeOnTheScreen();
    await user.press(screen.getByRole('button', { name: 'Add one more
    ↪ Shimla Apple' }));
    expect(await screen.findByRole('button', { name: '2 items, ₹378.
    ↪ View cart' })).toBeOnTheScreen();
    expect(screen.getByText('Saved ₹62')).toBeOnTheScreen();
    await waitFor(() => expect(cart().itemCount).toBe(2));
    expect(calls.filter((c) => c.method === 'PUT').map((c) =>
    ↪ c.body)).toEqual([
      { productId: 'g-001', quantity: 1 },
      { productId: 'g-001', quantity: 2 },
    ]);
    await user.press(screen.getByTestId('cart-bar'));
    expect(router.push).toHaveBeenCalledWith('/cart');
  });
```

The test finds the ADD button by its role and label, "Add Shimla Apple", and then checks the cart bar's accessible name, "1 item, ₹189. View cart". If a screen reader can't find a button, neither can the test, so accessibility can't quietly break.

Screen Readers

TalkBack on Android and VoiceOver on iOS read the screen aloud and let users move between elements. To work well with them:

- **Every touchable needs a role and a label.** An icon button with a heart needs `accessibilityRole="button"` and a label such as "Like" or "Unlike", not "heart icon".
- **Labels describe the action or the content, briefly.** "Add one more Shimla Apple" is better than "plus".
- **Group what belongs together.** A product card should read as one sentence ("Shimla Apple, 4 pieces, ₹189, was ₹220, 14% off") rather than six separate stops.
- **Announce changes.** When something changes without a tap, such as an order's status, mark the text as a live region. The tracking screens use `accessibilityLiveRegion="polite"` on the status headline, so "On the way" is announced when it changes.
- **Hide decoration.** Illustrations that add nothing, such as the heart burst (Chapter 16), are hidden from screen readers; the like button announces the like.

Test with the real thing. Turn on TalkBack or VoiceOver and complete your app's main journey (log in, find a product, add it to the cart, pay) with your eyes closed. Everything you get stuck on is a bug.

Text Size

Many people set their phone's text larger. The design system's `Text` component scales with the system setting but caps the scale at 1.8 times, so dense rows such as a bill stay usable:

```
/**
 * Themed text in Inter. Text scales with the system font size; the
 ↪ scale is
 * capped at 1.8x (`maxFontSizeMultiplier`) so dense rows stay usable.
 */
export function Text({
  variant = 'body',
  color = 'text',
  weight,
  align,
  tabular,
  strike,
  uppercase,
  style,
  maxFontSizeMultiplier = 1.8,
  ...rest
```

At large sizes, layouts must wrap rather than cut off. Check every screen at the largest system text size; rows of chips and price lines are the usual problems.

Touch Targets, Contrast and Color

- **Touch targets at least 44 by 44 points.** A small icon can sit in a larger touch area; the design system's `IconButton` does this.
- **Contrast.** Body text needs a contrast ratio of at least 4.5:1 against its background, and large text 3:1, following the Web Content Accessibility Guidelines that both stores point to. Bright brand colors on white often fail for small text. Because screens use color roles (Chapter 5), fixing a failing pair is a change in one place.
- **Never color alone.** A discount is green *and* says "14% OFF"; an error is red *and* explains itself in words; the unread dot comes with bold text.

- **Dark mode.** Every color in the apps comes from a theme role, so dark mode works everywhere (Chapter 5). Check contrast in both themes.

Reduce Motion

Some people feel unwell with movement on screen. The design system reads the system's reduce-motion setting with Reanimated's `useReducedMotion` and replaces springs and flying hearts with simple fades. Ask Claude to check every new animation against this setting; it's easy to forget one.

An Accessibility Audit Prompt

```
Audit apps/platter for accessibility. For every screen, check: each
touchable has a role and a clear label; related text is grouped
  into one
announcement; status changes are announced; decorative art is
  hidden;
targets are at least 44x44; layout survives the largest text
  size; and
nothing is shown by color alone. Write the findings as a table
  (screen,
problem, fix), fix them, add or update a test for each fix using
  the
accessible names, and run the checks.
```

Localization

The apps are written for India, in English, with prices in rupees. Real apps in India often support Hindi and other languages; global apps support dozens. Three layers are involved:

1. **Formatting.** Prices, dates, times and numbers depend on the locale. The apps already format money with `Intl.NumberFormat('en-IN')` (so ₹1,23,456 uses Indian grouping) and times with the locale. Never build a price or date by gluing strings together.
2. **Translation.** Every string a user can see moves into a message file per language, looked up by a key. A library such as `i18next` with `expo-localization` (which reports the device's languages) handles lookup, plurals and fallbacks.

3. **Layout.** Translations can be 30 to 50 percent longer than English, and right-to-left languages such as Arabic and Urdu mirror the layout. Use `start` and `end` rather than `left` and `right` in styles, and check that icons with direction (arrows, chevrons) flip.

Plurals deserve care. The apps already handle "1 item" versus "2 items" and "1 minute" versus "6 minutes" in tested helpers (Chapter 19). Other languages have more plural forms than English; a translation library's plural rules handle them, so give each message its plural variants rather than writing `if (n === 1)` in a screen.

A prompt to start:

```
Prepare Minutely for Hindi. Move every user-visible string into
en.json and hi.json files using i18next and expo-localization, keep
prices and dates formatted with Intl for the current locale, use
  plural
rules for counts, and leave the Hindi values as English
  placeholders
marked TODO for a translator. Add a test that renders Home in each
language. Don't translate anything yourself.
```

The last line matters. Machine translation of an interface is a starting point at best; for a real release, have a fluent speaker translate and review the strings in context.

Key Takeaways

- Make accessibility a project rule from the first session, and test through the same labels screen readers use.
- Give every touchable a role and a short, specific label; group related text; announce changes; hide decoration.
- Support large text (with a sensible cap), 44-point targets, sufficient contrast and reduce motion, and never use color alone.
- Localize in three layers: formatting with `Intl`, translation through message files with plural rules, and layouts that survive long text and right-to-left languages.
- Test with TalkBack and VoiceOver yourself, and have real translators review real translations.

CHAPTER 22 · PRODUCTION

Testing: Unit, Component, API and End to End

When Claude writes most of the code, tests are how you know it works, and how Claude knows it hasn't broken something three screens away. The apps in this book were built with tests from the first session: more than 300 Jest tests across the apps and packages, the server's own Vitest suite, and 80 more in the TV app. This chapter explains the four layers they use, adds the fifth, end-to-end tests on a device with Maestro, and shows how to make Claude a disciplined tester.

The Testing Pyramid for a Mobile App

Layer	What it checks	Tool here	Speed
Unit	Pure logic: bills, filters, ETAs, plurals	Jest, Vitest	Milliseconds
Component	One component's look and behavior	Jest with React Native Testing Library	Fast
Screen	A whole screen against a fake server	Jest with React Native Testing Library	Seconds
API	Server routes, rules and money	Vitest on the server	Fast
End to end	The real app on a device or emulator	Maestro	Minutes

The lower layers catch most bugs cheaply; the top layer catches what only a real device shows: navigation, the keyboard, permissions, the back button, a slow network.

Unit Tests: Logic Lives in Pure Functions

The single most useful habit in this codebase is that every rule worth testing lives in a plain function, outside any component: `pendingSummary` (Chapter 19), `etaHeadline`, `countdownWindow`, story ordering, the return reasons, the tap detector with its injected clock. Those functions are tested directly, with exact inputs and expected outputs, in milliseconds.

When you review Claude's code, look for logic inside components. "Move this rule into a function in `lib/` and test it" is one of the most valuable prompts you can give.

Screen Tests Against a Fake Server

The screen tests render a real screen with a real React Query client, but with an API client whose `fetch` is replaced by a small in-memory server. The test then behaves like a user: it presses buttons found by their accessible names, and checks what's on screen and what the server received.

```
  it('adds to the cart at once and the cart bar follows every tap',
  ↪ async () => {
    const user = userEvent.setup();
    const { client, calls, cart } = minutelyServer();
    await renderScreen(<HomeScreen />, client);
    await user.press(await screen.findByRole('button', { name: 'Add
    ↪ Shimla Apple' }));
    expect(screen.getByRole('button', { name: '1 item, ₹189. View cart'
    ↪ })).toBeOnTheScreen();
    await user.press(screen.getByRole('button', { name: 'Add one more
    ↪ Shimla Apple' }));
    expect(await screen.findByRole('button', { name: '2 items, ₹378.
    ↪ View cart' })).toBeOnTheScreen();
    expect(screen.getByText('Saved ₹62')).toBeOnTheScreen();
    await waitFor(() => expect(cart().itemCount).toBe(2));
    expect(calls.filter((c) => c.method === 'PUT').map((c) =>
    ↪ c.body)).toEqual([
      { productId: 'g-001', quantity: 1 },
      { productId: 'g-001', quantity: 2 },
    ]);
    await user.press(screen.getByTestId('cart-bar'));
    expect(router.push).toHaveBeenCalledWith('/cart');
```

```
  });
```

This one test checks a lot: the ADD button turns into a stepper, the cart bar's name and total follow each tap, the savings appear, the server's cart ends up with two apples, and the app sent exactly two `PUT` requests with the right bodies, no more. Because it finds elements by role and label, it also proves those labels exist for screen readers (Chapter 21).

Chapter 18 told the story of one flaky screen test and its fix. The lesson is worth repeating: when a screen test fails only sometimes, look for timers and animations that outlive the test, and fix the cause rather than adding retries.

API Tests on the Server

Money, permissions and state changes are decided on the server, so they're tested there: bills for every app with every fee and coupon, quantity limits, order transitions, who can see whose orders, the event stream. These tests start the Hono app in memory against a fresh SQLite database, which takes milliseconds, so there are many of them.

End to End With Maestro

Everything above runs in Node. The last layer runs the real app. Maestro is a popular tool for this in React Native: you describe a journey in a short YAML file, and Maestro drives an Android emulator or iOS simulator through it, waiting automatically for screens to settle.

A flow for Minutely's main journey looks like this:

```
appId: app.minutely.example
---
- launchApp:
    clearState: true
- tapOn: "Mobile number"
- inputText: "9876543210"
- tapOn: "Continue"
- inputText: "123456"
- assertVisible: "Delivery in .* minutes"
- tapOn: "Add Shimla Apple"
- assertVisible: "1 item, ₹189. View cart"
- tapOn: "1 item, ₹189. View cart"
- tapOn: "Proceed to pay"
```

```
- tapOn: "Pay .*"
- assertVisible: "Order confirmed"
```

Run it with `maestro test flows/minutely-order.yaml`. Maestro finds elements by their visible text or accessibility label, so the same labels that serve screen readers and Jest serve Maestro too. Text is matched as a regular expression, which is why `"Delivery in .* minutes"` matches any ETA.

> **Note:** The flows in this chapter couldn't be run for this book, because the build machine had no hardware acceleration for emulators. The labels they use are the ones the screen tests check. Run the flows on your own computer, and expect to adjust a step or two, such as how the one-time code field takes focus.

A good set of end-to-end flows is small: one per critical journey (sign in, order, pay, track; browse, add, return; post, like, message), run before every release and, ideally, on every pull request with a cloud service. EAS Workflows (Chapter 26) can run Maestro flows against a fresh build.

Testing What Tests Can't

Some things need a human with a phone:

- **Real devices:** a cheap Android phone, a recent iPhone, and a tablet if you support one.
- **Bad networks:** turn on a network throttle or use the phone in a lift. Every request needs a loading state, an error state and a retry.
- **Interruptions:** a phone call during payment, switching apps while an order is placed, the phone locking during tracking.
- **System settings:** largest text, dark mode, reduce motion, TalkBack and VoiceOver, another language.

Write these as a checklist (Appendix B has one) and go through it before each release.

Making Claude a Disciplined Tester

The project's rules said "New logic needs a test" and "Before saying a task is done, run typecheck, lint and tests, and fix everything they report." Those two lines did more for quality than any other part of the prompts. A few more habits:

- **Ask for a failing test first** when fixing a bug: "Write a test that reproduces this, show me it failing, then fix it."
- **Ask how a test could pass wrongly.** Claude is good at spotting tests that would pass even if the feature broke.
- **Never accept skipped tests or raised timeouts** as fixes.
- **Read the test names.** Good names read like a specification: "adds to the cart at once and the cart bar follows every tap".

```
Add Maestro flows for Platter's three critical journeys (sign in
  and
order, pay and track, rate a delivered order) in flows/platter/,
  using
the accessibility labels the screen tests already use. Then list
  any
screen where a label a flow needs is missing, and add it with a
  test.
```

Key Takeaways

- Test in layers: pure logic, components, screens against a fake server, server routes, and a few end-to-end flows on a device.
- Keep rules in pure functions; it's what makes most of the code testable in milliseconds.
- Find elements by role and accessible name, so tests, screen readers and Maestro all agree.
- Keep end-to-end flows few and focused on critical journeys, and run them before every release.
- Make "new logic needs a test" and "run the checks before saying done" project rules for Claude.

CHAPTER 23 · PRODUCTION

Payments for Real: Razorpay, Stripe, Apple Pay, Google Pay and UPI

Every app in this book has a payment sheet that says, honestly, "This is a demo: no money is charged." This chapter shows how to replace it with real payments, in the patterns professional apps use, and what's different about taking money in a mobile app.

> **Warning:** Payment rules, fees, onboarding and store policies change, and they differ by country. Check each provider's current documentation and your account dashboard before you build, and get advice on tax and legal requirements for your business. This chapter teaches the technical pattern; it isn't financial or legal advice. None of the live payment flows in this chapter could be run for this book.

First, Which Store Rules Apply?

Before choosing a provider, check what you're selling. Both app stores separate two kinds of purchases:

- **Physical goods and services used outside the app**: food, groceries, products, rides, tickets. Platter, Cartly and Minutely are all this kind. You

use a payment provider such as Razorpay or Stripe, and the stores take no commission.

- **Digital goods and features used in the app**: subscriptions to content, premium features, coins, extra filters. These generally must use the store's own in-app purchase system (Google Play Billing, Apple's In-App Purchase), with the store's commission. A streaming subscription like Reelhouse's (Chapter 30) falls here.

The exact rules, and the exceptions that some countries now require, are in the Google Play Payments policy and Apple's App Review Guidelines (section 3.1). Read the current versions before you design checkout; getting this wrong is a common reason for rejection.

The Golden Rules Still Apply

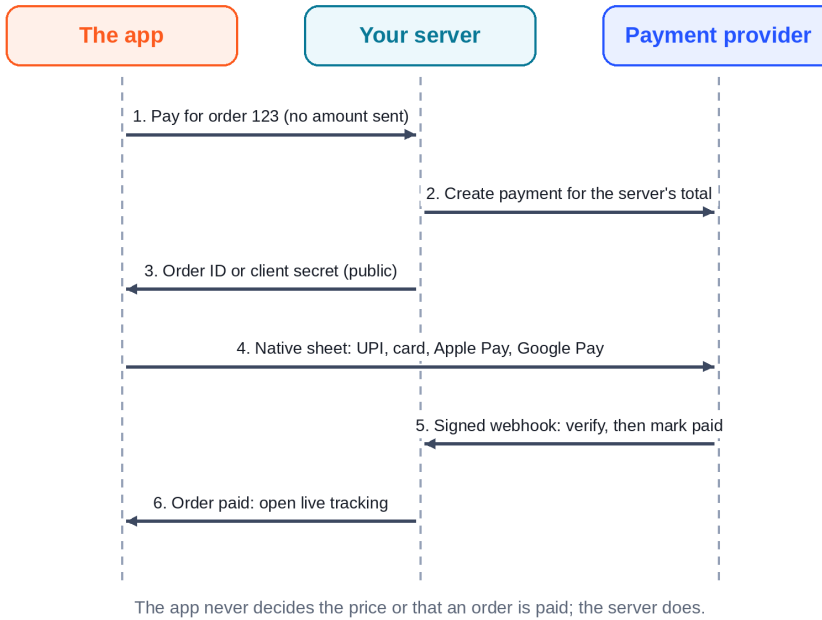
A phone doesn't change the fundamentals of safe payments:

1. **The server decides the amount.** The app sends the cart; the server prices it and creates the payment for that amount. The apps already work this way (Chapter 6): the bill comes from the server.
2. **Secret keys never ship in the app.** Anything in your app bundle can be extracted. The app gets only public identifiers.
3. **An order is paid only when the server has verified it**, with a signature check or a webhook from the provider, never because the app said so.
4. **Handle every message exactly once.** Providers retry webhooks, and users tap twice.
5. **Test mode first.** Use test keys and test cards until everything is proven.

The Flow in a Mobile App

Every provider below follows the same shape:

1. The user taps "Pay ₹683". The app asks **your server** to create a payment for the order.
2. The server prices the order, asks the provider to create a payment (a Razorpay order or a Stripe PaymentIntent), stores its ID with your order, and returns the public details (an order ID or a client secret).



The payment flow. The app never sets the price; the order is marked paid only after the server verifies a signed message from the provider.

3. The app opens the provider's **native payment sheet** with those details. The user pays with UPI, a card, Apple Pay or Google Pay inside the provider's own interface.
4. The provider tells the app the result and, separately, tells **your server** with a signed webhook.
5. The server verifies the signature, checks the amount and currency, and marks the order paid. The app moves to tracking when the server says so.

The app's job is small and safe: show the sheet and wait for the server. It never decides that an order is paid.

Razorpay for India

Razorpay supports UPI, cards, netbanking and wallets, and offers a React Native SDK (`react-native-razorpay`) that opens its checkout. Because it contains native code, it doesn't run in Expo Go; you use a development build (Chapter 26). Test the Android build early: developers report different

experiences with whether extra native configuration is needed, so check the library's current README for Expo.

After a successful checkout, the app receives a payment ID, the order ID and a signature, and sends them to your server. The server recomputes an HMAC-SHA256 of `order_id|payment_id` with its key secret, compares it with a timing-safe comparison, and only then marks the order paid. The webhook (`payment.captured`) is the backup that catches users who close the app at the wrong moment.

UPI

UPI is the most used way to pay in India, and on a phone it's especially smooth: the user picks a UPI app on their phone, approves the payment there, and returns. Razorpay's checkout handles this "intent" flow and the "collect" flow (a request sent to a UPI ID). Two things to design for:

- **The user leaves your app.** Your screen must survive being backgrounded, and the result may arrive only through the webhook. Show "Waiting for payment confirmation" and poll your server or listen on the order's event stream.
- **Pending isn't failed.** A UPI payment can take a while to confirm. Don't tell the user it failed until the provider says so.

Stripe, With Apple Pay and Google Pay

Stripe's React Native SDK (`@stripe/stripe-react-native`) offers a ready-made **PaymentSheet** that collects cards and wallets, and a `usePlatformPay` hook for Apple Pay on iOS and Google Pay on Android. In outline:

```
const { initPaymentSheet, presentPaymentSheet } = useStripe();

async function pay(orderId: string) {
  // 1. Your server creates the PaymentIntent and returns its
    client secret.
  const { clientSecret } = await api.createStripePayment(orderId);
  // 2. Configure and present Stripe's native sheet.
  await initPaymentSheet({
    merchantDisplayName: 'Platter',
    paymentIntentClientSecret: clientSecret,
```

```
    applePay: { merchantCountryCode: 'US' },
    googlePay: { merchantCountryCode: 'US', testEnv: true },
  });
  const { error } = await presentPaymentSheet();
  if (error) return showError(error.message);
  // 3. Wait for the server: the webhook marks the order paid.
  router.replace(`/order/${orderId}`);
}
```

The code is an outline, not a drop-in: check names and options against the SDK's current documentation. Apple Pay also needs a merchant ID and a payment processing certificate set up in your Apple Developer account and in Stripe, and the SDK's config plugin to add the entitlement; Google Pay needs the Google Pay API enabled for production. Neither wallet works in Expo Go.

On the server, the `payment_intent.succeeded` webhook, with Stripe's signature header verified over the raw request body, is what marks the order paid.

> **Note:** At the time of writing, Stripe's sign-up for new businesses in India is by invitation. Many Indian apps use Razorpay (or another Indian provider) for domestic customers and Stripe for international ones, chosen on the server per region.

Designing the Payment Experience

The apps' payment screens already follow the patterns users trust; keep them when the money becomes real:

- **The amount and method on the button**: "Pay ₹683 with UPI".
- **Saved methods first**, with the last used one selected.
- **Clear progress** while the provider's sheet is open and while the server confirms.
- **Plain failure messages** with a way forward: "Payment didn't go through. You weren't charged. Try again or choose another method."
- **No double charges**: disable the pay button while a payment is in progress, and make the server's "create payment" call idempotent for the same order.

- **Refunds** shown in the order with an amount, a method and a date (Chapter 14).

A Prompt for Real Payments

```
Replace Platter's demo payment with Razorpay in test mode. The
  server
creates a Razorpay order for the server-calculated total, stores
  its ID
on our order, and verifies both the checkout signature and the
payment.captured webhook (HMAC-SHA256, timing-safe, raw body),
  checking
amount and currency, and processing each event once. The app opens
Razorpay checkout with react-native-razorpay in a development
  build,
then waits for the order's event stream to say "paid". Test the
  server
logic offline with known good, tampered and replayed messages.
  Never use
live keys. Finish by listing what can only be verified with real
  test
keys on a device, and anything in Razorpay's rules you're unsure
  about.
```

The last sentence is the most important. Claude can't reach the provider from your build machine, and a list of honest unknowns is a checklist for you.

Key Takeaways

- Physical goods and services use a payment provider; digital goods and features usually must use the store's in-app purchases. Check the current store rules first.
- The server sets the amount, holds the secrets and marks orders paid only after verifying a signed message.
- The app's job is to open the provider's native sheet and wait for the server.
- Design for UPI's app switching and pending states; set up Apple Pay and Google Pay with their certificates and entitlements in development builds.
- Put the amount on the button, prevent double payments, and explain failures plainly.

CHAPTER 24 · PRODUCTION

Push Notifications, Deep Links and Analytics

Three systems connect a professional app to the world outside it. Push notifications bring users back ("Your order is out for delivery"). Deep links take them straight to the right screen from a notification, an email, a message or a website. Analytics tell you what they actually do once they're there. This chapter adds all three to the apps, with care for privacy.

Push Notifications

The apps already have a notifications screen (Cartly's, in Chapter 14) and live order updates while the app is open. Push extends both to when the app is closed.

How Push Works

1. The app asks the user for permission to send notifications.
2. If they agree, the operating system gives the app a **push token**, an address for this app on this device.
3. The app sends the token to **your server**, which stores it against the signed-in user.
4. When something happens (an order ships), your server sends a message to that token through a push service: Firebase Cloud Messaging for

Android, Apple Push Notification service for iOS, or Expo's push service, which forwards to both.

5. The phone shows the notification, and when the user taps it, the app opens on the right screen.

With Expo, `expo-notifications` handles permission, the token and tapping. The outline:

```
import * as Notifications from 'expo-notifications';
import Constants from 'expo-constants';

export async function registerForPush(api: Api) {
  const { status } = await Notifications.requestPermissionsAsync();
  if (status !== 'granted') return;
  const projectId = Constants.expoConfig?.extra?.eas?.projectId;
  const { data: token } = await
    Notifications.getExpoPushTokenAsync({ projectId });
  await api.savePushToken(token);
}
```

Things to know before you start:

- **Push needs a development build.** From Expo SDK 53, Expo Go doesn't support remote push notifications; local notifications still work there. Android needs Firebase Cloud Messaging credentials, which EAS can store for you; iOS needs an Apple push key.
- **Android 13 and later ask the user**, as iOS always has. Before Android 13, notifications were allowed by default.
- **Test on physical devices**, or an Android emulator with Google Play services.

Asking at the Right Moment

The single biggest mistake is asking for notification permission at first launch, before the user knows why. Professional apps ask in context: right after the first order is placed, with a short explanation of what they'll get ("We'll tell you when your order is on its way"). If the user says no, the app keeps working, and the account screen offers a way to turn notifications on later.

Sending Notifications That Help

- **Transactional first.** Order confirmed, out for delivery, delivered, refund completed: users want these.
- **Marketing sparingly, and with consent.** Offers and "your cart misses you" messages need the user's agreement in many places, and they're the main reason people turn notifications off. Give separate switches for order updates and offers.
- **Use channels on Android.** Separate notification channels ("Order updates", "Offers") let users silence one without the other.
- **Make every notification a deep link** to the screen it's about.
- **Clean up tokens.** When a push service says a token is no longer valid, delete it; when a user signs out, unlink the token from their account.

Deep Links

A deep link opens the app on a specific screen. Expo Router makes this almost free: every file in `app/` is already a route, so `app/order/[id].tsx` answers `/order/123`.

There are two kinds of link:

- **Custom scheme links** such as `platter://order/123`. Each app in this book declares its scheme in `app.json` (`"scheme": "platter"`). They're easy, and good for notifications and internal use, but any app can claim a scheme, and they don't work if the app isn't installed.
- **Universal links (iOS) and App Links (Android)** such as `https://platter.example/order/123`. They're ordinary web addresses that open the app if it's installed and the website if not. They need proof that you own the domain: an `apple-app-site-association` file and an `assetlinks.json` file served from the site's `/.well-known/` folder, plus the associated domain and intent filters in the app's configuration.

Use universal links for anything shared outside the app: emails, messages, social posts, QR codes.

Handle Every Link Safely

A deep link is input from outside the app, so treat it like any other untrusted input:

- **Check that the user may see the target.** A link to someone else's order must show "not found", which the server already enforces.
- **Send signed-out users to sign in first**, then on to the target.
- **Ignore unknown links rather than crashing.** Minutely's banner links show the idea in miniature:

```
/**
 * Where an offer banner goes. Seed data links categories as
 * "/category/fruits" and coupon offers as "/offers" (the cart's coupon
 * sheet). Anything else is ignored rather than routed to a 404.
 */
export function bannerHref(link: string | null | undefined): string |
↪ null {
  if (!link) return null;
  const category = /^\/category\/([a-z0-9-]+)$/.exec(link);
  if (category) return `/category/${category[1]}`;
  if (link === '/offers') return '/cart?coupons=1';
  if (link === '/search') return '/search';
  return null;
}
```

Only known shapes become routes; anything else is ignored.

Analytics

Analytics answer the questions you can't answer by asking: where do people drop out of checkout? Which banner gets tapped? Did the new filter sheet help people find products?

Measure a Few Things Well

Start with a small **event plan** written down before any code: a dozen events that describe your main journey, each with a name and a few properties.

Event	Properties
`app_opened`	source (icon, notification, link)
`search_performed`	query length, results count
`product_viewed`	product ID, category, position in list
`cart_item_added`	product ID, quantity, screen
`checkout_started`	item count, total band
`payment_completed`	method, total band
`order_delivered`	minutes from order

From these, you can build the funnel that matters most: viewed, added, started checkout, paid. Choose an analytics provider (Firebase Analytics, Amplitude, Mixpanel, PostHog and others all have React Native SDKs), and wrap it in one small module of your own, `track(event, properties)`, so screens never call the provider directly and you can switch providers or add consent checks in one place.

Privacy Comes First

- **Collect the minimum.** Don't send names, phone numbers, emails, addresses or exact amounts in analytics events. Use IDs and bands ("₹500 to ₹999").
- **Ask where the law requires it.** Rules on consent differ by country; India's Digital Personal Data Protection Act, Europe's GDPR and US state laws all apply to apps used there.
- **Declare it.** Everything you collect must appear in the Play Data safety form and Apple's privacy details (Chapters 27 and 28), including data collected by SDKs you add.
- **Apple's tracking rule.** If you track users across other companies' apps or websites (for advertising, for example), iOS requires you to ask permission with the App Tracking Transparency prompt.

A Prompt for All Three

```
Add push notifications to Platter for order updates only: ask for
permission after the first order is placed, with a one-line
  explanation;
store the Expo push token on the server against the user; send a
  push on
each order status change from the simulator; and open the order
  screen
when a notification is tapped, after sign-in if needed. Add an
"Order updates" switch in Profile. Then add an analytics module
  with a
track() function that logs to the console in development, and
  call it
for the seven events in docs/EVENTS.md, with no personal data in
  any
event. Test the server side and the routing logic.
```

Key Takeaways

- Push needs permission, a token stored on your server, and a development build; ask for permission in context, not at launch.
- Send transactional notifications freely, marketing sparingly and with consent, and make every notification a deep link.
- Expo Router turns every screen into a link; use universal links and App Links for anything shared outside the app.
- Treat deep links as untrusted input: check permissions, handle signed-out users, and ignore unknown shapes.
- Plan a small set of analytics events, wrap the provider in one module, collect no personal data, and declare everything you collect.

CHAPTER 25 · PRODUCTION

Security and Privacy

A mobile app runs on a device you don't control, talks over networks you don't control, and is installed by people who trust you with their phone number, address, and sometimes their payment details. Security and privacy are where "it works" and "it's ready" are furthest apart. This chapter walks through the main risks for apps like the ones in this book, what the code already does, and what must change before real users arrive.

Assume the App Is Public

Everything inside your app bundle can be read. Anyone can download it from the store, unpack it and read your JavaScript, configuration and images. That one fact decides most mobile security:

- **No secrets in the app.** API secret keys, payment secret keys, database passwords and signing keys belong on the server. Environment variables that start with `EXPO_PUBLIC_` are built into the bundle, so they're public by definition; the apps use one only for the API address.
- **The server makes every decision that matters.** Prices, bills, coupons, whether an order is paid, who can see what. The apps already work this way (Chapter 6), so a modified app can't give itself a discount.
- **Hiding logic in the client isn't security.** Obfuscation slows people down; it doesn't stop them.

Storing Tokens Securely

When a user signs in, the server returns a token that proves who they are. The apps store it so the user stays signed in. The shared session code accepts any storage with three methods:

```
/** The subset of AsyncStorage (or localStorage) the session needs. */
export interface TokenStorage {
  getItem: (key: string) => Promise<string | null>;
  setItem: (key: string, value: string) => Promise<void>;
  removeItem: (key: string) => Promise<void>;
}
```

In the demo, that storage is AsyncStorage, which is a plain, unencrypted file. That's fine for a demo and wrong for production. On a real device, store tokens in the **Keychain** on iOS and the **Keystore**-backed encrypted storage on Android, which `expo-secure-store` provides. Because the session takes an interface, the change is small:

```
Store the session token with expo-secure-store instead of
  AsyncStorage
in all four apps, through the existing TokenStorage interface,
  keeping
the web build on localStorage. Add a test for the adapter, and
  make sure
a token stored by the old version is moved over once and then
  deleted.
```

The last sentence is the kind of detail that matters in a real release: existing users shouldn't be signed out by an update.

Store the minimum. The token is enough; don't keep the user's profile, address or order history in plain storage. Cached data should be cleared on sign-out, which the apps already do (Chapter 14).

Authentication Done Right

The demo logs in with a fixed one-time code (123456), which is obviously only for development. A real one-time password login needs:

- **Codes from an SMS provider**, random, six digits, valid for a few minutes, usable once.

- **Rate limits** on sending codes (per phone number and per device) and on guessing them, with a lockout after a few wrong tries. Without limits, attackers can run up your SMS bill or guess codes.
- **Short-lived access tokens** (minutes to an hour) with longer-lived refresh tokens that can be revoked, so a stolen phone or a leaked token can be cut off from the server.
- **Sign out everywhere** in the account screen.

Apple also requires that if your app offers third-party or social sign-in, it offers an equivalent privacy-focused option such as Sign in with Apple, with some exceptions; check the current App Review Guidelines.

The Network

- **HTTPS only.** Both platforms block plain HTTP by default in release builds; don't turn that protection off. The demo uses `http://localhost` only in development.
- **Validate everything on the server.** The demo server validates request bodies with Zod schemas, so a malformed request is refused before it reaches any logic; make sure every route in your own server does the same.
- **Lock down CORS and add rate limits on the server.** The demo server allows requests from any origin, for development convenience; a production API should allow only its own web origins and limit requests per user and per IP address.
- **Certificate pinning** (accepting only your server's certificate) adds protection against interception on compromised networks, but it can lock users out if certificates change unexpectedly. Use it only with a plan for rotation.

Permissions

Ask only for what you need, only when you need it, and explain why. The apps need almost nothing: notifications (Chapter 24), and in a real version perhaps location to suggest the delivery address, and the camera or photo library for Lumo. Each permission:

- is requested at the moment it's used, never at launch;
- comes with a clear purpose string (iOS requires one in the app's configuration, and reviewers read them);
- has a graceful fallback when refused (type the address instead of locating it).

Prefer the photo picker, which lets users choose specific photos without giving the app the whole library, over broad storage permissions.

Privacy Is a Product Feature

Users and stores both expect it:

- **Collect the minimum**, keep it for the shortest time, and know where every piece of personal data goes.
- **Account deletion.** Both Apple and Google require that apps with accounts let users request deletion of their account and data; Google Play also requires a way to request it from the web, outside the app. Build it from the start.
- **A privacy policy** that matches what the app does, linked from the app and the store listing.
- **Store declarations.** Google Play's Data safety form and Apple's privacy details ("nutrition labels") must describe what your app and its SDKs collect, and why. Apple also requires a privacy manifest declaring data use and "required reason" APIs, including in third-party SDKs; Expo lets you set the app's manifest entries in `app.json`.
- **Laws.** India's Digital Personal Data Protection Act, Europe's GDPR and US state privacy laws may all apply to your users. Get advice.

A Security Review Prompt

Ask Claude for a review before every release, and give it the shape of the answer you want:

```
Do a security and privacy review of this repository before release.
Check for: secrets in the app or in git history; tokens in
  unencrypted
storage; anything the client decides that the server should;
  endpoints
without authentication, authorization checks or validation; missing
rate limits on login and OTP; CORS and HTTP settings; permissions
  asked
without need; personal data in logs or analytics; and account
  deletion.
Rank findings by severity with file and line, and explain how
  each could
be exploited in one sentence. Don't change code yet.
```

Then fix the findings one by one, each with a test. A review like this is a strong first pass, not a guarantee; for an app that handles payments or sensitive data, an independent security review before launch is money well spent.

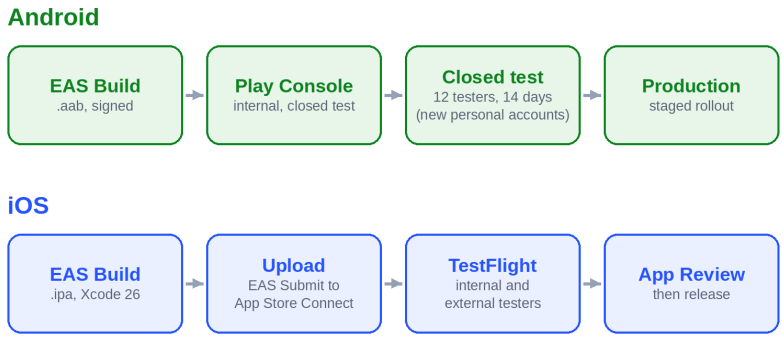
Key Takeaways

- Everything in the app bundle is public: keep secrets and decisions on the server.
- Store tokens in the Keychain or Keystore with `expo-secure-store`, keep little on the device, and clear it on sign-out.
- Real OTP login needs random single-use codes, rate limits, short-lived tokens and revocation.
- Use HTTPS only, validate on the server, restrict CORS and rate-limit the API.
- Ask for permissions in context with clear reasons, build account deletion, and make store privacy declarations match the app and its SDKs.

CHAPTER 26 · PRODUCTION

Builds and Releases With EAS

Up to now, the apps have run in a browser and in development. Shipping them means producing signed native builds: an Android App Bundle for Google Play and an iOS app archive for the App Store. Expo Application Services (EAS) does this in the cloud, so you can build an iOS app without owning a Mac. This chapter sets up builds, versions, signing and over-the-air updates for the apps.



EAS Update ships JavaScript-only fixes to installed apps without a store release.

From build to store on each platform. EAS Update ships JavaScript-only fixes to installed apps without a store release.

Expo Go, Development Builds and Release Builds

There are three ways to run an Expo app, and knowing which you need saves a lot of confusion:

- **Expo Go** is an app from the stores that runs your JavaScript with a fixed set of native libraries. It's great for the first hour of a project, but it can't include native code you add (payments, push, some maps), and it doesn't support remote push from SDK 53 on.
- **A development build** is your own app with your native libraries and Expo's developer tools. You install it once on your phone or emulator and then iterate on JavaScript as quickly as in Expo Go. From the moment you add native libraries such as Stripe, Razorpay or notifications, this is how you develop.
- **A release build** is what users get: optimized, signed, without developer tools. It's what you upload to the stores, and what you should test performance on (Chapter 20).

Setting Up EAS

Install the command-line tool, sign in with a free Expo account, and link the app:

```
npm install -g eas-cli
eas login
cd apps/platter
eas init
eas build:configure
```

`eas build:configure` creates an `eas.json` with build profiles. A typical one:

```
{
  "cli": { "appVersionSource": "remote" },
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal"
    },
    "preview": {
      "distribution": "internal",
      "channel": "preview"
```

```
    },
    "production": {
      "autoIncrement": true,
      "channel": "production"
    }
  },
  "submit": { "production": {} }
}
```

- **development** builds the development client, shared with your team as an installable file.
- **preview** builds a release-like app for testers outside the stores.
- **production** builds the store version and increments the build number each time.

> **Tip:** In a monorepo like this book's, run EAS commands from the app's folder (`apps/platter`), and check Expo's monorepo guide if a build can't find a workspace package. Ask Claude to set up `eas.json` for all four apps and verify the first build of each.

App Identity

Each app's `app.json` already has the fields that identify it in the stores:

```
{
  "expo": {
    "name": "Platter",
    "slug": "platter",
    "scheme": "platter",
    "version": "1.0.0",
    "orientation": "portrait",
    "userInterfaceStyle": "automatic",
    "ios": { "bundleIdentifier": "app.platter.example",
    ↪ "supportsTablet": false },
    "android": { "package": "app.platter.example", "edgeToEdgeEnabled":
    ↪ true },
    "web": { "output": "single", "bundler": "metro", "name": "Platter",
    ↪ "shortName": "Platter", "themeColor": "#FC5A1E" },
    "plugins": [
      "expo-router",
      "expo-font",
      ["expo-splash-screen", { "backgroundColor": "#FC5A1E" }]
    ],
    "experiments": { "typedRoutes": false }
  }
}
```

- `name` is what appears under the icon.
- `ios.bundleIdentifier` and `android.package` are permanent IDs. They can never change once the app is published, so choose them carefully, in reverse-domain form on a domain you own (`com.yourcompany.platter`). The `.example` IDs in this book's code are placeholders that you must replace.
- `version` is the version users see ("1.0.0"). The build number (iOS) and version code (Android) are separate numbers that must increase with every upload; with `appVersionSource: "remote"` and `autoIncrement`, EAS manages them for you.

Before the first store build, add the real **icon** (1024 by 1024 pixels, no transparency for iOS), the **adaptive icon** layers for Android, and the **splash screen**.

Building

```
eas build --platform android --profile production
eas build --platform ios --profile production
```

The Android build produces an `.aab` (Android App Bundle), which Google Play turns into optimized downloads for each device. The iOS build produces an `.ipa`. Both run on EAS's servers and take several minutes; the command prints a link to follow progress and download the result.

Signing

Every app must be signed, and the signing keys matter:

- **Android:** EAS can generate an upload keystore and keep it for you. Google Play App Signing (required for new apps) holds the final app signing key; you sign uploads with your upload key. If you lose the upload key, Google can reset it; keep a backup anyway with `eas credentials`.
- **iOS:** EAS can create and manage your distribution certificate and provisioning profiles through your Apple Developer account. You need a paid membership ($99 a year).

Submitting

EAS Submit uploads a build to the stores:

```
eas submit --platform android --profile production
eas submit --platform ios --profile production
```

For Android, the very first upload of a new app must be done by hand in the Play Console; later ones can use EAS Submit with a Google service account key. For iOS, EAS Submit uploads to App Store Connect, where the build appears in TestFlight after processing. Chapters 27 and 28 continue from there.

Over-the-Air Updates

Most changes to a React Native app are JavaScript: a fixed bug, a new label, a changed screen. **EAS Update** delivers those to installed apps without a store release. The app checks for an update when it starts and applies it, by default, on the next launch.

```
eas update --channel production --message "Fix coupon rounding"
```

Rules for using it safely:

- **Only JavaScript and assets can be updated.** A change to native code, a new native library, or a permission needs a new store build. The **runtime version** in `app.json` marks which builds an update is compatible with; the `fingerprint` policy computes it from your native code, so an update can't reach a build it would break.
- **Test the update first.** Publish it to the `preview` channel, check it in a preview build, then publish the same change to `production`.
- **Roll back if needed.** EAS can republish a previous update; keep the last good one in mind.
- **Stay within store rules.** Both stores allow updating your app's code this way, but not changing its main purpose or adding features that would have needed review. Use updates for fixes and small improvements; ship features through the stores.

Versions and Release Notes

Use semantic versions: increase the third number for fixes (1.0.1), the second for new features (1.1.0), and the first for big changes (2.0.0). Keep a `CHANGELOG.md`, and ask Claude to draft release notes from the git log in plain, user-facing words ("You can now reorder a past order with one tap"), not commit messages.

Automating It

EAS Workflows (YAML files in `.eas/workflows/`) or any CI service can run the checks, build, run Maestro flows (Chapter 22), and submit, on every merge to your main branch or every tag. A good first automation is small: on every pull request, run `npm run check`; on every tag, build and submit to the internal testing tracks.

```
Set up EAS for apps/platter: eas.json with development, preview and
production profiles, remote app versions with auto-increment,
  channels
for updates, and the fingerprint runtime version policy. Replace
  the
.example bundle IDs with com.mycompany.platter. Add an EAS
  workflow that
runs npm run check on pull requests and builds both platforms on
  tags
starting with v. Explain each setting in a short comment in the PR.
Don't run any build.
```

Key Takeaways

- Develop in a development build once you add native libraries; test performance and behavior in release builds.
- `eas.json` defines development, preview and production profiles; let EAS manage build numbers and signing credentials.
- Bundle IDs and package names are permanent: choose them carefully on a domain you own.
- `eas build` makes signed store builds in the cloud; `eas submit` uploads them.

- EAS Update ships JavaScript fixes without a store release; protect it with runtime versions, test on a preview channel first, and keep features for store releases.

CHAPTER 27 · PRODUCTION

Shipping to Google Play

Google Play is the larger of the two stores by number of users, especially in India, and it's the first store most teams ship to. This chapter takes Platter from a signed build to a production release on Play, including the testing requirement that catches many new developers by surprise.

> **Warning:** Store requirements change several times a year. Everything here was checked against Google's documentation in October 2026; check the Play Console's current requirements before you submit.

The Requirements in 2026

- **A Play Console developer account**: a one-time $25 registration fee, with identity verification. Organizations need a D-U-N-S number.
- **Target API level.** From 31 August 2026, new apps and updates must target Android 16 (API level 36). Android TV apps must target at least Android 14 (API level 34). Expo SDK 58 builds target the required level by default.
- **Android App Bundle (**`.aab`**)**, signed, with Google Play App Signing.
- **Testing for new personal accounts.** Personal developer accounts created after 13 November 2023 must run a **closed test with at least 12 testers who stay opted in for 14 days in a row** before they can apply for production access. Organization accounts are exempt.

- **Developer verification.** Google has begun requiring Android developers to register their apps. Registration became required for stores including Google Play in Brazil, Indonesia, Singapore and Thailand on 30 September 2026, with a global rollout planned for 2027. If you publish on Play with a verified account, you're largely covered; read the current Android developer verification pages.

Step 1: Create the App

In the Play Console, create an app: its default language, name, whether it's an app or a game, and whether it's free or paid. A free app can never become paid later.

Step 2: Set Up the Store Listing

The listing is your app's shop window:

- **App name** (up to 30 characters) and **short description** (up to 80).
- **Full description** (up to 4,000 characters): what the app does, its main features, written for people, not search engines.
- **App icon**: 512 by 512 pixels.
- **Feature graphic**: 1024 by 500 pixels, shown at the top of the listing.
- **Phone screenshots**: at least two; up to eight. Use the real app. This book's device-framed screenshots show how; the stores prefer plain screenshots or simple frames with short captions.
- **Category, contact email and privacy policy URL.**

Never use another company's name, logo or trademarks in your listing, and never claim features the app doesn't have.

Step 3: Complete App Content

The **App content** section holds the declarations Google reviews. Each needs care:

- **Privacy policy**, matching what the app collects.

- **App access**: if parts of the app need a login, give reviewers instructions and a working test account. For an OTP login like the apps', provide a test phone number and fixed code that work only for reviewers, or a reviewer bypass.
- **Ads**: whether the app contains ads.
- **Content rating**: a questionnaire that produces ratings for each region.
- **Target audience and content**: the age groups the app is for. Apps aimed at children have extra rules.
- **Data safety**: what data the app and its SDKs collect and share, why, whether it's encrypted in transit, and whether users can ask for deletion. This must match the app (Chapter 25).
- **Account deletion**: if users can create accounts, a link where they can request deletion from the web, as well as in the app.
- **Financial features, health, government** and other declarations if they apply.

Step 4: Internal Testing

Upload the first `.aab` by hand to the **internal testing** track and add up to 100 testers by email. Internal tests are available within minutes and are the quickest way to try the real store build on real phones. Install it, go through the release checklist (Appendix B), and fix anything you find.

Step 5: Closed Testing (12 Testers for 14 Days)

For a new personal account, create a **closed testing** track, add at least 12 testers (an email list or a Google Group), and send them the opt-in link. They must stay opted in for 14 consecutive days. Use the time well:

- Ask testers to use the app for real tasks and send feedback, with a simple form or an email address.
- Watch the Play Console's crash and ANR reports (Chapter 29).
- Ship fixes to the closed track as new builds; the 14 days continue.

After 14 days, apply for production access in the Dashboard. Google asks about your testing, what you learned and your launch readiness; answer specifically. Review of the application can take some days.

Step 6: Production With a Staged Rollout

Create a production release with your build and release notes, and choose a **staged rollout**: release to a small share of users first (for example 5 percent), watch crashes and reviews for a day or two, then increase to 20, 50 and 100 percent. If something goes wrong, halt the rollout; users who haven't updated stay on the old version.

The first review of a new app can take longer than later ones. Updates are usually reviewed within a day or two.

Common Reasons for Rejection

- **Broken or blocked access**: reviewers couldn't log in. Test the reviewer account yourself the day you submit.
- **Data safety mismatch**: the form says no data is collected, but an SDK collects device IDs.
- **Misleading metadata**: screenshots or descriptions that don't match the app, or that use other brands.
- **Permissions without clear need**, especially location in the background, SMS and call logs, which have strict policies.
- **Payments policy**: selling digital goods without Google Play Billing (Chapter 23).
- **Minimal functionality**: an app that's just a website in a frame, or does too little.

When a rejection arrives, read it carefully, fix exactly what it says, explain the fix in the appeal or resubmission notes, and resubmit. Claude is useful here: paste the rejection (without personal details) and ask it to find the cause in the code and the listing.

A Prompt for the Listing

```
Draft the Google Play listing for Platter: a 30-character name, an
80-character short description, and a full description under 4,000
characters with the main features as short paragraphs. Use plain,
  honest
language; mention no other company or brand; claim nothing the app
doesn't do. Then draft the Data safety answers from the code
  (what we
collect, why, shared or not, encrypted in transit, deletion),
  listing
every SDK we use and what it collects, and flag anything you're
  unsure
about for me to check.
```

Key Takeaways

- In 2026, Play requires a $25 account, target API 36 (API 34 for TV), signed App Bundles and, for new personal accounts, a 12-tester, 14-day closed test.
- Complete every App content declaration carefully; give reviewers working access.
- Test on the internal track first, use the closed test for real feedback, then release to production with a staged rollout.
- Most rejections come from access, data safety, metadata, permissions or payments; fix exactly what the reviewer reports.

CHAPTER 28 · PRODUCTION

Shipping to the App Store

Apple's App Store reviews every app and every update by hand, against its App Review Guidelines. That makes the first submission feel daunting, but the process is predictable once you know what reviewers check. This chapter takes Platter from an EAS build to a release on the App Store.

> **Warning:** Apple updates its requirements several times a year. Everything here was checked against Apple's developer news and documentation in October 2026; check App Store Connect's current requirements before you submit.

The Requirements in 2026

- **Apple Developer Program membership**: $99 a year. Organizations need a D-U-N-S number.
- **Build tools.** Since 28 April 2026, apps uploaded to App Store Connect must be built with Xcode 26 or later, using the iOS 26 SDK. EAS's build servers handle this when you use a current Expo SDK.
- **Minimum iOS version.** Since 9 September 2026, iOS apps uploaded must support iOS 13 or later as their minimum. Expo SDK 58 apps require a much newer iOS version than that anyway.
- **Age ratings** use Apple's updated rating system, required since 31 January 2026: answer the new questionnaire in App Store Connect.
- **A privacy policy, privacy details and a privacy manifest** (Chapter 25).

Step 1: Create the App Record

In App Store Connect, create a new app with its name, primary language, bundle ID (it must match `ios.bundleIdentifier` in `app.json`), and an SKU of your choosing. EAS Submit can create the record for you on the first upload.

Step 2: Upload and TestFlight

`eas submit --platform ios` uploads the build. After Apple processes it (usually minutes), it appears in **TestFlight**:

- **Internal testers**: up to 100 members of your App Store Connect team, available immediately.
- **External testers**: up to 10,000 people by email or public link. The first build for external testing goes through a short beta review.

Use TestFlight the way you'd use Play's closed test: real people, real tasks, real devices, with feedback sent from the TestFlight app.

Step 3: The Product Page

- **Name** (up to 30 characters) and **subtitle** (up to 30).
- **Description**, **keywords** (100 characters, comma-separated) and **promotional text** (which you can change without a new build).
- **Screenshots** for the iPhone display sizes App Store Connect asks for (the largest size can be scaled for smaller ones), and for iPad if you support it. The apps in this book are phone-only (`supportsTablet: false`).
- **App previews** (short videos), optional.
- **Support URL**, **marketing URL** and **privacy policy URL**.
- **Category** and **age rating**.

As with Play: no other company's names or logos, no claims the app can't back up.

Step 4: App Privacy

The **App Privacy** section asks what data the app collects, whether it's linked to the user's identity, and whether it's used for tracking. Apple shows the answers on your product page. They must cover third-party SDKs, and they must agree with your privacy manifest and your Play Data safety form; reviewers and users compare them.

Step 5: Review Information

This is where many first submissions fail. Provide:

- **Sign-in details**: a demo account that works, with any one-time code the reviewer will need. For OTP logins, a reviewer number with a fixed code is standard.
- **Notes**: anything a reviewer needs to see the app's features, such as "Order tracking moves through each stage within two minutes in the review environment."
- **Contact information** for someone who can answer quickly.

Step 6: Submit for Review

Choose the build, answer the export compliance question (most apps using only standard HTTPS encryption qualify for an exemption; you can declare this once in `app.json` with `ios.config.usesNonExemptEncryption: false`), choose manual or automatic release, and submit. Many reviews finish within a day or two. When approved, release the app manually, automatically, or as a **phased release** over seven days, which is Apple's version of a staged rollout.

What App Review Checks

Apple's guidelines are long, but most rejections of apps like these come from a handful of them:

- **App completeness (2.1)**: crashes, broken links, placeholder content, a demo account that doesn't work. Reviewers will find the one broken screen.

- **Accurate metadata (2.3)**: screenshots and descriptions that match the app.
- **Minimum functionality (4.2)**: the app must do something useful beyond a website in a frame.
- **Payments (3.1)**: digital goods and subscriptions must use In-App Purchase; physical goods and services must not (Chapter 23).
- **Sign in with Apple (4.8)**: required as an option in many apps that offer third-party sign-in.
- **Account deletion (5.1.1)**: apps that let users create accounts must let them start account deletion in the app.
- **Privacy (5.1)**: permissions requested only when needed, with clear purpose strings, and privacy details that match.
- **User-generated content (1.2)**: apps like Lumo, where people post and message, must offer a way to report content, block users, and filter objectionable material, and must act on reports.

If you're rejected, the message cites the guideline. Reply in App Store Connect's Resolution Center if you think the reviewer misunderstood, or fix and resubmit. Be specific and polite; reviews of resubmissions are usually quick.

Lumo and the Rules for Social Apps

Lumo shows why it pays to read the guidelines before you build. A real social app needs, from its first version:

- **Report** on every post, comment, story and message.
- **Block** on every profile.
- **A way to act on reports** quickly, with a moderation tool or service.
- **Terms of use** that users accept, forbidding objectionable content.
- **Age-appropriate defaults** for younger users.

None of these exist in the demo. Ask Claude to add them before you ever submit a social app.

```
Prepare Lumo for App Review guideline 1.2 (user-generated content).
Add Report to every post, comment, story and message (with
  reasons),
Block and Unblock on profiles (hiding blocked users everywhere), a
server route to record reports with a simple admin list, and terms
acceptance at sign-up. Add server and screen tests, and list
  anything
the guideline asks for that this doesn't yet cover.
```

Key Takeaways

- In 2026, App Store uploads need Xcode 26 and the iOS 26 SDK, a minimum iOS of 13 or later, the updated age-rating questionnaire, and a $99 yearly membership.
- Upload with EAS Submit, test with TestFlight, then complete the product page, privacy details and review information.
- Give reviewers a working demo account and clear notes; most rejections are about completeness, metadata, payments, privacy or account deletion.
- Use a phased release, and answer rejections specifically in the Resolution Center.
- Social apps need reporting, blocking and moderation before their first submission.

CHAPTER 29 · PRODUCTION

After Launch: Crashes, Reviews, Updates and Experiments

Launch day is the start of an app's life, not the end of its project. The apps users love are the ones that keep getting better, quickly and safely, based on what real people do. This chapter covers the loop that keeps a professional app healthy: watch, listen, fix, improve.

Watch: Crashes and Errors

The first thing to set up, before launch, is crash and error reporting. A service such as Sentry, Firebase Crashlytics or Bugsnag catches JavaScript errors and native crashes on users' phones and sends them to a dashboard with the stack trace, the device, the app version and the steps leading up to it.

- **Upload source maps** with every build and update, so stack traces point to your TypeScript, not minified code. The services' Expo integrations do this during EAS builds.
- **Watch the store dashboards too.** The Play Console's Android vitals report crashes and **ANRs** ("App Not Responding", when the app freezes for several seconds). Google uses these rates in ranking and may warn users about apps above its thresholds. App Store Connect and Xcode show crashes and hangs for iOS.

- **Track the crash-free rate per release.** Professional apps aim for well above 99 percent of users without a crash. A new release that drops the rate gets its staged rollout halted.
- **Add context, not personal data.** Attach the screen name and the user's ID, never their phone number or address.

A prompt that turns a crash into a fix:

```
Here is a crash report from Sentry for Platter 1.2.0 (stack trace
below, personal data removed). Find the cause in the code, write
  a test
that reproduces it, fix it, and tell me whether the fix can ship
  as an
EAS update or needs a store build. Then look for the same mistake
anywhere else in the apps.
```

The last sentence is where Claude earns its keep: the same mistake is rarely made only once.

Watch: Performance and Business Metrics

Beyond crashes, keep an eye on a few numbers each week:

- **Startup time and slow screens**, from your monitoring service's performance traces.
- **The main funnel** from Chapter 24: viewed, added, checked out, paid. A drop after a release is a bug until proven otherwise.
- **Retention**: how many new users come back after a day, a week and a month.
- **Server health**: error rates, response times, and the event stream for live tracking.

Listen: Store Reviews and Support

Reviews are public, affect ranking, and are full of bug reports.

- **Reply to reviews**, especially negative ones, briefly and helpfully: acknowledge, explain the fix or ask them to contact support. Both consoles let you reply.

- **Ask for reviews at a good moment** with the platforms' in-app review prompts (the `expo-store-review` library), for example after a successful delivery, never after a failure, and not often; the platforms limit how frequently the prompt appears.
- **Make support easy** from the account screen: help articles for common questions, and a way to contact a person about an order.
- **Tag and count.** Ask Claude to group a month of reviews and support messages by theme, with counts and examples. The biggest theme is your next priority.

```
Here are last month's 300 Play Store reviews for Minutely
  (exported CSV,
names removed). Group them into themes with a count and two short
example quotes each, separate bugs from feature requests, and
  suggest
the three changes that would most improve the rating. Don't invent
themes with fewer than five reviews.
```

Fix: Hotfixes and Updates

When something breaks in production:

1. **Assess.** How many users, how bad, is money involved? Halt the staged rollout if a new release caused it.
2. **Reproduce** with a failing test.
3. **Fix** in a small change, run the checks, and review it.
4. **Ship** the fastest safe way: an EAS update for JavaScript-only fixes (to the preview channel first, then production), or a new store build with an expedited review request if native code changed. Apple offers expedited review for critical fixes; use it sparingly.
5. **Write it down.** A short note on what happened, why, and what will prevent it next time.

A regular release rhythm, such as every two weeks, keeps updates small and reviews quick. Small releases are easier to test, easier to roll back and easier to explain.

Improve: Feature Flags and Experiments

Professional apps rarely release a big change to everyone at once.

- **Feature flags** turn features on and off from the server without a release. Ship the new checkout behind a flag, turn it on for your team, then 10 percent of users, then everyone. If it misbehaves, turn it off in seconds.
- **Experiments (A/B tests)** show two versions to random halves of users and compare a metric that matters, such as checkout completion. Decide the metric and the length of the test before you start, and don't stop early because one day looks good.
- **Remove old flags.** Each flag left in the code is a branch nobody tests. Ask Claude to remove a flag's dead side once the decision is made.

Services such as Firebase Remote Config, LaunchDarkly, Statsig and PostHog provide flags and experiments, or your own server can return a small set of flags with the user's profile.

Keep the Platform Current

Mobile platforms move quickly. Plan for:

- **One Expo SDK upgrade a year at least.** Each SDK supports a specific React Native version; the stores' target API and Xcode requirements move every year, and older SDKs eventually can't meet them. Expo's upgrade guide and `npx expo install --fix` do most of the work; your tests do the rest.
- **Dependency updates** monthly, in small batches, with the full checks.
- **New store requirements**: read the Play Console and App Store Connect announcements, and put deadlines in your calendar.

Claude is a good upgrade partner, provided you ask for evidence:

```
Upgrade this monorepo from Expo SDK 58 to the next SDK, following
  Expo's
upgrade guide for that version. Upgrade one package at a time,
  run the
checks after each, and keep a log of every breaking change you
  had to
handle and how. Then rebuild the web exports, retake the
```

```
  screenshots,
and compare them with the old ones; list every visual difference.
```

The Loop

Every week: look at crashes, the funnel and reviews; pick the most important problem; fix it with a test; ship it safely; and check the numbers moved. That loop, kept up for months, is what turns a well-built app into a professional one.

Key Takeaways

- Set up crash reporting with source maps before launch, watch Android vitals and Apple's crash reports, and halt rollouts that hurt the crash-free rate.
- Watch a few numbers weekly: startup, the main funnel, retention and server health.
- Reply to reviews, ask for ratings at happy moments, and use Claude to group feedback into themes.
- Fix production issues with a failing test first, and ship by the fastest safe route.
- Release big changes behind feature flags, test them with experiments, remove old flags, and keep the SDK and dependencies current.

PART VII

Bonus

CHAPTER 30 · BONUS

Reelhouse: Building for Android TV (and Apple TV)

Phones are held a few inches from the face and touched. Televisions sit three meters away and are driven by a remote with four arrows, a select button and Back. Everything about the interface changes: text sizes, layout, navigation, and above all **focus**, the highlight that shows what the select button will do. This bonus chapter builds Reelhouse, a streaming app for Android TV in the style of the big streaming services, with an original brand and fictional catalog, from the same Expo and React Native skills as the rest of the book.

The 10-Foot Interface

Designers call TV the "10-foot" interface, after the viewing distance. Its rules:

- **Big type and big targets.** Body text around 24 to 28 pixels on a 1920 by 1080 canvas, titles far larger. Reelhouse lays everything out on a 1920 by 1080 canvas and scales it to the screen.
- **Dark, cinematic surfaces.** Streaming apps are dark so that the artwork glows and the room stays dim. Reelhouse is dark only, with a violet and cyan brand.
- **One obvious focus.** At every moment exactly one thing is focused, and it must be unmistakable from across the room: Reelhouse uses a white ring, a violet glow and a slight scale-up.



Reelhouse's home screen: a hero billboard for the focused title, the side rail, and the Continue watching row.

- **Rows, not pages.** Content sits in horizontal rows that the remote moves through: right for the next card, down for the next row.
- **Safe margins.** Keep important content away from the screen's edges, which some TVs crop.
- **Back always works.** Every screen and overlay has a sensible answer to Back.

How Expo Builds for TV

Expo apps run on Android TV and Apple TV through **react-native-tvos**, a community-maintained fork of React Native with TV support: focus handling, remote events and TV-specific views. For Expo SDK 58 it's `react-native-tvos` 0.88.0-0, which must match the SDK's React Native version. You install it as the `react-native` dependency and add the `@react-native-tvos/config-tv` config plugin, which edits the native projects for TV when you prebuild with `EXPO_TV=1`. Not every Expo library supports TV; the Expo "Building for TV" guide lists which do.

Reelhouse lives in `tv/`, outside the workspace, because its `react-native` is the TV fork. It copies the theme tokens, the `Art` component and the illustrations from `@pro/ui` rather than importing them. Its configuration:

```
{
  "expo": {
    "name": "Reelhouse",
    "slug": "reelhouse-tv",
    "version": "1.0.0",
    "orientation": "landscape",
    "backgroundColor": "#0A0A10",
    "icon": "./assets/icon.png",
    "android": {
      "package": "app.reelhouse.example"
    },
    "ios": { "bundleIdentifier": "app.reelhouse.example" },
    "web": {
      "output": "single",
      "bundler": "metro",
      "name": "Reelhouse",
      "shortName": "Reelhouse",
      "themeColor": "#7C5CFF",
      "backgroundColor": "#0A0A10"
    },
    "plugins": [
      "expo-font",
      [
        "@react-native-tvos/config-tv",
        {
          "androidTVBanner": "./assets/tv_banner.png",
          "androidTVIcon": "./assets/icon.png"
        }
      ]
    ]
  }
}
```

The plugin's options point to the TV banner and icon. Prebuilding with `EXPO_TV=1` writes them into the Android project.

The Prompt

```
Build Reelhouse TV, the bonus streaming app in docs/PRD.md section 5, in
tv/: its own Expo SDK 58 project (NOT part of the npm workspace) that uses
`react-native-tvos` 0.88.0-0 as its `react-native` dependency and the
`@react-native-tvos/config-tv` plugin, following Expo's "Build Expo apps
for TV" guide. It is dark only, built for a 1920x1080 screen viewed from
across a room, and every action works with a D-pad remote: arrow keys,
select, back. Use the violet/cyan Reelhouse brand from docs/DESIGN.md.

Since tv/ is outside the workspace, copy what it needs from @pro/ui (theme
tokens, the Art component and the art files) into tv/src/ rather than
```

```
importing the workspace packages, and keep its data in a local typed
catalog module (fictional titles, seasons, episodes, channels, live
matches); no server is needed.

Screens: profile picker, Home (rotating hero billboard that follows the
focused title, rows including Continue watching with progress, a numbered
Top 10, Live sports with LIVE badges, tall Originals posters), side
navigation rail that expands on focus, Title details (seasons and
episodes, More like this), Search with an on-screen keyboard grid, Player
with auto-hiding controls and a Next episode card, and a Live TV guide
↪ grid.
Posters and backdrops are composed from the 3D art on rich gradients, with
title "logos" set in bold display type.

Focus is the heart of a TV app: a clear focus ring/glow and scale on every
focusable, sensible initial focus on every screen, rows that scroll to
↪ keep
the focused card visible, and Back that always goes somewhere sensible.

Verify it two ways:
1. Make it run on the web too (react-native-web), with arrow keys and
   Enter driving focus, so it can be checked in headless Chromium at
   1920x1080, device scale 1. Capture every screen, with real focus states
   visible, and fix anything that looks unpolished.
2. Run `EXPO_TV=1 npx expo prebuild --platform android --clean` and
↪ confirm
   the generated AndroidManifest declares the leanback launcher intent,
   android.software.leanback and touchscreen not required, and a 320x180
   banner with the app name (add the banner image). Do not build the APK;
   there is no Android SDK here. Remove the generated android/ folder
   ↪ after
   checking, and note what you found.

Add tests for the focus/navigation logic and the catalog helpers, plus
typecheck and lint for tv/. Save screenshots to tv/screenshots/
(01-profiles.png, ...). Keep it focused enough for one session of about an
hour and a half. Finish with a summary of the screens, how focus works,
what prebuild produced, and anything left undone.
```

Three choices in this prompt made the session work:

- **"Focus is the heart of a TV app"** and the paragraph after it turned a vague quality into a checklist: a focus ring and scale on everything focusable, initial focus on every screen, rows that scroll to keep focus visible, Back that always goes somewhere sensible.
- **"Make it run on the web too, with arrow keys and Enter driving focus"** gave Claude a way to check the app without a TV or an emulator: drive it with the keyboard in a headless browser and take screenshots of real focus states.
- **"Run prebuild and confirm the generated AndroidManifest"** checked the Android TV-specific configuration, which a web build can't show.

What the Session Built

Session 6 ran for 48 minutes over 124 turns and cost $10.89. It reported 76 tests passing and 25 screenshots, every one taken by driving the app with arrow keys, Enter and Escape only.

Profiles and Home



The profile picker.

The app opens on **Who's watching?**, with four profiles, including a Kids profile that sees only children's titles. Home is a full-screen **hero billboard** for a featured title (its artwork, "match" score, year, rating, seasons and genres, a synopsis, and Resume, More info and My list buttons), rotating through featured titles while the buttons have focus, and following the focused card once you move into the rows.

The **side rail** on the left shows only icons until it receives focus, then expands with labels: Search, Home, Movies, Series, Sports, Live TV, My list and Settings. Pressing left from anywhere in the rows lands on the current section.



The side rail expanded with its labels after receiving focus.

Rows

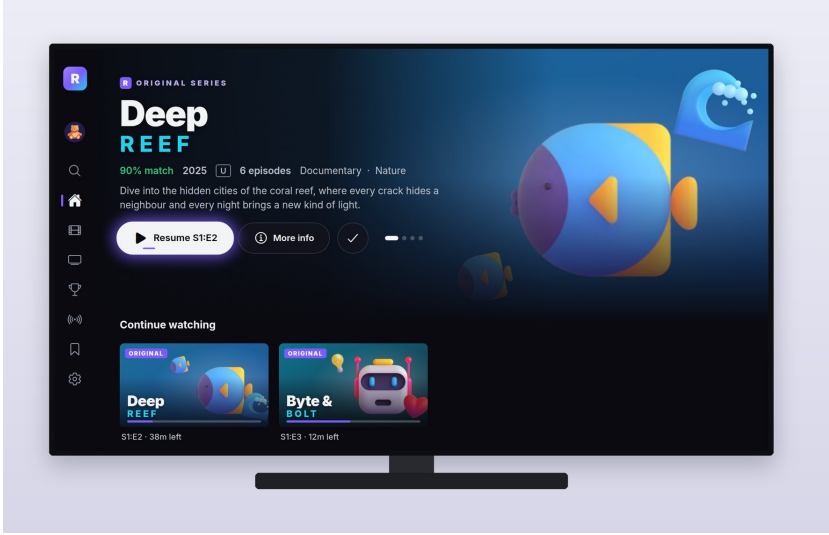


The numbered Top 10 row, with the focused poster raised and ringed.

Below the hero sit the rows: **Continue watching** with progress bars and "13m left", the **Top 10 in Northbay today** with large outlined numbers behind the posters, **Live sports** with red LIVE badges and score strips, New releases, Because you watched, tall **Originals** posters, and Genres.



Live sports with LIVE badges and score strips.



The Kids profile's home, with only children's titles.

Title Details

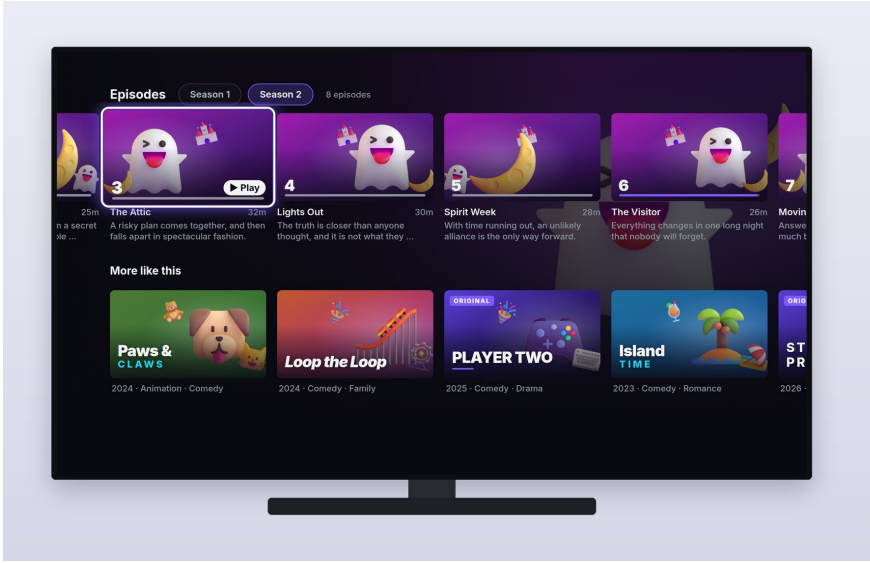


Title details with Resume, Trailer and My list, then seasons and episodes.

The details page shows the title "logo" in display type, the metadata and cast, **Resume S2:E6** with progress, Trailer and My list, then **season chips** and an **episode row** with numbers, titles, durations, synopses and progress. Further down, **More like this**. The backdrop dims as focus moves down the page, so text stays readable.

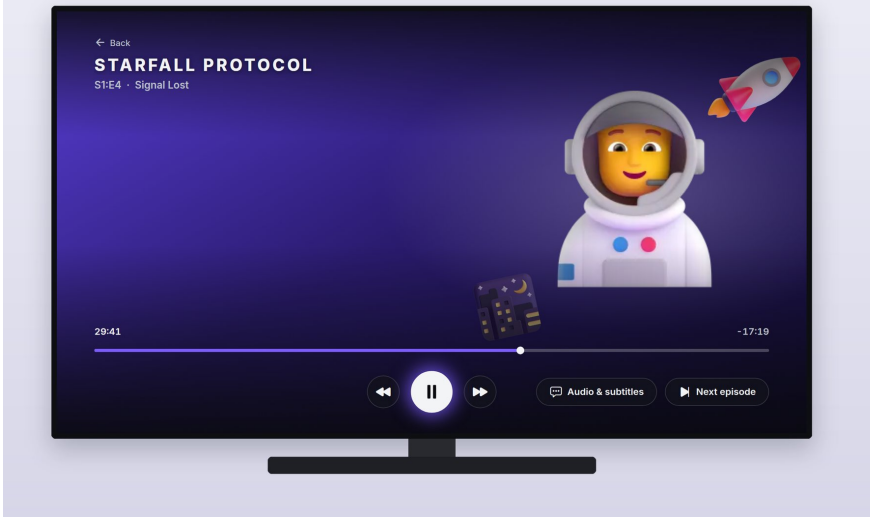
Search

Typing with a remote is slow, so the search screen makes every press count: a grid keyboard with letters, digits, space, delete and clear; results that update with each letter; and trending searches when the query is empty.

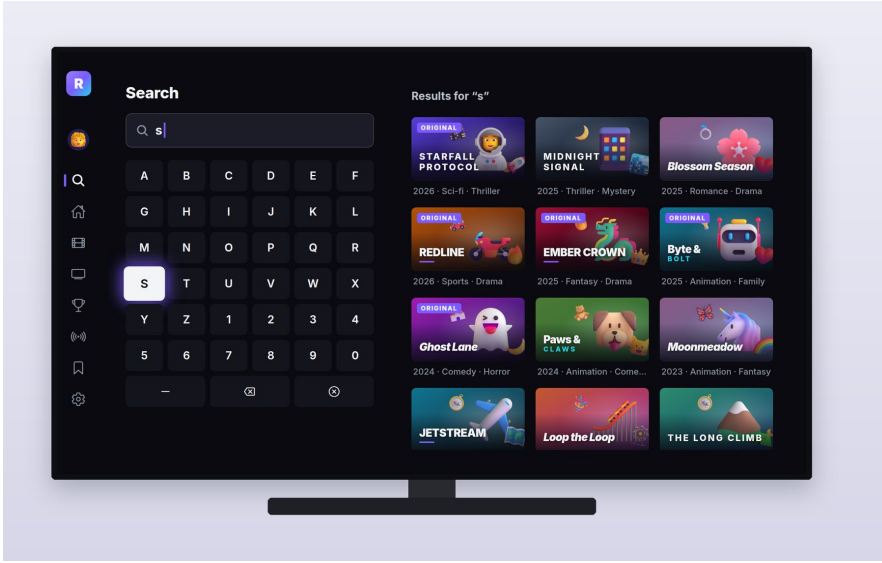


The episode row with the focused episode's Play badge, and More like this.

The Player



The player with its controls, and the Next episode card counting down.



Search with an on-screen keyboard and live results.

The player is mocked (no real video), but its behavior is the real thing: controls appear on any key press and fade out after five seconds of playback; the first key press only brings them back rather than also acting; left and right on the seek bar skip ten seconds; an audio and subtitles menu; and near the end a **Next episode** card takes focus and counts down. Its rules are pure, tested functions:

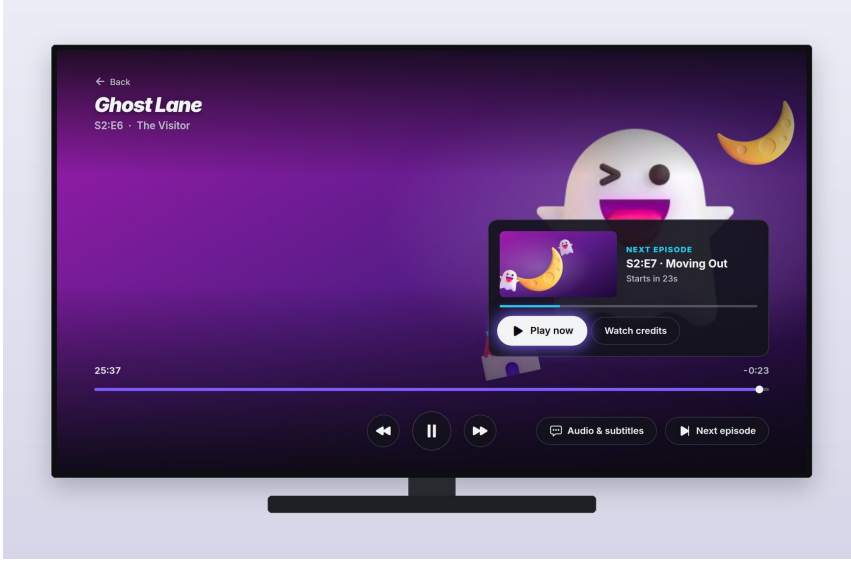
```
/** Pure player rules, kept apart from the screen so they are unit
↪ tested. */

/** The Next episode card appears this many seconds before the end. */
export const NEXT_CARD_SEC = 30;
/** Mock trailer length. */
export const TRAILER_SEC = 150;

/** Skip by `delta` seconds, clamped to the programme. */
export function seekBy(position: number, delta: number, duration:
↪ number): number {
  return Math.max(0, Math.min(duration, position + delta));
}

/** Controls hide only while playing, with no menu open, after a quiet
↪ spell. */
export function controlsShouldHide({
  playing,
  menuOpen,
  idleMs,
  hideAfterMs,
```

```
}: {
  playing: boolean;
  menuOpen: boolean;
  idleMs: number;
  hideAfterMs: number;
}): boolean {
  return playing && !menuOpen && idleMs >= hideAfterMs;
}
```



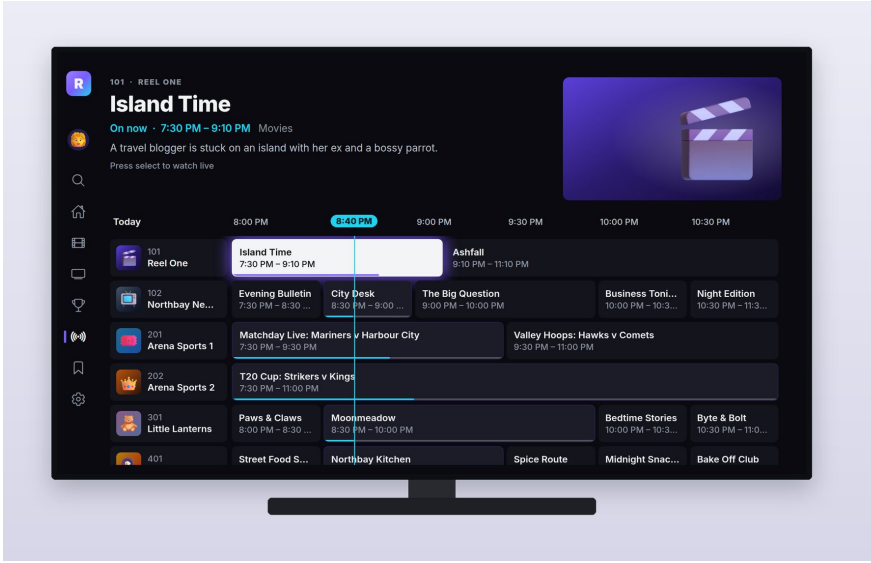
The Next episode card, which takes focus and counts down before the end.

Live TV

The **guide** shows a preview of the focused programme at the top, a time ruler with a cyan "now" line, and a grid of channels by time. Programmes on now open live; later ones set a reminder.

Focus, Step by Step

On Android TV, the native focus engine moves focus between focusable views in the direction pressed. Your job is to make every focusable look focused, to tell the engine where focus should go in the awkward places, and to scroll content so the focused item stays visible.



The live TV guide: a preview panel, a time ruler with a "now" line, and channels by time.

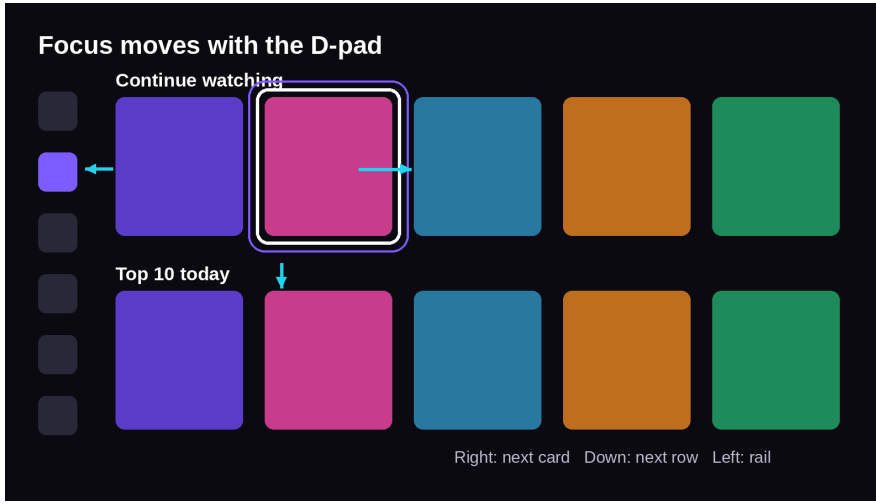
One Focusable Component

Everything the remote can land on is a `Focusable`. It tracks its focus state, animates its scale (instantly under reduce motion), draws the ring and glow, and takes focus when its screen asks it to:

```
  const [focused, setFocused] = useState(false);
  const host = useRef<HostInstance | null>(null);
  const [scale] = useState(() => new Animated.Value(1));
  const reduceMotion = useReduceMotion();
  const screen = useScreenFocus();
  const takeFocus = shouldTakeFocus(screen.active, screen.target,
  ↪ focusKey, autoFocus);

  useEffect(() => {
    if (!takeFocus) return;
    // Let the screen finish laying out (it may have just become visible).
    const id = setTimeout(() => requestFocus(host.current as unknown as
    ↪ FocusableHost), 0);
    return () => clearTimeout(id);
  }, [takeFocus, screen.target.nonce]);

  useEffect(() => {
    const to = focused ? focusScale : 1;
    if (reduceMotion) {
      scale.setValue(to);
      return;
    }
    const anim = Animated.timing(scale, {
```



How focus moves: right to the next card, down to the next row (landing where you left it), left to the rail.

```
      toValue: to,
      duration: motion.focusMs,
      useNativeDriver: Platform.OS !== 'web',
    });
    anim.start();
    return () => anim.stop();
  }, [focused, focusScale, reduceMotion, scale]);
```

The `autoFocus` and `focusKey` props are how each screen sets **initial focus** and **restores focus**. Screens under the top one stay alive, so when you press Back from a title's details, focus returns to exactly the card you left.

Focus Groups

Rows and panels are **focus groups**. On Android TV they're `TVFocusGuideViews` from react-native-tvos, which remember the last focused child (so moving down into a row lands where you were), can name a destination for first entry (the rail's current section), and can trap directions:

```
export function FocusGroup({ style, children, entry, trap = [] }:
↪ FocusGroupProps) {
  return (
    <TVFocusGuideView
      autoFocus
      style={style}
      destinations={entry ? [entry] : undefined}
      trapFocusUp={trap.includes('up')}
```

```
      trapFocusDown={trap.includes('down')}
      trapFocusLeft={trap.includes('left')}
      trapFocusRight={trap.includes('right')}
    >
      {children}
    </TVFocusGuideView>
  );
}

/** Keeps focus inside (a menu or dialog) until it closes. */
export function FocusTrap({ style, children }: { style?:
↪ StyleProp<ViewStyle>; children: ReactNode }) {
  return (
    <TVFocusGuideView autoFocus trapFocusUp trapFocusDown trapFocusLeft
    ↪ trapFocusRight style={style}>
      {children}
    </TVFocusGuideView>
  );
}
```

Trapping is how the player's seek bar keeps left and right for skipping, and how a menu keeps focus until it closes.

Remote Keys

Remote buttons arrive as TV events on Android TV and as keyboard events on the web. One tiny module maps both to the same names, so screen logic handles "select" or "playPause" without caring where it came from:

```
export type RemoteKey = 'up' | 'down' | 'left' | 'right' | 'select' |
↪ 'playPause' | 'rewind' | 'fastForward';

const TV_EVENTS: Record<string, RemoteKey> = {
  up: 'up',
  down: 'down',
  left: 'left',
  right: 'right',
  select: 'select',
  playPause: 'playPause',
  rewind: 'rewind',
  fastForward: 'fastForward',
};

const KEYBOARD: Record<string, RemoteKey> = {
  ArrowUp: 'up',
  ArrowDown: 'down',
  ArrowLeft: 'left',
  ArrowRight: 'right',
  Enter: 'select',
  ' ': 'playPause',
  MediaPlayPause: 'playPause',
  MediaRewind: 'rewind',
  MediaFastForward: 'fastForward',
};
```

```
/** react-native-tvos eventType to a remote key (null for focus/blur and
↪ other events). */
export function remoteKeyFor(eventType: string): RemoteKey | null {
  return TV_EVENTS[eventType] ?? null;
}

/** KeyboardEvent.key to a remote key. */
export function remoteKeyForKeyboard(key: string): RemoteKey | null {
  return KEYBOARD[key] ?? null;
}
```

Scrolling to Keep Focus Visible

TV rows move under a fixed **focus line**: the focused card stays near the left edge while the row slides, until the row runs out of content and the last cards scroll in. That's a calculation, and it's tested:

```

/** Horizontal scroll offset that shows card `index` at the row's focus
↪ line. */
export function rowOffsetForIndex(metrics: RowMetrics, index: number):
↪ number {
  const { itemWidth, gap, count, viewportWidth } = metrics;
  if (count <= 0) return 0;
  const i = Math.max(0, Math.min(count - 1, index));
  const maxOffset = Math.max(0, rowContentWidth(metrics) - viewportWidth);
  const offset = i * (itemWidth + gap);
  return Math.max(0, Math.min(maxOffset, offset));
}
```

On the web, a small spatial-navigation layer reproduces the same rules for arrow keys, which is what made the screenshots possible.

The Review That Found Two Bugs

Looking through the 25 screenshots found two problems the session hadn't noticed. In the Top 10 screenshot, a card scrolled underneath the collapsed side rail and showed through it. In the guide, the preview panel read "Island Time on Reel One.", a placeholder rather than a synopsis. A short follow-up session fixed both:

```
Two visual bugs in Reelhouse TV (tv/), found by reviewing the screenshots:

1. tv/screenshots/04-home-top10.png: when a row scrolls, cards slide
   underneath the collapsed side navigation rail and show through it: the
   "Ember Crown" card is visible behind the rail icons at the far left.
   The rail must sit above the content on an opaque background (with a
```

```
   soft fade into the content is fine), and rows should clip at the rail's
   edge. Check every screen with rows (Home, Movies, Series, Sports, Kids,
   My list, details' More like this) the same way.
2. tv/screenshots/19-live-tv-guide.png: the guide's preview panel shows
   placeholder text, "Island Time on Reel One." Give every programme in
   ↪ the
   catalog a real one-line synopsis and show it there.

Fix both, re-capture every affected screenshot, run tv's typecheck, lint
and tests, and summarize what changed.
```

Session 6b ran for under eight minutes and cost $1.83. It made the rail opaque with a short fade, clipped rows at the rail's edge with a tested helper, wrote a one-line synopsis for every programme, made a missing synopsis a type error, and added a test that rejects placeholder text and overlong lines. It also caught its own regression: its first, wider fade dimmed the first letter of "Island Time", which it saw in the re-captured screenshot and fixed. The TV app finished with 80 tests passing.

The screenshot session itself had found three more focus bugs while capturing: the rail's group memory overrode its current section, right on the seek bar jumped to Pause, and My list had no initial focus.

Android TV Configuration

TV apps on Google Play must declare themselves correctly. The session ran `EXPO_TV=1 npx expo prebuild --platform android --clean` and checked the generated manifest:

- the `LEANBACK_LAUNCHER` category on the main activity, so the app appears on the TV home screen;
- `android.software.leanback` declared with `required="false"`, so the same app can also install on phones (set the plugin's `androidTVRequired` option for a TV-only app);
- **touchscreen not required**, since TVs have no touchscreen;
- an `android:banner` pointing at a 320 by 180 banner image that includes the app name, which the TV launcher shows.

Then it removed the generated `android/` folder; the configuration lives in `app.json`, and EAS generates the native project at build time.

Google Play Requirements for TV

From Google's documentation in October 2026:

- **Target API level**: Android TV apps must target at least Android 14 (API level 34).
- **64-bit and 16 KB page sizes**: from 1 August 2026, TV apps must support both 32- and 64-bit architectures and 16 KB memory page sizes.
- **Launcher assets**: a 320 by 180 banner containing the app name, and a 160 by 160 icon at xhdpi.
- **TV app quality guidelines**: full D-pad navigation, no dependence on touch, a working Back button, content that fits the screen, and more. Google reviews TV apps against these before they appear on TV devices. You opt into the Android TV form factor in the Play Console and add TV screenshots.

Testing on an Emulator or a TV

The build machine for this book had no emulator support, so Reelhouse was checked on the web and through prebuild only; its native focus wiring has been type-checked but never run on a device. On your computer, create an Android TV emulator in Android Studio (an Android TV system image, API 31 or later), then:

```
cd tv
EXPO_TV=1 npx expo prebuild --platform android --clean
npx expo run:android
```

Or install a development build on a real Android TV or streaming stick with developer options enabled. Test every screen with the remote only: no mouse, no touch.

Apple TV

The same code can target Apple TV: react-native-tvos supports tvOS, and the config plugin prepares the iOS project when you prebuild for iOS with `EXPO_TV=1`. You'd need Apple TV app icons and top-shelf images, and you'd test on the tvOS simulator in Xcode. Session 6 didn't configure tvOS, and on

Apple TV some libraries are unsupported; the focus engine also behaves a little differently, so plan a separate round of focus testing.

Making It Real

Reelhouse is a convincing shell; a real streaming app adds:

- **Real video** with a TV-capable player library (`expo-video` or `react-native-video`), adaptive streaming (HLS or DASH), and **DRM** for licensed content.
- **Accounts and profiles on a server**, with sign-in that suits a TV: a code shown on screen that the user enters on their phone.
- **Subscriptions** through Google Play Billing and Apple's In-App Purchase (Chapter 23), with access checked on the server.
- **Continue watching sync** across devices, written to the server every few seconds during playback.
- **Accessibility**: captions everywhere, audio description tracks, and screen reader support through TalkBack on Android TV.

```
Add real video playback to Reelhouse's player with expo-video, if
  it
supports Android TV in our SDK (check the docs and tell me),
  playing a
public HLS test stream. Keep the existing controls, auto-hide,
  seek and
Next episode behavior and their tests; wire play, pause and seek
  to the
real player; and report what you could and couldn't verify
  without a TV.
```

Key Takeaways

- TV is a 10-foot, remote-driven interface: big type, dark surfaces, rows, safe margins, and one unmistakable focus.
- Expo builds for Android TV with react-native-tvos and the config-tv plugin when prebuilding with `EXPO_TV=1`.

- Build every focusable from one component, set and restore focus per screen, use focus groups for rows, and keep the focused card visible with tested scroll math.
- Verify a TV app with keyboard-driven screenshots on the web, prebuild checks of the manifest, and finally a real TV or emulator.
- Android TV on Play needs target API 34, 64-bit and 16 KB page support, the leanback declarations, a 320 by 180 banner and full D-pad navigation.

Appendix A: The Prompt Library

Every prompt in this book, in one place. The session prompts are the exact text given to Claude Code to build the apps; the production prompts are starting points for your own app. Adapt the names, paths and details to your project, and keep the habits they share: say what to build, say how to check it, and ask for an honest summary of what's left.

The Session Prompts

These eight prompts built everything in this book, in order. Each ran in a fresh session with the repository's `CLAUDE.md` loaded.

Session 1: The Foundation

```
Read CLAUDE.md, docs/PRD.md and docs/DESIGN.md. This session sets up the
foundation that all four apps will share. Build it in this order, and
↪ check
each step before moving on:

1. The npm workspace: a root package.json with workspaces apps/*,
↪ packages/*
   and server; shared tsconfig (strict); ESLint (flat config, expo +
   typescript rules) and Jest configured so `npm run typecheck`, `npm run
   lint` and `npm test` run across every package. Use the exact versions
   Expo SDK 58 expects (expo ~58.0.22, react-native 0.88.1, react 19.3.0;
   check with `npx expo install --check` once an app exists).

2. packages/ui, the design system in docs/DESIGN.md: tokens (colors per
   brand with light and dark, type scale with Inter, spacing, radii,
   shadows), a ThemeProvider that takes a brand name, and every component
   listed in DESIGN.md, built with React Native primitives and
   ↪ Reanimated 4
```

```
   so they render on Android, iOS and web. An `Art` component and ArtTile
   that load the illustrations in packages/ui/art by name (typed from
   art/index.json). Component tests with @testing-library/react-native for
   the stateful ones (QuantityStepper, Price, CountdownTimer, Button).

3. server/: the Node + TypeScript API from the PRD, Hono on port 4000,
   better-sqlite3, a seed script with rich, believable fictional data for
   all four apps (at least 24 restaurants with full menus, 60 products
   ↪ with
   reviews, 20 users with posts, stories and comments, 120 grocery items
   ↪ in
   12 categories, coupons), OTP login (code 123456) returning a JWT,
   ↪ carts,
   orders, and order-status simulation pushed over Server-Sent Events.
   Tests for the routes with Vitest or Jest and supertest-style requests.

4. packages/api: typed fetch client for every route, React Query hooks,
   an SSE helper that works in React Native and on web, shared types.
   Base URL from EXPO_PUBLIC_API_URL, defaulting to http://localhost:4000.

Do not create the four apps yet; the next sessions do that. Finish by
running typecheck, lint and tests, fixing everything, and then write a
short summary: what you built, the commands, and anything you decided that
differs from the docs.
```

Session 2: Platter

```
Build Platter, the food delivery app in docs/PRD.md section 1, as
apps/platter: an Expo SDK 58 app with Expo Router, using @pro/ui (brand
"platter") and @pro/api. It should feel as polished as the best food
delivery apps: dense but calm layouts, real loading skeletons, empty and
error states, smooth press feedback, a sticky cart bar, bottom sheets.

Screens: OTP login, Home, Search, Restaurant (menu, customization sheet),
Cart and checkout (coupons, tip, bill details, address and payment
↪ sheets),
Order tracking (live SSE status, SVG map with moving rider, ETA, rider
card), Orders, Rate order, Profile. Tabs: Home, Search, Orders, Profile.

Work like a professional team:
1. Plan first: list the routes, components and data each screen needs, and
   anything missing from @pro/ui or @pro/api. Add missing pieces to the
   shared packages (with tests) rather than inside the app.
2. Build screen by screen. After each one, export the app for web and
   look at it: run the server, serve the web build, and capture the screen
   in headless Chromium at 393x852 CSS pixels with device scale 3
   ↪ (Playwright
   is installed globally; require('playwright') works). Compare it with
   ↪ the
   PRD and fix anything that looks unpolished: alignment, spacing, text
   cut off, images missing, overlapping elements.
3. Add tests for the app's logic (bill calculation display, cart behavior,
   tracking state) and component tests for the main screens.
4. Make sure `npx expo export -p web` works for the app, and add an
   `npm run export:web -w apps/platter` script.
5. Save your final screenshots of every screen to
```

```
↪ apps/platter/screenshots/
   (PNG, named 01-login.png, 02-home.png, ...), with the tracking screen
   captured mid-delivery.

Finish with typecheck, lint and tests passing for the whole workspace,
↪ then
a summary of the screens, the components you added to the shared packages,
and any problems you hit.
```

Session 3: Cartly

```
Build Cartly, the e-commerce store in docs/PRD.md section 2, as
apps/cartly: an Expo SDK 58 app with Expo Router, using @pro/ui (brand
"cartly") and @pro/api. Look at apps/platter first and follow the same
structure and conventions (tests in src/test, export:web script,
screenshots folder), and reuse the shared components instead of copying.

It should feel like the best shopping apps: information-dense product
cards with ratings, MRP strike-through and discount badges, deals with
↪ live
countdowns, fast search with suggestions, filter and sort sheets, a
product page with an image gallery and a sticky buy bar, and a checkout
that builds trust (delivery date, offers, secure payment note).

Screens: Home, Search with suggestions, Product listing with
filters/sort, Product detail (gallery, variants, pincode delivery
check, offers, highlights, reviews), Cart (save for later), Checkout
(address, payment sheet), Order confirmation, Orders and Order details
↪ with
shipment tracking steps, Return request flow, Wishlist, Notifications,
Account. Tabs: Home, Categories, Wishlist, Cart (with count badge),
↪ Account.

As before: plan first and add missing shared pieces to @pro/ui or @pro/api
with tests; build screen by screen and check each one in headless Chromium
at 393x852 CSS pixels, device scale 3, against the PRD, fixing anything
unpolished; add logic and screen tests; save final screenshots of every
screen to apps/cartly/screenshots/ (01-home.png, ...). Finish with
typecheck, lint and tests passing for the whole workspace, and a summary.
```

Session 3b: Finishing Cartly

```
The previous session built Cartly (apps/cartly) from docs/PRD.md section 2
and was cut off by a time limit before it finished. It had also started
moving shared pieces out of apps/platter into @pro/ui and @pro/api (see
`git status` and `git diff`); that refactor may be half done.

Finish the job:
1. Review the uncommitted changes and complete the refactor so Platter and
   Cartly both use the shared pieces, with nothing duplicated and nothing
   left broken.
2. Run typecheck, lint and tests for the whole workspace and fix every
   problem.
3. Re-export both apps for web and recapture any screenshot in
```

```
   apps/platter/screenshots or apps/cartly/screenshots whose screen
   ↪ changed.
4. Write a summary of what Cartly contains, what moved into the shared
   packages, and anything still missing from the PRD.
```

Session 4: Lumo

```
Build Lumo, the social photo app in docs/PRD.md section 3, as apps/lumo:
an Expo SDK 58 app with Expo Router, using @pro/ui (brand "lumo") and
@pro/api. Follow the structure of apps/platter and apps/cartly and reuse
↪ the
shared packages (session, sheets, components) instead of copying.

It should feel like the best photo-sharing apps: content first, quiet
chrome, crisp icons, smooth gestures. The "photos" are rich illustrated
compositions: layered gradients plus the 3D art from @pro/ui (a scene
component in @pro/ui that composes a backdrop, a hero illustration and
small accents from a post's seed, so every post looks distinct).

Screens: Feed (stories row with unseen/seen rings, posts with double-tap
like and heart burst, like/comment/share/save, captions, comment count),
Story viewer (progress bars, tap left/right, hold to pause), Comments
sheet, Create post (pick art, filter strip, caption, share), Explore grid
with mixed tile sizes, Profile (stats, bio, highlights, grid and tagged
tabs), Notifications, Messages list and a chat thread. Tabs: Home,
Explore, Create, Activity, Profile.

As before: plan first and add missing shared pieces to @pro/ui or @pro/api
with tests; build screen by screen and check each one in headless Chromium
at 393x852 CSS pixels, device scale 3, fixing anything unpolished; add
logic and screen tests; save final screenshots of every screen to
apps/lumo/screenshots/ (01-feed.png, ...), with the story viewer captured
mid-story and a heart burst captured on a post if you can. Keep the work
focused so it fits in one session of about an hour. Finish with typecheck,
lint and tests passing for the whole workspace, and a summary.
```

Session 5: A Flaky Test, and Minutely

```
Two jobs in this session.

1. A flaky test. `npm run check` failed once in
apps/cartly/src/test/screens/orders-returns.test.tsx ("Order details ›
tracks the shipment live and lets you cancel before it ships") while the
whole workspace's tests ran in parallel; run alone it passes. Find the
↪ real
cause (timing, fake timers, shared state, SSE) and fix it properly,
↪ without
skipping the test or just raising timeouts. Prove it with several runs of
the full `npm test` from the root.

2. Build Minutely, the 10-minute grocery app in docs/PRD.md section 4, as
apps/minutely: an Expo SDK 58 app with Expo Router, using @pro/ui (brand
"minutely") and @pro/api, following the structure of the other apps and
reusing the shared packages.
```

```
It should feel like the best quick-commerce apps: speed everywhere (a bold
"Delivery in 9 minutes" header, instant ADD steppers, a cart bar that's
always one tap away), bright category tiles, savings called out in green,
and a tracking screen with a countdown ring and the SVG map.

Screens: Login, Home (address, search, category grid, offer banners,
product rails with ADD steppers), Search, Category page (left category
rail, product grid, filters), Product sheet, Cart (savings banner,
delivery partner tip, bill), Checkout and payment, Tracking (countdown
↪ ring,
SVG map, status steps, partner card), Orders, Account. Tabs: Home,
Categories, Orders, Account.

As before: plan first, add missing shared pieces with tests, check each
screen in headless Chromium at 393x852 CSS pixels with device scale 3 and
fix anything unpolished, add logic and screen tests, save final
↪ screenshots
of every screen to apps/minutely/screenshots/ (01-home.png, ...), with
tracking captured mid-delivery. Keep it focused enough for one session of
about an hour. Finish with typecheck, lint and tests passing for the whole
workspace, and a summary.
```

Session 6: Reelhouse TV

```
Build Reelhouse TV, the bonus streaming app in docs/PRD.md section 5, in
tv/: its own Expo SDK 58 project (NOT part of the npm workspace) that uses
`react-native-tvos` 0.88.0-0 as its `react-native` dependency and the
`@react-native-tvos/config-tv` plugin, following Expo's "Build Expo apps
for TV" guide. It is dark only, built for a 1920x1080 screen viewed from
across a room, and every action works with a D-pad remote: arrow keys,
select, back. Use the violet/cyan Reelhouse brand from docs/DESIGN.md.

Since tv/ is outside the workspace, copy what it needs from @pro/ui (theme
tokens, the Art component and the art files) into tv/src/ rather than
importing the workspace packages, and keep its data in a local typed
catalog module (fictional titles, seasons, episodes, channels, live
matches); no server is needed.

Screens: profile picker, Home (rotating hero billboard that follows the
focused title, rows including Continue watching with progress, a numbered
Top 10, Live sports with LIVE badges, tall Originals posters), side
navigation rail that expands on focus, Title details (seasons and
episodes, More like this), Search with an on-screen keyboard grid, Player
with auto-hiding controls and a Next episode card, and a Live TV guide
↪ grid.
Posters and backdrops are composed from the 3D art on rich gradients, with
title "logos" set in bold display type.

Focus is the heart of a TV app: a clear focus ring/glow and scale on every
focusable, sensible initial focus on every screen, rows that scroll to
↪ keep
the focused card visible, and Back that always goes somewhere sensible.

Verify it two ways:
1. Make it run on the web too (react-native-web), with arrow keys and
```

```
   Enter driving focus, so it can be checked in headless Chromium at
   1920x1080, device scale 1. Capture every screen, with real focus states
   visible, and fix anything that looks unpolished.
2. Run `EXPO_TV=1 npx expo prebuild --platform android --clean` and
↪ confirm
   the generated AndroidManifest declares the leanback launcher intent,
   android.software.leanback and touchscreen not required, and a 320x180
   banner with the app name (add the banner image). Do not build the APK;
   there is no Android SDK here. Remove the generated android/ folder
   ↪ after
   checking, and note what you found.

Add tests for the focus/navigation logic and the catalog helpers, plus
typecheck and lint for tv/. Save screenshots to tv/screenshots/
(01-profiles.png, ...). Keep it focused enough for one session of about an
hour and a half. Finish with a summary of the screens, how focus works,
what prebuild produced, and anything left undone.
```

Session 6b: Reelhouse Fixes

```
Two visual bugs in Reelhouse TV (tv/), found by reviewing the screenshots:

1. tv/screenshots/04-home-top10.png: when a row scrolls, cards slide
   underneath the collapsed side navigation rail and show through it: the
   "Ember Crown" card is visible behind the rail icons at the far left.
   The rail must sit above the content on an opaque background (with a
   soft fade into the content is fine), and rows should clip at the rail's
   edge. Check every screen with rows (Home, Movies, Series, Sports, Kids,
   My list, details' More like this) the same way.
2. tv/screenshots/19-live-tv-guide.png: the guide's preview panel shows
   placeholder text, "Island Time on Reel One." Give every programme in
   ↪ the
   catalog a real one-line synopsis and show it there.

Fix both, re-capture every affected screenshot, run tv's typecheck, lint
and tests, and summarize what changed.
```

Production Prompts

Chapter 20: Performance: Lists, Images, Startup and Smooth Animation

```
Lumo's feed and Cartly's product list will hold thousands of
  items in
production. Move both to FlashList (the version bundled with our
  Expo
SDK), keep every existing test passing, make sure post and
  product rows
are memoized with stable props, and tell me anything that behaves
differently, such as scroll restoration or the sticky header.
```

```
Review this app for performance problems a user would feel on a
mid-range Android phone. Look for: lists that re-render whole rows,
lists nested in scroll views, images loaded at full size, work
  done at
startup that could wait, animations driven from JavaScript, and
  anything
that blocks a press handler. Rank what you find by how much a
  user would
notice it, explain each in one sentence, and don't change any
  code yet.
```

Chapter 21: Accessibility and Localization

```
Audit apps/platter for accessibility. For every screen, check: each
touchable has a role and a clear label; related text is grouped
  into one
announcement; status changes are announced; decorative art is
  hidden;
targets are at least 44x44; layout survives the largest text
  size; and
nothing is shown by color alone. Write the findings as a table
  (screen,
problem, fix), fix them, add or update a test for each fix using
  the
accessible names, and run the checks.
```

```
Prepare Minutely for Hindi. Move every user-visible string into
en.json and hi.json files using i18next and expo-localization, keep
prices and dates formatted with Intl for the current locale, use
  plural
rules for counts, and leave the Hindi values as English
  placeholders
marked TODO for a translator. Add a test that renders Home in each
language. Don't translate anything yourself.
```

Chapter 22: Testing: Unit, Component, API and End to End

```
Add Maestro flows for Platter's three critical journeys (sign in
  and
order, pay and track, rate a delivered order) in flows/platter/,
  using
the accessibility labels the screen tests already use. Then list
  any
screen where a label a flow needs is missing, and add it with a
  test.
```

Chapter 23: Payments for Real: Razorpay, Stripe, Apple Pay, Google Pay and UPI

```
Replace Platter's demo payment with Razorpay in test mode. The
  server
creates a Razorpay order for the server-calculated total, stores
  its ID
on our order, and verifies both the checkout signature and the
payment.captured webhook (HMAC-SHA256, timing-safe, raw body),
  checking
amount and currency, and processing each event once. The app opens
Razorpay checkout with react-native-razorpay in a development
  build,
then waits for the order's event stream to say "paid". Test the
  server
logic offline with known good, tampered and replayed messages.
  Never use
live keys. Finish by listing what can only be verified with real
  test
keys on a device, and anything in Razorpay's rules you're unsure
  about.
```

Chapter 24: Push Notifications, Deep Links and Analytics

```
Add push notifications to Platter for order updates only: ask for
permission after the first order is placed, with a one-line
  explanation;
store the Expo push token on the server against the user; send a
  push on
each order status change from the simulator; and open the order
  screen
when a notification is tapped, after sign-in if needed. Add an
"Order updates" switch in Profile. Then add an analytics module
  with a
track() function that logs to the console in development, and
  call it
for the seven events in docs/EVENTS.md, with no personal data in
  any
event. Test the server side and the routing logic.
```

Chapter 25: Security and Privacy

```
Store the session token with expo-secure-store instead of
  AsyncStorage
in all four apps, through the existing TokenStorage interface,
  keeping
the web build on localStorage. Add a test for the adapter, and
  make sure
a token stored by the old version is moved over once and then
  deleted.
```

```
Do a security and privacy review of this repository before release.
Check for: secrets in the app or in git history; tokens in
  unencrypted
storage; anything the client decides that the server should;
  endpoints
without authentication, authorization checks or validation; missing
rate limits on login and OTP; CORS and HTTP settings; permissions
  asked
without need; personal data in logs or analytics; and account
  deletion.
Rank findings by severity with file and line, and explain how
  each could
be exploited in one sentence. Don't change code yet.
```

Chapter 26: Builds and Releases With EAS

```
Set up EAS for apps/platter: eas.json with development, preview and
production profiles, remote app versions with auto-increment,
  channels
for updates, and the fingerprint runtime version policy. Replace
  the
.example bundle IDs with com.mycompany.platter. Add an EAS
  workflow that
runs npm run check on pull requests and builds both platforms on
  tags
starting with v. Explain each setting in a short comment in the PR.
Don't run any build.
```

Chapter 27: Shipping to Google Play

```
Draft the Google Play listing for Platter: a 30-character name, an
80-character short description, and a full description under 4,000
characters with the main features as short paragraphs. Use plain,
  honest
language; mention no other company or brand; claim nothing the app
doesn't do. Then draft the Data safety answers from the code
```

```
  (what we
collect, why, shared or not, encrypted in transit, deletion),
  listing
every SDK we use and what it collects, and flag anything you're
  unsure
about for me to check.
```

Chapter 28: Shipping to the App Store

```
Prepare Lumo for App Review guideline 1.2 (user-generated content).
Add Report to every post, comment, story and message (with
  reasons),
Block and Unblock on profiles (hiding blocked users everywhere), a
server route to record reports with a simple admin list, and terms
acceptance at sign-up. Add server and screen tests, and list
  anything
the guideline asks for that this doesn't yet cover.
```

Chapter 29: After Launch: Crashes, Reviews, Updates and Experiments

```
Here is a crash report from Sentry for Platter 1.2.0 (stack trace
below, personal data removed). Find the cause in the code, write
  a test
that reproduces it, fix it, and tell me whether the fix can ship
  as an
EAS update or needs a store build. Then look for the same mistake
anywhere else in the apps.
```

```
Here are last month's 300 Play Store reviews for Minutely
  (exported CSV,
names removed). Group them into themes with a count and two short
example quotes each, separate bugs from feature requests, and
  suggest
the three changes that would most improve the rating. Don't invent
themes with fewer than five reviews.
```

```
Upgrade this monorepo from Expo SDK 58 to the next SDK, following
  Expo's
upgrade guide for that version. Upgrade one package at a time,
  run the
checks after each, and keep a log of every breaking change you
  had to
handle and how. Then rebuild the web exports, retake the
  screenshots,
```

```
and compare them with the old ones; list every visual difference.
```

Chapter 30: Reelhouse: Building for Android TV (and Apple TV)

```
Add real video playback to Reelhouse's player with expo-video, if
  it
supports Android TV in our SDK (check the docs and tell me),
  playing a
public HLS test stream. Keep the existing controls, auto-hide,
  seek and
Next episode behavior and their tests; wire play, pause and seek
  to the
real player; and report what you could and couldn't verify
  without a TV.
```

Patterns Worth Reusing

- **Plan first.** "Write a plan to PLAN.md before any code" catches misunderstandings while they're cheap.
- **Name the patterns, not the adjectives.** "Instant ADD steppers and a cart bar one tap away" gives Claude something to check; "modern and fast" doesn't.
- **Ask for visual proof.** "Check each screen in headless Chromium at 393x852 and fix anything unpolished" made every session review its own screenshots.
- **Make checks the definition of done.** "Finish with typecheck, lint and tests passing for the whole workspace."
- **Size the session.** "Keep it focused enough for one session of about an hour."
- **Ask for the gaps.** "Finish with a summary of what's left undone" turns a confident report into a useful one.
- **Review before change.** For audits, "rank the findings and don't change code yet" keeps you in charge of priorities.

Appendix B: Release Checklists

Use these before every release. Copy them into your repository as `RELEASE.md`, tick them off in a pull request, and add the lessons from each release.

Every Release, Both Platforms

Code and checks

- `npm run check` passes: typecheck, lint, every test.
- Maestro flows for every critical journey pass on an Android emulator and an iOS simulator.
- The version number is increased and the changelog updated.
- No secrets in the app, the repository or its history; only public `EXPO_PUBLIC_` values.
- Analytics events checked: no personal data.

On real devices (release build)

- A cheap Android phone and a recent iPhone: sign in, the main journey, payment in test mode, tracking, sign out.
- Startup under about two seconds; scrolling smooth on the longest list.
- Slow network and no network: every screen shows loading, error and retry states.

- Interruptions: a call during payment, switching apps while ordering, locking during tracking.
- Push notifications arrive, and tapping one opens the right screen.
- Deep links and universal links open the right screen, signed in and signed out.

Accessibility and settings

- TalkBack and VoiceOver: the main journey can be completed without looking.
- Largest system text size: nothing cut off, nothing overlapping.
- Dark mode and light mode, with sufficient contrast.
- Reduce motion: no movement-heavy animations.
- Each supported language, including long translations.

Store and privacy

- Privacy policy matches what the app and its SDKs collect.
- Data safety form and App Privacy details are up to date.
- Account deletion works in the app (and from the web for Google Play).
- The reviewer's demo account works today.
- Screenshots and descriptions match this version.

Rollout

- Crash reporting receives events from this build, with source maps.
- Staged rollout (Play) or phased release (App Store) chosen.
- Someone watches crashes and reviews for the first 48 hours.

Google Play

- Developer account verified; for a new personal account, the 12-tester, 14-day closed test completed and production access granted.
- Target API level 36 (from 31 August 2026).
- Signed Android App Bundle (`.aab`), with Google Play App Signing.

- Store listing: name (30 characters), short description (80), full description (4,000), 512 x 512 icon, 1024 x 500 feature graphic, at least two phone screenshots.
- App content: privacy policy, app access instructions, ads, content rating, target audience, Data safety, account deletion URL, other declarations.
- Release notes in plain language.
- Staged rollout percentage set; ready to halt.

App Store

- Apple Developer Program membership active ($99 a year).
- Built with Xcode 26 and the iOS 26 SDK (EAS on a current Expo SDK).
- Minimum iOS version 13 or later.
- Age rating questionnaire answered (updated system).
- Privacy manifest entries for the app and SDKs; App Privacy details complete.
- Product page: name, subtitle, description, keywords, screenshots for the required sizes, support and privacy URLs.
- Review information: demo account, one-time code, notes, contact.
- Export compliance answered.
- Sign in with Apple offered if third-party sign-in is offered and no exception applies.
- For apps with user-generated content: report, block, moderation and terms.
- Release option chosen: manual, automatic or phased.

Android TV

- Built with `react-native-tvos` matching the SDK's React Native version and `@react-native-tvos/config-tv`, prebuilt with `EXPO_TV=1`.
- Manifest: `LEANBACK_LAUNCHER` category, `android.software.leanback` declared, touchscreen not required.
- 320 x 180 banner with the app name; 160 x 160 icon at xhdpi.

- Target API level 34 or higher.
- 64-bit and 16 KB page size support (from 1 August 2026).
- Every screen usable with the D-pad only: visible focus everywhere, initial focus on every screen, Back always works.
- Content inside safe margins; text readable from three meters.
- Tested on an Android TV emulator (API 31 or later) and on a real TV or streaming stick.
- Android TV form factor opted in on the Play Console, with TV screenshots and a TV banner.
- Subscriptions through Google Play Billing, checked on the server.

Appendix C: Troubleshooting

The problems below are the ones you're most likely to meet with this book's code and with Expo apps in general. For each, try the fix, then ask Claude with the exact error message and what you were doing.

Setting Up

`npm install` **fails with peer dependency errors in** `tv/`**.** The TV fork's version, `0.88.0-0`, doesn't satisfy ranges such as `>=0.78` in other packages. The `tv/.npmrc` file sets `legacy-peer-deps` for this reason; make sure you run `npm install` inside `tv/`, not from the root.

The app shows "Network request failed" or loads forever. The server isn't running, or the app can't reach it. Start it with `npm run server` and check `http://localhost:4000` in a browser. On an Android emulator, set `EXPO_PUBLIC_API_URL=http://10.0.2.2:4000`, because `localhost` inside the emulator means the emulator itself. On a physical phone, use your computer's local network address and make sure both are on the same Wi-Fi.

The demo login doesn't accept the code. Use phone 9876543210 and code 123456. If you changed the seed data, run `npm run seed` to reset the database.

Orders move too fast or too slowly to watch. Set `SIM_SPEED` when starting the server; 2 runs the simulation twice as fast, 0.5 half as fast.

Running the Apps

Metro says it can't find `@pro/ui` or `@pro/api`. Run `npm install` from the repository root so the workspace links the packages, then restart Metro with `npx expo start --clear`.

A change doesn't show up. Restart Metro with `--clear` to empty its cache. For changes to `app.json`, plugins or native libraries, you need a new development build; JavaScript reloading isn't enough.

"This library isn't available in Expo Go" or a native module is missing. You've added a library with native code. Build a development build (`npx expo run:android`, `npx expo run:ios`, or `eas build --profile development`) and use it instead of Expo Go.

Push notifications don't arrive. Expo Go doesn't support remote push from SDK 53 on: use a development build. Check permission was granted, the token reached your server, Firebase credentials are set for Android, and you're on a physical device or an emulator with Google Play services.

Text or layout looks different on Android and iOS. Fonts and line heights differ slightly. Use the design system's `Text` with its line heights rather than raw styles, and check both platforms. Shadows also differ; check them on both platforms.

Tests

A test passes alone and fails when everything runs. Look for timers and animations that outlive the test, shared state between tests, or assertions that don't wait. Chapter 18 shows a real example. Use the shared `testQueryClient()` and `findBy...` or `waitFor` for anything asynchronous.

Jest says it "did not exit one second after the test run". Something left a timer or connection open: often a query client with default cache timers or an event stream. Use the test query client and close streams in `afterEach`.

"Unable to find an element with role button and name …". The label changed or doesn't exist. Print the tree with `screen.debug()`; then fix the component's `accessibilityLabel`, which also fixes it for screen readers.

Builds and Stores

EAS build fails in a monorepo. Run EAS from the app's folder, make sure the lockfile is committed at the root, and read the build log's first error rather than the last. Expo's monorepo guide covers the settings.

"Version code has already been used" (Play) or "build number already exists" (App Store). Every upload needs a higher build number. Use `"appVersionSource": "remote"` and `"autoIncrement": true` in `eas.json`.

Play rejects the bundle for its target API level. Upgrade to an Expo SDK that targets the required level (API 36 from 31 August 2026; API 34 for TV) and rebuild.

App Review rejects with guideline 2.1. Usually a crash, a broken screen or a demo account that didn't work. Test the exact build and account the reviewer used, fix it, and explain the fix in your reply.

An EAS update doesn't reach users. Check the channel matches the build's channel, the runtime version matches, and the app has been restarted twice (once to download, once to apply, by default).

Android TV

Focus disappears or jumps somewhere odd. Every screen needs one element with initial focus. Wrap rows in focus groups so they remember their last item, and set a destination for groups that should be entered at a specific place, like the rail's current section.

The app doesn't appear on the TV's home screen. The manifest needs the `LEANBACK_LAUNCHER` category and a banner. Prebuild with `EXPO_TV=1` so the config plugin adds them, and check the generated `AndroidManifest.xml`.

Remote buttons don't reach your handler. Use the TV event handler from react-native-tvos for keys such as play/pause, and map them in one place (Chapter 30's `remoteKeys.ts`). Arrow keys normally move focus and don't need handling.

Appendix D: Glossary

10-foot interface. An interface designed to be used from across a room, as on a TV.

Accessibility label. The text a screen reader announces for an element, and the name tests use to find it.

ANR (App Not Responding). An Android error when an app's main thread is blocked for several seconds. Play's Android vitals report ANR rates.

App Bundle (`.aab`). The format Google Play requires; Play builds optimized APKs for each device from it.

App Links / universal links. Ordinary web addresses that open your app if it's installed, verified by files on your domain.

Claude Code. Anthropic's AI coding assistant that works in your terminal and repository, used to build every app in this book.

`CLAUDE.md`. A file in the repository that Claude Code reads at the start of every session: the project's layout, commands and rules.

Config plugin. A small module that changes the native Android and iOS projects during prebuild, configured in `app.json`.

D-pad. The four-way directional pad on a TV remote, with a select button in the middle.

Deep link. A link that opens an app on a specific screen.

Design system. A shared set of tokens (colors, type, spacing) and components that every screen uses, so all apps look consistent.

Development build. Your own debuggable app with your native libraries, used in place of Expo Go once you add native code.

EAS (Expo Application Services). Expo's cloud services: Build, Submit, Update and Workflows.

EAS Update. Over-the-air delivery of JavaScript and asset changes to installed apps.

Expo. A framework and set of tools on top of React Native.

Expo Go. An app that runs Expo projects without building them, limited to its own native libraries.

Expo Router. File-based navigation for Expo apps: each file in `app/` is a screen and a URL.

Feature flag. A server-controlled switch that turns a feature on or off without a release.

Focus (TV). The highlighted element that the select button will activate.

Focus group. A container that manages focus for its children, remembering the last focused one.

Hermes. The JavaScript engine React Native uses on phones.

Idempotent. An operation that has the same effect whether it runs once or many times; essential for payment webhooks.

Leanback. Android's name for the TV experience; TV apps declare the leanback feature and launcher category.

Live region. An element whose changes a screen reader announces automatically.

Maestro. A tool that runs end-to-end tests on a device or emulator from YAML flows.

Monorepo. One repository holding several apps and packages.

MRP (maximum retail price). The printed price in India, often shown struck through beside the selling price.

Optimistic update. Updating the screen as if a request succeeded, then rolling back if it fails.

Prebuild. Generating the native Android and iOS projects from `app.json` and config plugins.

Privacy manifest. A file Apple requires that declares data collection and the use of certain APIs, for the app and its SDKs.

React Native. A framework for building native apps with React and JavaScript or TypeScript.

React Query (TanStack Query). A library for fetching, caching and updating server data in React.

react-native-tvos. A community fork of React Native with Android TV and Apple TV support.

react-native-web. A library that renders React Native components in a browser.

Reanimated. A library that runs animations on the UI thread for smoothness.

Runtime version. A label that marks which builds an EAS update is compatible with.

Server-Sent Events (SSE). A simple way for a server to push a stream of events to a client over HTTP, used for live order tracking.

Staged rollout / phased release. Releasing an update to a growing share of users so problems can be caught early.

TestFlight. Apple's beta testing service.

Token (design). A named value such as a color role or spacing step.

Token (auth). A string that proves who the user is to the server.

UPI (Unified Payments Interface). India's real-time payment system, used through UPI apps.

Webhook. A message a service sends to your server when something happens, such as a payment succeeding.

Appendix E: Sources

Facts in this book about versions, store requirements and platform rules were checked in October 2026 against the sources below. Requirements change often; check the current version of each before you rely on it.

Topic	Fact	Source
Expo	Expo SDK 58 is current: `expo` 58.0.0 published 29 September 2026, 58.0.22 the latest patch at the time of writing	npm registry publish times for `expo`
Expo	SDK 58 pairs with React Native 0.88.1, React 19.3.0, react-native-web ~0.21.4, Expo Router ~58.0.29, Reanimated 4.7.1, Worklets 0.13.0, Gesture Handler ~3.3.0, Screens ~4.30.1 and FlashList 2.3.2	`bundledNativeModules.json` in the `expo` 58.0.22 package
Expo	From SDK 53, Expo Go doesn't support remote push notifications; use a development build. Android needs Firebase Cloud Messaging credentials	Expo documentation: Notifications reference and push notifications FAQ
TV	The TV fork matching SDK 58 is `react-native-tvos` 0.88.0-0	npm registry

Topic	Fact	Source
TV	Expo builds for TV with `react-native-tvos` as the `react-native` dependency and the `@react-native-tvos/config-tv` plugin, applied when `EXPO_TV=1` at prebuild; the TV version must match the SDK's React Native version; not every library supports TV	Expo documentation, "Building for TV" guide (expo/expo repository)
TV	The Android TV emulator needs an Android TV system image, API 31 or later	Same guide
Google Play	From 31 August 2026, new apps and updates must target Android 16 (API 36); Android TV apps API 34 or higher	developer.android.com, "Target API level requirements for Google Play apps"
Google Play	From 1 August 2026, TV apps must support 32- and 64-bit and 16 KB page sizes; launcher banner 320 x 180 with the app name; 160 x 160 xhdpi icon	developer.android.com, TV app quality guidelines
Google Play	TV manifest: `android.software.leanback`, touchscreen not required, `android:banner`, `LEANBACK_LAUNCHER`	developer.android.com, "Get started with TV apps"
Google Play	$25 one-time registration; new personal accounts (created after 13 November 2023) need a closed test with at least 12 testers opted in for 14 consecutive days before production	Play Console Help
Android	Developer verification: app registration required for stores including Google Play in Brazil, Indonesia, Singapore and Thailand from 30 September 2026; global rollout planned for 2027	Android Developers Blog (June 2026) and Android Developer Console Help

Topic	Fact	Source
Apple	Since 28 April 2026, uploads must be built with Xcode 26 and the iOS 26 SDK; since 9 September 2026, iOS apps must target iOS 13 or later; age ratings use the updated system since 31 January 2026	developer.apple.com, "Upcoming requirements"
Apple	Apple Developer Program $99 a year; TestFlight up to 100 internal and 10,000 external testers	Apple Developer documentation
Apple	App Review Guidelines sections cited: 1.2 (user-generated content), 2.1 (completeness), 2.3 (metadata), 3.1 (payments), 4.2 (minimum functionality), 4.8 (sign-in options), 5.1 (privacy, including account deletion)	developer.apple.com, App Review Guidelines
Payments	Stripe's React Native SDK provides PaymentSheet and the `usePlatformPay` hook for Apple Pay and Google Pay; Google Pay isn't supported in Expo Go	stripe.dev API reference for `@stripe/stripe-react-native`; Stripe documentation
Payments	Razorpay's React Native SDK needs a development build, not Expo Go	Expo forums and the `react-native-razorpay` repository
Payments	Stripe sign-up for new businesses in India is by invitation (since May 2024)	Stripe India documentation, checked for *Vibe Coding for Beginners* (2026)
Art	Microsoft Fluent Emoji 3D artwork, used for every illustration in the apps, is MIT-licensed (via `@lobehub/fluent-emoji-3d` 1.1.0)	github.com/microsoft/fluentui-emoji; npm registry
Tools	Claude Code version used for every session: 2.1.316	`claude --version`

What Was Verified, and What Wasn't

Verified for this book: every app and the TV app were built by recorded Claude Code sessions whose prompts are in Appendix A; `npm run check`

(typecheck, lint, more than 300 Jest tests and the server's Vitest suite) and the TV app's checks (80 tests) pass; every screenshot is a capture of the real web build in headless Chromium; the Android TV manifest was checked after `EXPO_TV=1 npx expo prebuild`.

Not verified here: the apps were not run on physical phones, emulators or a TV, because the build machine had no hardware acceleration and no Android SDK; no store submission was made; no payment provider, push service or analytics service was called. The production chapters describe what to do and what to check on your own devices and accounts.

About the Author

Aadhavan Muthurengan is an automation and artificial intelligence professional with more than 15 years of experience in the automotive industry.

Throughout a career focused on product development and validation, Aadhavan has designed and built a wide range of in-house tools that automate complex engineering workflows, speed up testing, and help teams deliver reliable, high-quality results. That work taught a lesson this book is built on: a tool is only as good as the care that goes into checking it. AI coding assistants make it possible to build far more, far faster, and the habits in these pages, plan first, test everything, look at every screen, and ship in small, safe steps, are what turn that speed into apps people can trust.

Thank you for reading *Vibe Coding Professional Mobile Apps*. May your apps be fast, your reviews five stars, and your crash reports empty.

More Books by the Author

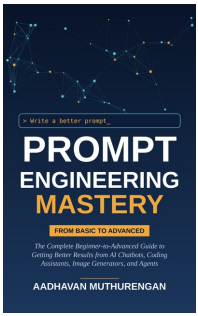
If you enjoyed this book, you may like these, also by Aadhavan Muthurengan. All are available on Amazon in paperback and Kindle editions.



Vibe Coding for Beginners: Build Real Apps with Claude Code, from Your First Prompt to Tested, Deployed Software. The natural first step before this book. Build real web and mobile apps with an AI coding assistant, from the first prompt to tested, deployed software, including payments and a full production project.



Building AI Agents with Claude: A Beginner's Guide to Agents That Use Tools, Remember, and Get Real Work Done, from Your First Agent to Production. Build AI agents that use tools, remember context and complete real tasks, from a first agent to production.



Prompt Engineering Mastery: The Complete Beginner-to-Advanced Guide to Getting Better Results from AI Chatbots, Coding Assistants, Image Generators, and Agents. Practical techniques for getting reliable, high-quality results from AI tools, from everyday chat to coding assistants and agents.



The System Design Interview Playbook: A Step-by-Step Framework, 20 Worked Designs, Tested Building Blocks and Mock Interviews to Prepare for Senior Engineering Interviews at Top Tech Companies. How large systems like the backends behind delivery, shopping and social apps are designed, with 20 worked designs.



Data Structures and Algorithms in Python, Hands-On: Every Structure Built, Tested and Timed, with Interview Patterns, Exercises and Solutions. Build every core data structure yourself, test it, benchmark it, and master the interview patterns.



Python Programming Hands-On: Learn Python from Basic to Advanced by Writing Real Programs, with Exercises, Solutions and Every Example Tested. Python from your first line of code to tested, packaged programs, a database app, a web API and automation scripts.